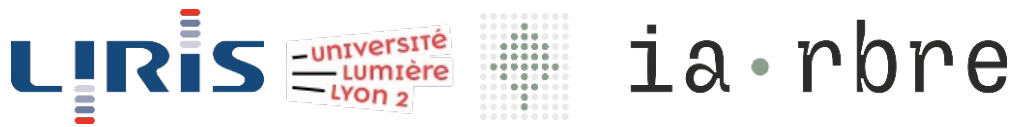
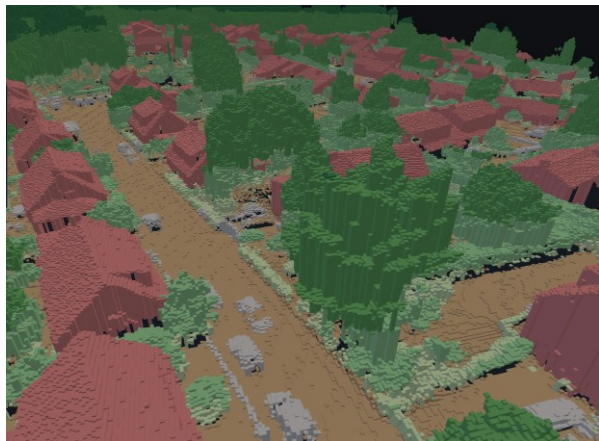


PART OF THE IA.RBRE PROJECT



# Master's Internship Report

*Voxelisation of Urban LiDAR Data: a Column-Compressed Semantic 3-D Representation for the IA.rbre Project*



*Author:*

**Nikolaos VYNIOS**  
**Master 2 Informatique - Gamagora**

*Laboratory supervisors:*

**John SAMUEL - Gilles GESQUIÈRE**

*Academic supervisor:*

**Didier PUZENAT**

**May 2026 - September 2026**

## Abstract

Cities adapting to climate change need to know where trees can grow, and that question is volumetric: crowns need open air, roots need soil, and both compete with buildings and infrastructure. The IA.rbre project is building a decision-support platform for urban greening in the Lyon Metropolis, and this internship supplies its volumetric layer. We present the design, implementation and validation of a voxeliser: a system that reads IGN's classified HD LiDAR tiles and represents the city as semantic voxels, organised into vertical columns independent of their neighbours, following the framework proposed by the supervising team. Each  $(x, y)$  position carries a vertical profile of material intervals (ground, vegetation, building), far more compact than the points it summarises. The structure is completed by a three-state empty-space decoder that distinguishes measured air from unmeasured building interior and from underground. Validation on 74 million points confirms the compactness of the representation, lossless for plan-view products; a majority-based simplification, tested, was rejected because measurement showed that it moves the plan-view vegetation footprint noticeably away from the raw footprint. Production runs range from a 751-million-point merged area to the entire Metropolis, 2,842 tiles and 51.9 billion points processed at three resolutions by a partitioned out-of-core pipeline, and the structure exports maps, statistics, in-browser 3-D viewers and 3D Tiles. A class-aware ray-casting layer treats vegetation as partially transmissive, and a first direct-sun-hours experiment measures what the volumetric model contributes over an opaque one.

# Table of Contents

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>Abstract.....</b>                                                      | <b>2</b>  |
| <b>1 Introduction.....</b>                                                | <b>4</b>  |
| 1.1 The LIRIS laboratory.....                                             | 4         |
| 1.2 The VCity project.....                                                | 4         |
| 1.3 The IA.rbre project: operation and funding.....                       | 4         |
| <b>2 Problem statement.....</b>                                           | <b>5</b>  |
| 2.1 The subject, objectives and design constraints.....                   | 5         |
| 2.2 Input data.....                                                       | 6         |
| 2.3 Output data.....                                                      | 6         |
| 2.4 Scope: initial brief and delivery.....                                | 7         |
| <b>3 State of the art.....</b>                                            | <b>9</b>  |
| 3.1 Voxel data structures.....                                            | 9         |
| 3.2 Processing methods on voxel grids.....                                | 10        |
| 3.3 Systems and tools.....                                                | 10        |
| <b>4 Proposal.....</b>                                                    | <b>13</b> |
| 4.1 Architecture.....                                                     | 13        |
| 4.2 Design.....                                                           | 15        |
| 4.3 Implementation.....                                                   | 19        |
| 4.4 Ray-casting queries: from occlusion to class-aware transmittance..... | 21        |
| <b>5 Results.....</b>                                                     | <b>25</b> |
| 5.1 Tests.....                                                            | 25        |
| 5.2 Validation.....                                                       | 26        |
| 5.3 Demonstration.....                                                    | 33        |
| 5.4 Discussion and limitations.....                                       | 38        |
| 5.5 Overall assessment.....                                               | 39        |
| <b>6 Conclusion and perspectives.....</b>                                 | <b>41</b> |
| 6.1 Against the original framing note.....                                | 41        |
| 6.2 Perspectives.....                                                     | 42        |
| 6.3 Acknowledgements.....                                                 | 43        |
| <b>Bibliography.....</b>                                                  | <b>44</b> |
| <b>Appendices.....</b>                                                    | <b>47</b> |
| Appendix A - Detailed algorithms and pseudo-code.....                     | 47        |
| Appendix B - CLI, module and configuration reference.....                 | 51        |
| Appendix C - Experiment data and serialisation format.....                | 53        |
| Appendix D - Supplementary figures.....                                   | 65        |
| Appendix E - IGN HD LiDAR classification codes.....                       | 69        |
| Appendix F - Glossary.....                                                | 70        |

# 1 Introduction

This report presents an internship carried out at the LIRIS laboratory within the IA.rbre project, on the design and implementation of a voxel representation of urban space from airborne LiDAR data. This introduction presents the host laboratory of the internship (1.1), the research project it belongs to (1.2), then the IA.rbre project and its funding (1.3).

## 1.1 The LIRIS laboratory

This internship took place at LIRIS (Laboratoire d'InfoRmatique en Image et Systèmes d'information), a computer-science laboratory in Lyon whose supervising institutions are the CNRS, INSA Lyon, Université Claude Bernard Lyon 1, Université Lumière Lyon 2 and the École Centrale de Lyon. The laboratory counts roughly 330 members across 12 teams, covering research areas ranging from computer graphics and geometry to data science, vision and simulation. During this internship I joined the Origami team, whose speciality is computer graphics, with complementary expertise in computer vision and algorithmics. The work reported here was carried out in that environment under the supervision of John Samuel and Prof. Gilles Gesquière, and with the guidance of Corentin Gautier and Arthur Villarroya-Palau.

## 1.2 The VCity project

Within LIRIS, the VCity project aggregates projects from several teams, mainly around geometric modelling and data science applied to the city: modelling it, analysing it and visualising it. The project builds on the growing availability of open urban data produced by local authorities, data that is at once three-dimensional, semantic and topological, and it contributes to the Open Geospatial Consortium standards that make such data interoperable. VCity's software ecosystem (among others the py3DTilers tooling and the pySunlight solar-analysis line [1], both detailed in chapter 3) is the immediate technical context of this internship: the voxel structure built here is designed to interoperate with those tools and to feed the same city-scale analyses.

## 1.3 The IA.rbre project: operation and funding

IA.rbre (Intelligence Artificielle pour l'arbre) is the project this internship takes place in. It co-develops a web platform for the processing and analysis of large-scale data, which helps cities decide where and how to plant trees, over the territory of the Lyon Metropolis and with the explicit goal of being replicable in other French cities [2]. The platform addresses two questions: where planting is feasible (underground networks, soil, existing infrastructure, building overhangs) and where it is most needed (sunlight, urban cooling, air quality, biodiversity, equity)?

The consortium is led by the TelesCoop cooperative, with the Lyon Metropolis (supported by Exo-dev) and Université Lumière Lyon 2 through LIRIS as partners. The project runs over 36 months (2025 to 2027) and is funded under the France 2030 programme by the Banque des Territoires under the DIAT call, "Démonstrateurs d'IA frugale au service de la transition écologique des territoires" [3]. Two of its stated principles concern this report directly: frugality (favouring, where possible, interpretable and resource-light methods over heavy learning) and open science (open data, free software, full reproducibility), which the design choices of chapter 4 answer.

## 2 Problem statement

### 2.1 The subject, objectives and design constraints

The subject, as phrased by the laboratory's framing note, reads: data structures for representing volumetric urban information as voxels. That single line rests on a concrete operational need of the IA.rbre project. Deciding where a tree can be planted requires reasoning about volume, not surface: the crown needs open air, the roots need soil, and both compete with buildings, roads and infrastructure. The project's existing modules (vegetation detection, desealing, sunlight) reason in 2.5 dimensions, over surfaces and rasters; the framing note asked this internship to add the volumetric dimension, from the standpoint of what a city's LiDAR survey can actually measure. The problem is therefore this: given the classified airborne LiDAR coverage of the Lyon Metropolis, build a representation of urban volume that is simultaneously semantic (each occupied cell carries a material class), compact enough for metropolitan areas to fit on an ordinary workstation, and queryable in the terms urban greening cares about, which are overwhelmingly vertical: what is the topmost material here? what is the canopy height? how much open air sits above this pavement? is this column penetrable?

From the outset, the supervisors proposed guidelines and design limits that channel the work towards its goal. First, no octrees: the framing note sets that avenue aside, the scientific interest of the project lying in an interoperable geospatial data structure rather than a spatial access hierarchy. The review in chapter 3 shows this family of structures remains active, with octree systems from 2023 and 2024 among the closest recent work. Second, the structure must be neighbourhood-indifferent, treating each  $(x, y)$  position as an independent vertical profile, which parallelises processing trivially and lets areas assemble by concatenation. Third, the vertical dimension must be compressed as runs of identical material; the supervisors sketched this encoding in a meeting before it was ever called run-length encoding, and chapter 4 formalises it. Fourth, the results must remain interoperable with the wider geospatial ecosystem the laboratory works in, which in practice means an export path to 3D Tiles and web viewers. Fifth, the implementation must be reproducible by non-specialists within the IA.rbre platform: Python, minimal dependencies, no build chain.

Following these constraints, the internship set itself five measurable objectives:

- O1, lossless voxelisation: isolated classified HD LiDAR tiles become a semantic column structure; no information may be discarded at construction.
- O2, compactness at known fidelity: the representation reduces the point data as far as fidelity allows; no step loses information as a side effect, and any lossy simplification is an explicit, measured choice.
- O3, scale: entire areas of the Metropolis process on a workstation with a few tens of gigabytes of memory, which rules out any step holding all the data at once.
- O4, analysis-readiness: the structure directly serves maps, statistics, three-dimensional inspection, standard geospatial exports and ray-casting queries.
- O5, a rigorous semantics of empty space: the structure distinguishes measured air from unmeasured interior and from the ground, because the correctness of downstream physical reasoning (shading, and ultimately plantability) is bounded by that distinction.

Chapter 5 evaluates each objective against the measurements.

Three assumptions narrow the scope. Semantic classification is not part of the problem: IGN delivers the point clouds already labelled, and re-solving classification would duplicate an industrial pipeline. Underground data is out of scope as an input: no subsurface dataset is consumed, and the representation treats what lies below the ground surface as a decoder state, leaving the integration of network and soil

data to future work. Finally, orthophotos are not an input: the pipeline can drape its class maps over an orthophoto for visual verification, but no computation reads the imagery.

## 2.2 Input data

The input is a single data source: the classified HD LiDAR tiles covering the Lyon Metropolis, produced under IGN's national programme [4] and distributed for the Metropolis through the Grand Lyon 2023 inventory [5]. Each tile is a LAZ file (the LASzip-compressed form [6] of the ASPRS LAS format [7]) covering 500 x 500 m on the ground, named by the coordinates of its south-west corner. The coordinate reference system, read from the file headers, is RGF93/CC46 (EPSG:3946), the conformal conic zone covering Lyon; the code's reader validates each tile's coordinate system on load and rejects any tile with a missing or foreign reference frame, a precaution born of early experiments on the 2018 acquisition (1 km tiles in Lambert-93, EPSG:2154). Vintage uniformity remains a property of the corpus a run is pointed at.

A tile contains about 16 million points at the median. A point in the delivered format carries its three coordinates, an intensity, a return number, a scan angle, a GPS timestamp and a classification code. Four of these values serve our problem: the three coordinates, which place the point in the grid, and the classification code, which carries the semantics; the rest feeds no targeted query and is not consumed.

The classification codes carry the input's semantics: across all 2,842 files, the codes actually present run from 1 to 9 (complete table and format details in Appendix E). Code 8 calls for clarification: in the Grand Lyon deliveries it is used as an upper canopy tier sitting above class 5, and it is substantial: 3.32 billion points, 6.4% of the corpus, present in 2,691 of the 2,842 tiles. The system names it and renders it as such. Two facts about this input shape the design: classification is already done, and the data arrives tile by tile (the Metropolis never arrives as a single file, so the representation must merge).

## 2.3 Output data

The output has one main product and a set of derived ones. The main product is the ColumnStore: the run-compressed column structure of chapter 4, presented either as a compressed archive for exchange or as a memory-mappable directory for pipeline use, at a cost of about 16 bytes per occupied column plus 13 bytes per interval. It is the single source from which every other product derives, and it is reversible: an inverse voxeliser can reconstruct a standard LAS/LAZ point cloud from it for audit by any external tool. The derived outputs serve three audiences: analysis, inspection and integration (Table 1). Figure 1 summarises the inputs and outputs, and Table 1 details them artefact by artefact; chapters 4 and 5 justify and evaluate them respectively.

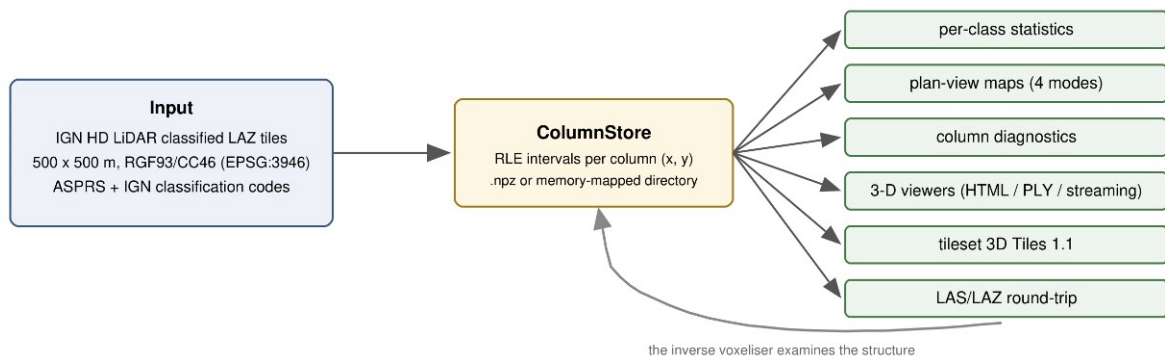


Figure 1: Inputs and outputs of the system: HD LiDAR tiles in, the ColumnStore at the centre, derived products out.

Table 1: Inputs and outputs of the system: each artefact, its role and its format.

| Artefact                          | Role                                                             | Format and detail                                                                                          |
|-----------------------------------|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Classified HD LiDAR tiles (input) | Single data source                                               | LAZ, 500 x 500 m, EPSG:3946; 2,842 tiles, ~16 M points per tile at the median; consumed: x, y, z and class |
| ColumnStore (main product)        | Run-compressed column structure, source of every derived product | Compressed archive or memory-mappable directory; ~16 bytes per occupied column + 13 per interval           |
| Statistics and diagnostics        | Analysis                                                         | Per-class statistics, per-column diagnostics                                                               |
| Plan-view maps                    | Inspection                                                       | Four modes per tile or area: maximum height, dominant class, class over orthophoto, interval count         |
| Three-dimensional viewers         | Inspection                                                       | Up to a city-scale streaming viewer                                                                        |
| Reconstructed point cloud         | Audit and interoperability                                       | Standard LAS/LAZ, via the inverse voxeliser, readable by any external tool                                 |
| 3D Tiles 1.1 tileset              | Geospatial integration                                           | Loadable by standard clients alongside IA.rbre's other layers [9]                                          |

## 2.4 Scope: initial brief and delivery

The five objectives of section 2.1 give measurable form to the five goals of the internship's framing note. This section states the correspondence, goal by goal: what each goal asked for, what this work delivers on it, and the choices made along the way with their reasons.

The framing note aimed to (i) design a voxelised data model covering vegetation, ground, buildings and underground structures; (ii) build that model from existing datasets including LiDAR, DSMs and CityGML; (iii) classify and categorise the voxels; (iv) experiment with shadow-casting or volumetric ray-casting algorithms accounting for interactions between voxels of different territorial classes; and (v) compare the results with existing 2-D and 2.5-D approaches, such as digital surface models and solar-radiation rasters, to assess the gains in accuracy, realism and interpretability.

All five goals were carried through. Three are the subject of chapters 4 and 5; shadow casting was carried to a measured area-scale experiment, and the comparison with 2-D and 2.5-D approaches to measured results over ten stratified square kilometres. One input choice and one measurement limit delimit the scope, and each decision is recorded here with its justification.

The input choice is a single data source: the system ingests airborne LiDAR only. DSM rasters and CityGML models are not consumed. The reason is evidential value: a uniformly classified source covering the whole Metropolis (2,842 tiles, 51,912,212,206 points, a single classification scheme) allows the representation and its encodings to be characterised rigorously over an entire territory. Adding heterogeneous sources would have multiplied the ingestion and reconciliation surface without deepening the question the work actually poses, namely how volumetric territorial information should be represented and compressed. Objective (ii) is therefore fulfilled on the most probative source, LiDAR; the integration of DSMs and CityGML is left to future work.

The measurement limit concerns the ground and the underground, which the available input cannot observe. Airborne LiDAR measures the first surface it hits; below the ground return it carries no information. The classification scheme present in the delivered tiles (codes 1 to 9) contains no soil horizon and no underground class: there is therefore nothing to represent in the input. The delivered model consequently covers vegetation, buildings, the ground surface and water. Representing soil horizons or buried structures would require an entirely different measurement modality – ground-penetrating radar, borehole records, soil surveys – so the obstacle is one of data acquisition. The voxel structure is not the limitation: `classes_config.py` already carries a code table sized for LAS 1.4 with room for such classes, and section 4.2 shows the decoder reasoning about the underground as a state. What is missing is the measurement.

Shadow casting (objective iv) was carried out. A three-dimensional DDA ray-traversal primitive over the column structure exists and is tested (`voxelizer/ray_trace.py`, section 4.4): it answers "what does a ray leaving this point in this direction encounter, and in what order" directly on the compressed representation, without expanding it. On that basis the experiment itself was run: solar positions computed for the summer solstice and validated against spherical astronomy, rays cast over two contrasting areas with class-aware occlusion where vegetation is partially transmissive and buildings opaque, and the result compared with the same computation performed with all materials opaque. Section 4.4.6 reports the design, the validation and the result.

The comparison with 2-D and 2.5-D approaches (objective v) was delivered. It is built so that the representation is the only variable: three models compared on the same LiDAR, answering the same queries through the same code, namely the volumetric column structure, a 2.5-D surface model that reduces each column to a single body between its lowest and highest returns, and a set of 2-D rasters reproducing the data model of the neighbouring greening module (section 3.3). A fourth model changes only the height of the sampling point, so as to separate that from the occlusion model. Over ten stratified square kilometres and at two grids, the surface model markedly overestimates canopy volume (footprint times height counts as canopy the whole column beneath a crown), cannot represent columns carrying two bodies separated by measured air, and overestimates direct sunlight wherever a canopy tops the ground; the cost runs the other way on one point, per-cell scalar reads remaining faster on the raster side. The figures are reported in chapter 5 (section 5.2.2), the design and checks in section 4.4.7, the tables in Appendix C. A comparison with an external product of independent provenance remains a later step (discussion in 6.1).

"2.5-D" names two different things, a data model and a storage layout, and only the first is the limitation this comparison addresses; section 4.4.1 states the distinction in full and applies it to this structure. The measured divergence above sits where that distinction places it: under the canopy, under overhangs and in the vertical structure of a street, where the digital surface model has no information to offer.

Objective (iii) - the classification and categorisation of the voxels - is achieved in full; objective (i) - the voxelised model itself - is carried through in full on everything the input measures, the ground and underground part falling under the LiDAR limit above. Both are the subject of chapters 4 and 5.

### 3 State of the art

Voxelisation, that is, the conversion of geometric data into a regular grid of volume elements, is a mature field whose design space is well worked. The survey by Aleksandrov *et al.* [10] organises it along two axes: the algorithm used to decide which voxels a primitive occupies (integer division for points, 3-D Bresenham for segments, conservative or 6-separating tests for triangles, parity tests for solids), and the data structure used to hold the result. For our problem the algorithmic axis is essentially settled: classified airborne LiDAR arrives as points, and points voxelise by quantisation, an integer division of each coordinate by the cell size. The open question, the one this internship had to answer, is this: what structure should carry a semantically labelled model of a Metropolis, and what structure best answers the vertical questions urban greening asks of it?

This chapter reviews voxel data structures (3.1), the processing methods built on them (3.2) and existing systems (3.3), each review stating its limits for our use, and concludes on the need those limits leave open.

#### 3.1 Voxel data structures

The naive starting point is the dense grid as a single array `grid[ix][iy][iz]`: constant-time access and a trivial implementation, but a prohibitive memory cost. A 500 m tile (median around 16 million points) at 1 m horizontal and 0.5 m vertical resolution covers on the order of fifty million cells (detail in Appendix C), the overwhelming majority of which is air; at 0.5 m cubed the count grows by several more factors. Even at one byte per cell this exceeds ordinary RAM as soon as a few tiles are merged, and nearly every byte encodes emptiness. Cutting space into chunks spreads that memory without reducing it: sparsity has to come from the structure itself. The structures reviewed below exist precisely to avoid this cost.

The classic solution is the octree, which recursively subdivides space only where there is content; it can also be no more than a simple decomposition of space whose smallest element is the voxel. Octree-structured voxel spaces have carried a large share of the LiDAR-processing literature. Wang and Tseng segment airborne point clouds incrementally, directly in such a space [11]. The voxel integration engine of Gorte *et al.* stores a five-level octree pyramid in SQLite, with four generic operators serving as a complete query API over terrain, buildings, roads and vegetation [12], and a related line of work extends the same framework to the analysis of very large voxel datasets [13].

Sparse-voxel DAGs (directed acyclic graphs) [14] compress further by sharing identical subtrees, in the proportions Table 2 reports.

These structures are powerful, but two considerations weighed against them here. They optimise spatial uniformity (the same subdivision logic on all three axes) whereas our queries are manifestly anisotropic: plantability, canopy structure and shading all interrogate the vertical column above or below an  $(x, y)$  position. None of these structures is natively organised in columns: a vertical question must reassemble its answer from scattered nodes. In addition, a framing constraint applies: the supervisors excluded octrees from the outset, the framing note seeking a different structural contribution (Table 2).

OpenVDB [15], the de facto standard for visual effects, stacks a fixed-depth hierarchy of dense nodes under a hash table, with tile compression of constant regions and DDA ray traversal; NanoVDB extends it to GPUs and out-of-core processing [16]. For a 500 m tile at 0.5 m it encodes occupancy in a few tens of megabytes where the dense grid demands hundreds. As Table 2 notes, its ecosystem is C++ and its value type is per voxel rather than per range: a column query assembles voxel by voxel.

Simpler flat schemes also exist: cutting the dense grid into fixed-size bricks and hashing the occupied ones parallelises trivially and maps directly onto block storage formats, and a  $16^3$  brick, 4,096 cells,

weighs a few kilobytes against a typical 32 KB L1 data cache. Bricks stay dense: a single occupied voxel still allocates the other 4,095.

At the other extreme, implicit representations store no grid at all: signed distance fields answer "how far is the nearest surface" from the function itself, in closed form for the parametric primitives Hart derives [17], and through an exact Euclidean distance transform of the occupancy image for sampled data [18], which makes exclusion-zone queries exact but per-class semantics awkward. Each of these options was evaluated during the design phase, and Table 2 summarises that evaluation; section 4.2.1 then argues the chosen structure against the project's best internal alternative, without reopening the review.

Closest to our encoding in spirit is the compression literature. Tzamarias *et al.* [19] compress the geometry of voxelised point clouds by organising the occupied voxels into runs and coding them in a single pass directly in the voxel domain, explicitly to avoid octrees and dense occupancy matrices. This confirms, from an unrelated community, that the run-based structure exploits point-cloud sparsity directly. Run-length encoding is itself among the field's oldest results [20], and the observation that column storage suits column queries is the founding argument of column-oriented database systems [21]. What the compression work does not provide is an analysis-ready structure: their runs encode binary occupancy meant for transmission, not per-run semantic labels, point densities and query operators.

## 3.2 Processing methods on voxel grids

A second body of work uses voxels not as an end product but as a unit of computation, and it matters to us mostly for what it implies about inputs.

Aljumaily *et al.* [22] classify urban airborne LiDAR of Dublin, roughly four times the density of our tiles, by random forest on per-voxel descriptors, cutting the number of elements to process by 99% against point-by-point processing; Poux and Billen [23] separate flat roofs from canopies without supervision, by geometry alone. Table 2 gives their inputs, outputs and scores.

In France, this classification problem has in effect been industrialised: IGN's HD LiDAR programme distributes point clouds in which every point already carries its class, assigned by the RandLA-Net-based myria3d pipeline [24] and validated on datasets such as FRACTAL [25] (100,000 annotated clouds, 250 km<sup>2</sup>, seven classes) [4]. This fact governs our design: classification is already in the input. The internship reorganises existing classifications without producing new ones, turning tens of millions of labelled points per tile into a structure that answers urban-analysis queries losslessly and efficiently, at metropolitan scale.

## 3.3 Systems and tools

Three systems frame our work most directly. LiVoxGen [26] is its direct ancestor: a C++ tool voxelising LiDAR into per-column voxel lists to derive canopy metrics (cover height and density). We kept its central idea, the column as the unit of organisation, and rejected the implementation wholesale, whose kd-tree solves in  $O(n \log n)$  a grid assignment that is  $O(n)$  integer arithmetic; Table 2 recaps its other obstacles.

Arches, developed in this laboratory, is its conceptual ancestor: Peytavie *et al.* model terrains as a 2-D grid of vertical stacks of segments (material, thickness), which natively represent the overhangs and cavities no height field can express [27]. Our system is its measurement-side counterpart, deriving the same stacked-column model from real LiDAR observations.

Finally, the VoxCity framework [28] builds semantic 3-D city mock-ups from freely available worldwide rasters, then runs sunlight and visibility simulations on them; it shares our output ambitions

but consumes derived 2.5-D rasters rather than raw classified LiDAR, with correspondingly coarser vertical fidelity (Table 2).

Within the VCity ecosystem itself, the two tools split the work: py3DTilers assembles 3D Tiles tilesets (the open OGC [9] format that streams large 3-D models as hierarchical tiles) from geometric formats such as OBJ, GeoJSON, IFC and CityGML, and pySunlight computes sunlight over those tilesets, reading them through py3DTilers and delegating the light computation to the C++ Sunlight library [1]. Both work on meshes, city representations rather than volumetric structures, and Jaillot *et al.* describe a generic approach for computing the impact of sunlight and shading on such large city models [29].

Within IA.rbre itself, a neighbouring module produces the 2-D and 2.5-D product line. It derives 1 m GeoTIFFs from the same Grand Lyon LiDAR and the corresponding orthophotography (a terrain model, a surface model and their difference) and merges in a pixel-by-pixel land-cover segmentation produced by the FLAIR-HUB model [30], ending in a three-class vegetation raster indexed on height above terrain. This is the concrete form of the 2.5-D data model that section 4.4.7 confronts this structure with: one value per cell, and no way to ask what exists underneath.

At the end of this review, no method surveyed combines the five properties our problem statement requires: native integration of the IGN HD LiDAR semantics; a neighbourhood-indifferent column organisation matched to vertical queries (a framing constraint, motivated in chapter 4); run-based compactness with class labels and per-run point counts; out-of-core operation so that metropolitan areas can be processed on an ordinary workstation; and a pure-Python, low-dependency implementation for reproducibility within the IA.rbre platform. Table 2 below condenses this reading method by method. The proposal of chapter 4, a run-compressed column structure with a three-state empty-space decoder and an out-of-core partitioning chain, is designed to answer this need.

| Method                                                   | Data structure                                                                          | Inputs                                          | Outputs                                                                                           | Remarks                                                                                                                                                                        |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dense grid, including cache-friendly brick variants [10] | 3-D array, possibly cut into dense bricks                                               | any voxelisable data                            | O(1)-addressable grid                                                                             | Simple and parallel; $\sim 5 \times 10^7$ cells per 500 m tile, mostly air; bricking helps locality but keeps the air inside occupied bricks; infeasible at metropolitan scale |
| Octree segmentation [11]                                 | octree-structured voxel space                                                           | raw LiDAR points                                | coplanar segments (roofs, walls, ground)                                                          | Mature; isotropic subdivision ill-suited to vertical queries; octrees excluded by the framing note                                                                             |
| Voxel integration engine [12], [13]                      | SQLite-backed octree, 5-level pyramid                                                   | DEM, LOD2 solids, roads, vegetation             | unified semantic voxel scene + 4 operators                                                        | Closest recent system; database dependency; octree core                                                                                                                        |
| Sparse-voxel DAG [14]                                    | DAG-deduplicated octree                                                                 | binary occupancy                                | compressed static scene                                                                           | 28 to 576 times fewer nodes than the same-resolution octree on the five tested scenes; binary only, no semantics                                                               |
| OpenVDB / NanoVDB [15], [16]                             | fixed-depth hierarchy hash-topped                                                       | volumetric values                               | O(1) access, DDA traversal, out-of-core                                                           | Industry standard; C++ ecosystem; per-voxel values, no per-run semantics                                                                                                       |
| Signed distance field [17], [18]                         | closed-form distance function or sampled distance grid                                  | occupancy or primitives                         | distance to the nearest surface                                                                   | Ideal for exclusion zones; per-class semantics requires one field per class                                                                                                    |
| RLE point-cloud compression [19]                         | runs in the voxel domain                                                                | voxelised point clouds                          | losslessly compressed geometry                                                                    | Validates the run-based structure; transmission-oriented, no semantics and no query API                                                                                        |
| Per-voxel RF classification [22]                         | regular grid + 7 descriptors/voxel                                                      | raw airborne LiDAR (297 M pts/km <sup>2</sup> ) | ground/roof/wall/vegetation labels, F1 $\approx$ 0.92                                             | Classification solved upstream for us by IGN (myria3d [24] / FRACTAL [25])                                                                                                     |
| Unsupervised per-voxel segmentation [23]                 | octree-derived voxel grid                                                               | XYZ point clouds                                | semantic labels via shape descriptors                                                             | Same conclusion: geometry separates canopy from buildings                                                                                                                      |
| LiVoxGen [26]                                            | per-column voxel lists (C++)                                                            | LAS via liblas                                  | canopy metrics                                                                                    | Direct ancestor; abandoned dependencies, oversized kd-tree, C++ build chain                                                                                                    |
| Arches / SBRT [27]                                       | 2-D grid of (material, thickness) stacks                                                | procedural rules                                | terrains with overhangs, cavities                                                                 | Laboratory heritage; generates what we measure                                                                                                                                 |
| VoxCity [28]                                             | uniform semantic voxel grid                                                             | open 2.5-D rasters (heights, land cover)        | city model + solar/visibility simulation                                                          | Shared goals; raster-derived, vertical fidelity coarser than LiDAR                                                                                                             |
| <b>Our approach</b>                                      | <b>sparse column structure; per-column RLE intervals in a numpy structure-of-arrays</b> | <b>IGN HD LiDAR classified LAZ (RGF93/CC46)</b> | <b>queryable ColumnStore; PNG maps; statistics; 3-D viewers; 3D Tiles 1.1; LAS/LAZ round trip</b> | <b>neighbourhood-indifferent; preserves semantics; out-of-core at metropolitan scale; pure Python</b>                                                                          |

Table 2: State-of-the-art summary. The last row places our approach in the same columns.

## 4 Proposal

Before the mechanisms, Figure 2 shows on a real six-metre transect what the proposal produces and what it can answer. Detailed algorithms and pseudo-code are gathered in Appendix A and referenced from the text.

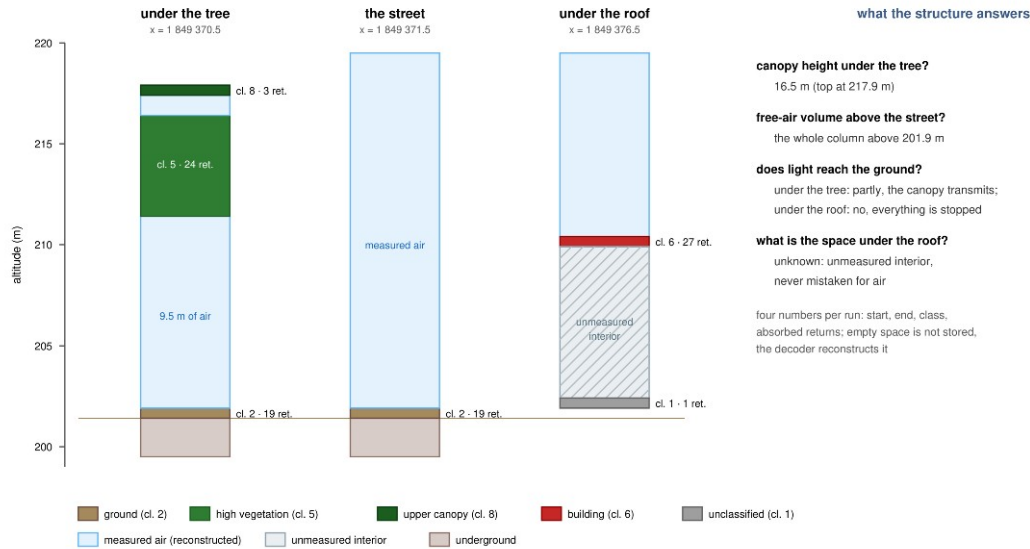


Figure 2: Three real columns of a 6 m transect (Porte des Alpes campus, 0.5 m cells,  $y = 5,170,722$ ): under the tree, the crown (classes 5 and 8) sits above 9.5 m of measured air over the ground; the street is a single run of ground; under the roof (class 6), the space with no ground return is an unmeasured interior. On the right, the structure's expected answers.

### 4.1 Architecture

The system is organised as a single Python package, voxelizer, whose modules group into five dependency levels, from the base of low-level services up to the top level of CLI and GUI consumers.

At load time, no module imports a module above it: the hierarchy is acyclic. Three late imports, executed only when a function is called, climb the hierarchy; visible in Figure 23, they take no part in loading. Import chains run to as many as ten dependency links spread over eleven levels (level 0 to level 10), and Figure 3 summarises the five architecture levels with the modules that sit in each; the full import graph, generated from the code, is Figure 23 in Appendix D.

This level organisation offers three concrete benefits. First, testability: each level is exercised against the lower levels only. Second, failure containment: each output stage (statistics, maps, diagnostics, 3-D export) depends only on the persisted structure and runs in its own process, so a rendering failure never costs the voxelisation that preceded it. Third, reuse: the same core serves five entry points (single-tile CLI, area CLI, graphical interface, 3-D visualisation, 3D Tiles export) without duplication. Each stage is thus isolated and separately replayable, both in a local environment and in the project's Docker image, which freezes the execution environment and makes results reproducible from one machine to another.



Figure 3: The five dependency levels of the voxelizer package and the modules in each, generated from the code's import graph.  
The full import graph is Figure 23 in Appendix D.

The data flow (Figure 4) is linear, from reading the LAZ tile to the ColumnStore populated with intervals. Everything downstream consumes only the structure, the single interface between measurement and analysis. It can be made persistent (compressed .npz, or raw memory-mappable directory) and reloaded without re-voxelising, so a failed or improved output stage never requires touching the input data.

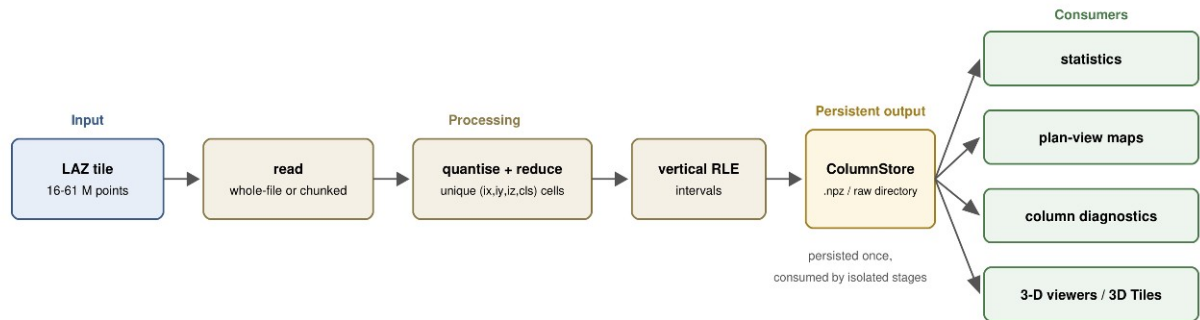


Figure 4: Data flow of the pipeline: from the LAZ tile, through quantisation and vertical RLE, to the persisted ColumnStore and its consumers.

Two transversal devices complete the architecture: an upfront estimator, which reads only an area's LAZ headers and predicts the structure size and peak memory before a single point is read (section 4.2.5 returns to it), and a run-diagnostics monitor that records system traces of any run in the background, invaluable during chapter 5's memory investigations. Reproducibility is handled at the outermost layer: auto-incremented run directories, wrapper scripts and a Docker image freezing the Python environment.

## 4.2 Design

### 4.2.1 The column structure and its intervals

Two of the framing constraints of section 2.1 govern the structure: neighbourhood-indifference, each  $(x, y)$  position analysable without its horizontal neighbours, and vertical compression as runs of identical material. The internship's contribution was to formalise, implement and validate that framework. The result is a sparse two-dimensional grid of vertical profiles: for each occupied column  $(ix, iy)$ , an ordered list of intervals  $(z\_start, z\_end, class, count)$ , where  $z\_end$  is exclusive, class is IGN's HD LiDAR semantic label, and count is the number of original LiDAR returns the interval absorbs. A thirty-metre tree becomes roughly two intervals, ground at the bottom and one long run of high vegetation above, instead of thousands of points (Figure 5); run-length encoding exploits the vertical continuity urban matter exhibits.

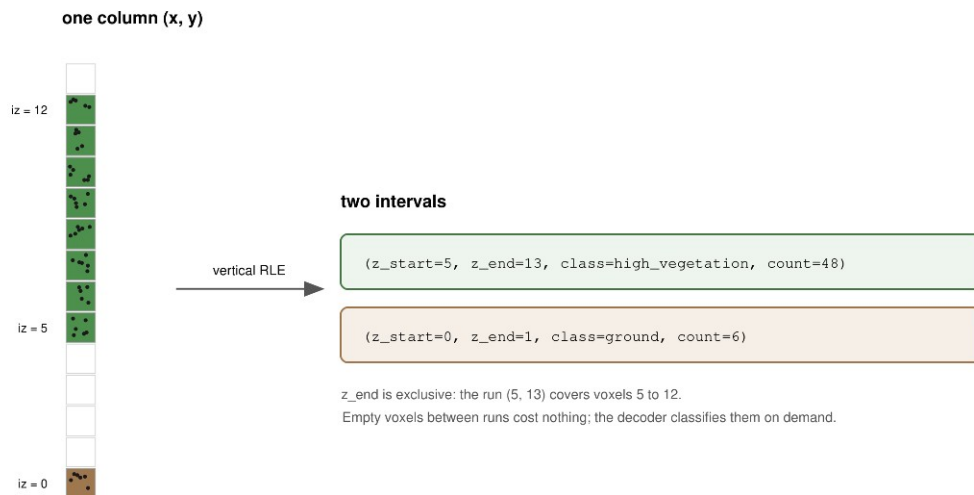


Figure 5: From points to intervals: a column and its run-length encoding ( $z\_end$  exclusive).

Two properties of this encoding deserve note. First, construction is conservative: if a voxel receives returns of two classes, both survive as superposed cells. The voxeliser records, it does not vote. Any lossy simplification (smoothing, gap filling, majority reduction) is deferred to explicit, separately validated post-processes (section 4.2.4), so the raw structure preserves the sensor data's information. Second, the encoding is neighbourhood-indifferent by construction: nothing in an interval refers to an adjacent column, which makes downstream computation trivially parallel, lets tiles merge by concatenation and makes the *out-of-core* mode of section 4.2.5 possible. The cost, horizontal relations recomputed on demand, is acceptable here: the project's queries are overwhelmingly vertical.

The compression obtained is measured in chapter 5: on the corpus's densest tile, the structure reduces tens of millions of points to a few million intervals, and Appendix C gives the full accounting, class by

class and grid by grid. The mean number of intervals per column is itself a diagnostic: vertical surfaces, walls and trunks in particular, fragment columns into short runs, which chapter 5's encoder experiments quantify.

The way these intervals are held in memory was rethought once: the first implementation, a dictionary of Python objects true to the language's idioms, proved ruinous at scale. The production layout is a flat numpy structure-of-arrays costing about 16 bytes per column plus 13 bytes per interval, a measured order-of-magnitude gain over the object form. The measurement that settled this choice, and the wrong estimate it corrected, are recorded in Appendix C.

Column-oriented databases arrive at the same layout for the same reason [21]: when every query touches one field of many records, arrays beat objects. This layout also makes persistence trivial (each array is a file in the raw-directory form, memory-mappable for the isolated-stage pipeline) and keeps every algorithm in vectorised numpy, in NumPy's sense: whole-array operations, with no per-point or per-column Python loop on the construction path. The query layer of section 4.4 is the exception: the ray walker, the algorithm that advances along a ray voxel by voxel, runs in Python, and section 5.4 records what that volume cost.

### 4.2.2 The voxelisation algorithm

The algorithm that fills the structure runs in two vectorised steps, that is, in whole-array operations. Step one quantises and reduces: each point's coordinates are divided by the cell sizes with integer division (both sizes are parameters; every table in this report states the grid it was measured at) to give integer indices, the resulting (ix, iy, iz, class) tuples are sorted canonically, and duplicates reduce to unique cells whose counts are summed. Step two performs the run-length encoding: within each column, consecutive cells of increasing iz and the same class merge into an interval; an interval boundary occurs exactly when the column changes, when the class changes, or when iz jumps. Both steps are pure numpy (sort, difference, cumulative sum), so the per-tile cost is dominated by reading and decompressing the LAZ itself.

Two implementation decisions shape behaviour at scale: the grid origin is fixed once per area, so all tiles quantise against the same origin and concatenate without resampling, and the read exists in a memory-bounded chunked variant, verified to produce structures identical to the whole-file path (chapter 5's test suite); the detail is in Appendix A.

### 4.2.3 Empty-space semantics: the ground index and the three-state decoder

A voxel structure derived from airborne LiDAR contains three kinds of "empty", and confusing them corrupts downstream physical computation. Figure 6 distinguishes them: measured air, which a laser beam actually crossed; unmeasured interior, typically under a roof that stopped every beam, which if treated as air would let simulated light pass through buildings; and underground, the solid earth inferred below the surface, which will later serve the rooting volume.

The decoder assigns one of these three states to every unoccupied voxel, anchored by a single per-column value: the ground index, the top of the highest class-2 interval, lower intervals being classification errors or a sloping surface spread over two voxels; the bridge case does not arise in this delivery (Appendix E). Columns with no ground return default to opaque interior, the "unmeasured interior" of Figure 2, an unknown zone not being treated as air, and an optional interpolation restores a terrain estimate under roofs. The edge cases and the precise rules are in Appendix A.

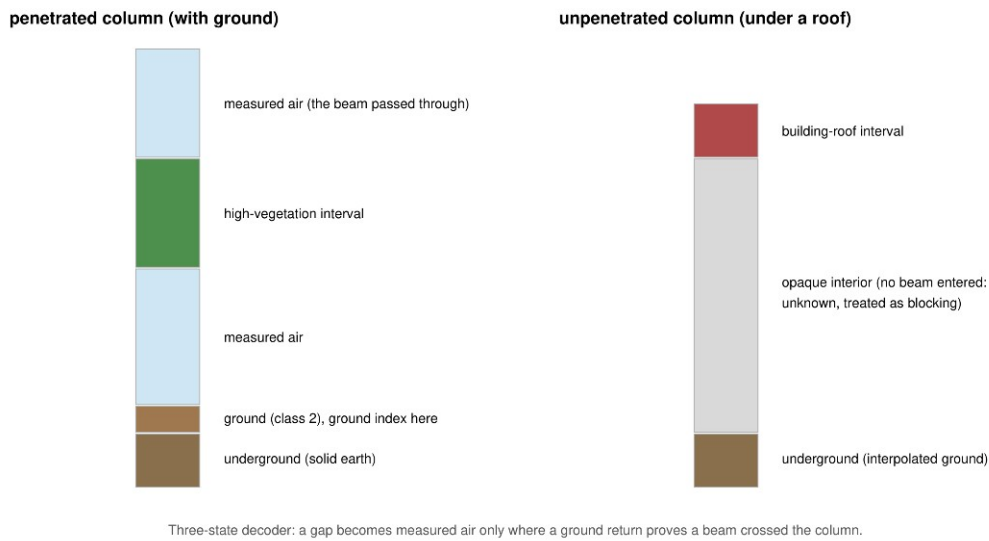


Figure 6: The three-state empty-space decoder, anchored by each column's ground index.

#### 4.2.4 Semantic refinement: encoder variants and post-processes

The raw structure is complete but noisy: isolated misclassified voxels, one-voxel holes in canopies, columns shredded by wall returns. Refinement is treated as a family of explicit transformations, each embodying a hypothesis about which information matters, each measured against the raw baseline in chapter 5: variant v1 (class smooth) smooths isolated grain with a majority window, v2 (gap fill) closes short gaps between same-class intervals, lossless for plan-view products, v3 (minor class drop) removes marginal intervals, and v4 (majority above five metres) reduces everything above five metres to the band's dominant class. Figure 7 shows what each variant rewrites in a column; the exact mechanics are in Appendix A.

v4 is the literal formalisation of a supervision-meeting suggestion ("thirty building points and five vegetation points: let's call that building"), included so its cost could be measured; chapter 5 measures that cost, substantial on the plan-view vegetation footprint, which is why v4 is reserved for visualisation and excluded from ecological analysis.

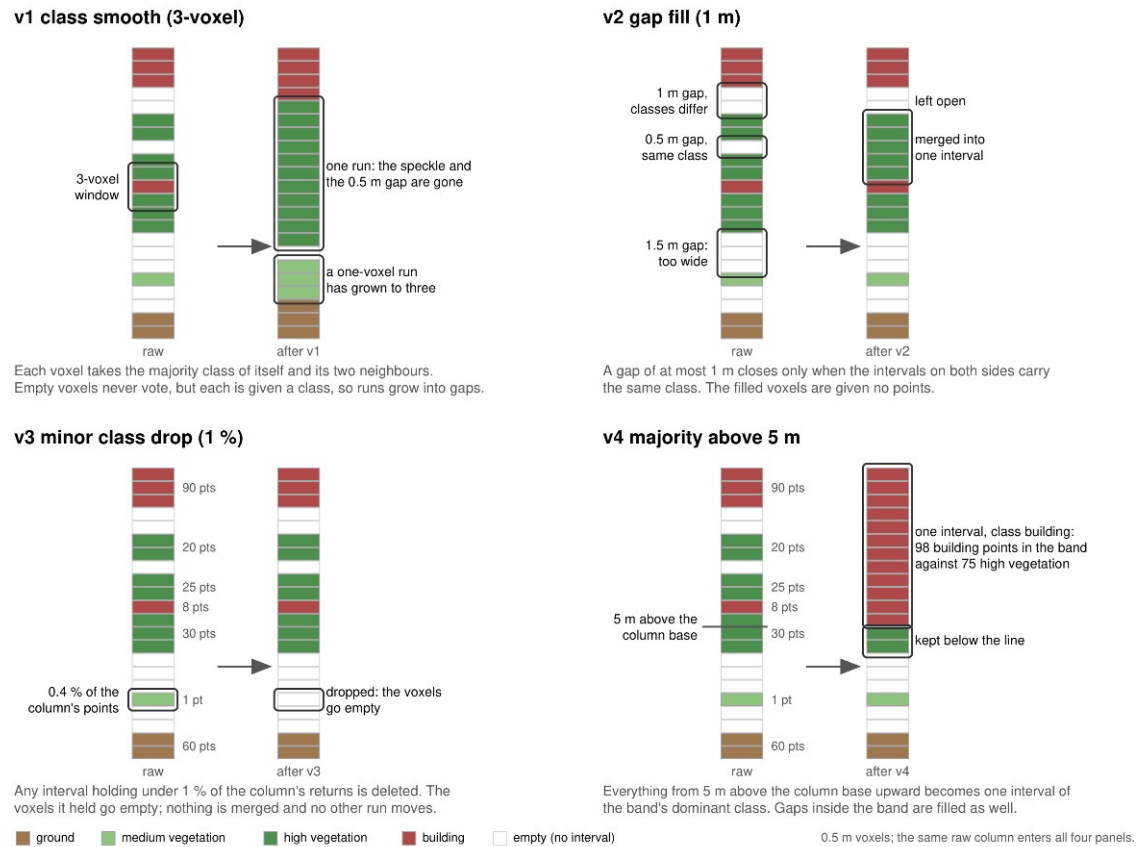


Figure 7: The four simplification variants of section 4.2.4: what each rewrites in a column, shown on the same interval stack in 0.5 m voxels. Dashed outlines mark what triggers the rule, solid outlines what it changed.

Three standalone post-processes complete these variants (a vote resolving overlaps, an absorption of vegetation enclosed in building, a denoise): batch operations that temporarily consult neighbourhoods and then return a structure that is again neighbourhood-indifferent; their rules are in Appendix A and their measured costs in chapter 5 and Appendix C. Another idea from the design notes, per-class surface-top rasters, was considered and set aside: a prettier product, nothing for the plantability objective.

#### 4.2.5 Designing for scale: metropolitan areas on commodity memory

The last design axis is accounting for scale, and its guiding principle is that no step may ever require the whole city in memory.

Area processing selects the tiles intersecting the requested box by reading headers only, voxelises each against the common grid origin and merges the structures by concatenation, cheap precisely because columns are independent. When the merged structure itself would exceed memory, the partitioned mode processes each tile to an independent fragment and aggregates in a streaming fashion: peak memory stays that of a single tile whatever the area. Full merging remains possible at that scale, because it is itself out-of-core, assembled on disk one key band at a time, with a result proven identical to the in-memory merge (section 5.1); a streaming variant, finally, downloads, voxelises and deletes each tile for machines short on disk space.

The upfront estimator of section 4.1 makes these modes usable by non-specialists: from headers alone, it predicts whether the requested run fits, keeping the more cautious of two estimates (covered area,

header point counts), and proposes the coarsen-or-partition alternatives with concrete numbers before any work begins. Its constants are calibrated on measurement and validated on cases held out of the fit, the estimate staying cautious without ever underestimating; the detailed calibration is in Appendix C, and chapter 5's post-mortem shows what was redesigned when a first visualisation path violated this section's guiding principle (Figure 8).

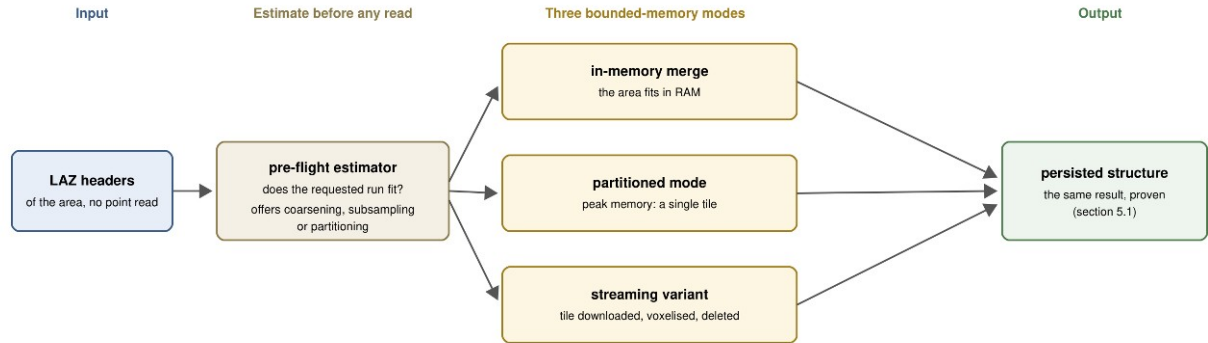


Figure 8: Scaling in one schema: the upfront estimator reads only the headers and chooses, with concrete numbers, among three bounded-memory execution modes; all reach the same persisted structure.

## 4.3 Implementation

### 4.3.1 Language and dependencies

The implementation language was itself an argued decision. The predecessor tool was in C++, and a rewrite in C++ would have been the reflex choice for a performance-sensitive voxeliser.

We chose Python only for three reasons. First, the critical path is not Python: every per-point operation executes inside numpy's compiled kernels [31], and profiling shows the per-tile cost is dominated by LAZ decompression, so a C++ port would contribute little where it counts. Second, reproducibility: the IA.rbre platform favours pipelines that colleagues can install with pip and read without a build system; the dependency footprint is a short pip-installable list (numpy, laspy [32] with the LASzip engine [6], matplotlib, Pillow, psutil, pyproj, pyglflib, plus two optional extras), each of whose roles Appendix B details. Third, continuity: every downstream IA.rbre component likely to consume the structure is in Python.

The one place this choice costs us, per-element loops that resist vectorisation, was removed by design rather than by optimisation: the algorithms of section 4.2 are formulated as array transformations precisely so that no such loop exists.

### 4.3.2 Structure internals and persistence

The structure-of-arrays of section 4.2.1 is realised by six flat numpy vectors (the internals, key packing and quantisation constants, are detailed in Appendix B). Persistence has two forms with distinct roles: a compressed .npz archive for exchange, and a raw directory, one file per array, for the pipeline itself. The raw form exists because of stage isolation: child processes memory-map the arrays read-only, run without copying, and a failing stage leaves the structure intact on disk.

### 4.3.3 Output stages

The per-tile pipeline emits four plan-view PNG maps, one per frequent question: maximum height (the urban silhouette), dominant class, class over optional orthophoto (display only, never a computation input), and intervals per column (a fragmentation map, a data-quality diagnostic). A statistics stage writes per-class point and interval counts with compression ratios. A column-diagnostics stage renders the structure's most complex columns as vertical sections, a histogram of intervals per column, and per-class panels; these figures were the main debugging instrument throughout, because a wrong algorithm shows up immediately as an implausible column profile. Area runs produce the same deliverables assembled across tiles, under the bounded-memory regime of section 4.2.5.

### 4.3.4 Three-dimensional visualisation

Visualisation was developed as a control instrument: seeing the structure in three dimensions is the most direct way to convince oneself that a result is plausible before measuring it. Three complementary paths provide it: a standalone HTML viewer that opens in a browser with no installation, PLY and PyVista exports for standard inspection tools, and a streaming viewer that streams the structure tile by tile around the camera under a fixed GPU budget, a fine-resolution area carrying hundreds of millions of intervals no browser loads as a single scene (Figure 16, in chapter 5, shows it in navigation). A fourth exporter converts the structure into a 3D Tiles 1.1 tileset [9] for standard geospatial clients; the output passes the official validator with no errors, under the caveats recorded in section 5.5. The internal mechanisms of the four paths are described in Appendix B, and Figures 18 to 20, in Appendix D, give their diagrams.

### 4.3.5 Query mechanism: the ray caster

The analysis-side aspect implemented on top of the structure is a ray walker in the manner of Amanatides and Woo [34]: a three-dimensional digital differential analyser that maintains the parametric distance to the next voxel boundary on each axis and advances the nearest one, traversing the structure voxel by voxel along the ray's direction. The decoder of section 4.2.3 supplies the semantics of the empty voxels it crosses, and a configuration object declares which states count as solid. The same walker therefore serves shadow casting (opaque interior and occupied voxels block), visibility (only occupied voxels block) and the future plantability queries. The walker returns the first solid intersection with its position, its class and the ray parameter. It is kept simple because it is the reference implementation: section 4.4 builds the shadow-casting machinery on it – an exact early-termination bound, class-aware transmittance and solar geometry – and tests every optimisation against its results.

### 4.3.6 Interfaces and reproducibility

The system exposes three interface levels, reflecting three audiences. For colleagues new to the technical side, a Tkinter interface offers a single-file mode and a coordinate-based area mode, preceded by the upfront dialogue of section 4.1 (predicted tile count and memory cost; continue, subsample or partition). For the user who scripts their processing, the CLI exposes the single-tile, area, visualisation, tileset and reconstruction entry points, with options factored so the tools cannot drift apart; the same pipeline runs on Windows, on Linux and in the project's Docker image.

For the auditor equipped with standard point-cloud tools, an inverse voxeliser reconstructs a LAS/LAZ point cloud from a structure, either one point per original return using the stored counts or one point per voxel, which lets any standard point-cloud tool examine what the structure actually contains and closes the loop for validation (chapter 5 uses it). Every run writes into an auto-incremented run directory, and a Docker image freezes the environment; the design goal is that a successor can reproduce any figure of this report from the repository and a tile list.

### 4.3.7 Interoperability: the structure as ordinary LAS/LAZ

An on-disk ColumnStore is readable only by this repository, which is acceptable for an intermediate product and unacceptable for a deliverable. The reconstruction path was therefore completed into a full inverse: `voxelizer/reconstruct.py` rewrites a structure as LAS or LAZ, the very format the measurements arrive in, and reads such a file back into a structure; any tool that reads LAS/LAZ therefore reads the structure's content directly, and an entire partitioned run converts both ways for archiving. Three export modes exist because three things deserve preserving: point density (density), voxel geometry (one\_per\_voxel), or both (exact). The exactness of the exact mode is a literal claim, pinned by test: re-voxelising an exact export reproduces the source structure byte for byte, verified by SHA-256 equality, and fifty tests cover the guarantee and its protections, including the negative cases.

Two design decisions support the guarantee: the grid travels inside the file (a proprietary 52-byte IARBRE record holds the origin, the two cell sizes and the EPSG code, since an origin guessed from the cloud lands to within half a cell), and only raw structures are archived, grouping being able to render an interval unconvertible with nothing to flag it. The cost was measured on three real tiles at three resolutions (Table 10): at fine resolutions the archive is smaller than the structure it replaces; at 0.5 m the two are near parity; at 1 m the case must be measured run by run. The detail of the modes, the protections and the cost study is in Appendix C.

One limitation bounds all of this. An exact LAZ is a lossless container of the structure on the grid it was built on, not a replacement for the source tiles: every point it contains sits at a voxel centre, so it re-voxelises losslessly to any coarser integer multiple and never to a finer grid, and the intensity, colour, return and timestamp fields were discarded before the first voxel existed. Keeping the archive instead of IGN's tiles is a bet that the grid geometry stays fixed.

### 4.3.8 Notable obstacles for the record

Three implementation obstacles shaped the code more than any feature request did. Memory, first and always: the naive geometry collector materialised Python tuples per column, which is invisible on a single tile and fatal on a hundred; the fix, related in chapter 5, was to keep every bulk path inside numpy end to end. Second, environment fragility on the target platform: stale compiled bytecode once produced an error no reading of the sources could explain; chapter 5 records the episode, and the lesson is to trust the build environment less than the code. Third, dataset vintage: the first experiments ran on the 2018 acquisition, delivered as 1 km tiles in Lambert-93 (EPSG:2154), and comments written then survived the move to the Grand Lyon 2023 delivery of 500 x 500 m tiles named by hectometres in RGF93/CC46 (EPSG:3946); the residue was eliminated against the authoritative source, the LAZ headers themselves, and the report states only header-verified facts.

## 4.4 Ray-casting queries: from occlusion to class-aware transmittance

### 4.4.1 What the structure makes "cheap", and what it made costly

The column structure answers vertical questions directly, the ones a ray-casting query needs. A ray entering a column meets an ordered, run-compressed list of materials, so the question "what does this ray encounter, and in what order" is answered on the compressed representation without expanding it.

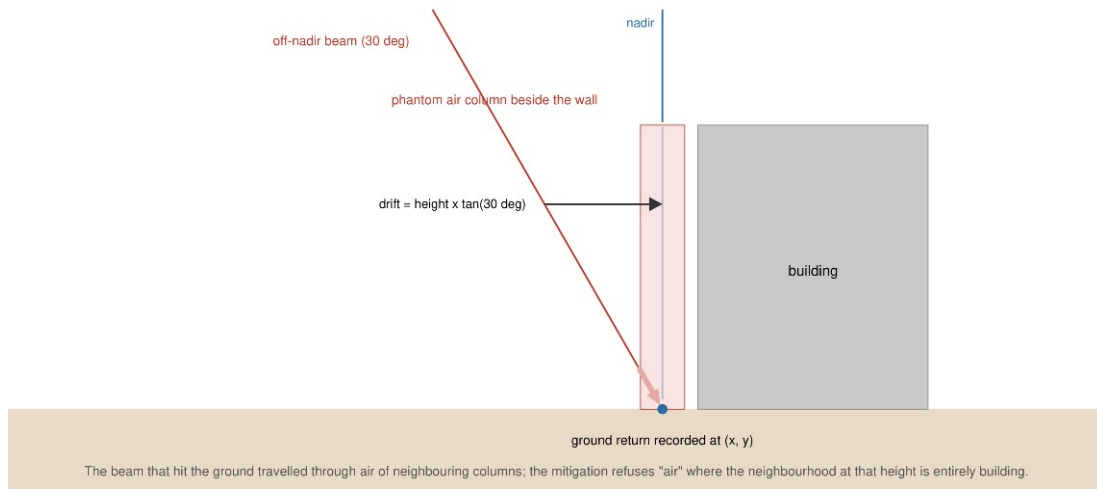
From this reference implementation (`voxelizer/ray_trace.py`, section 4.3.5), a point of vocabulary and a cost problem still separated it from the original objective (iv).

"2.5-D" names two things with opposite consequences: a data model (a height field, one value per cell, unable to represent an overhang or a canopy above clear ground) and a mere access pattern over a three-dimensional structure, which is only an optimisation. The column structure is a three-dimensional

model, the multiple intervals per column being precisely the overhangs a height field cannot express; section 4.4.7 rests on this distinction.

The cost, for its part, varied with the grid rather than with the geometry: an escaping ray walks voxel by voxel through empty air long after it has passed everything, to the point of making the shading experiment infeasible at fine resolutions. That limitation, and not a missing idea, explains why it had not been carried out (orders of magnitude in Table 11, Appendix C).

A measurement caveat accompanies these queries: the decoder's air rule assumes a nadir beam, whereas the delivery's scan angles cover  $\pm 30$  degrees (Figure 9). The argument that swath overlap would make ground returns near-nadir was tested on the data and does not hold; the implemented mitigation corrects the query without touching the structure, and chapter 5 records the residual error as a limitation (full measurements in Appendix A).



*Figure 9: The nadir-beam caveat: off-nadir beams attribute the crossed air to a neighbouring column; the decoder's neighbourhood check removes the phantom column.*

#### 4.4.2 An exact ceiling bound

The fix does not approximate: above the highest occupied voxel of a column and its four neighbours, any emptiness is provably air, and a climbing ray that crosses this ceiling can be terminated immediately. The proof is a theorem about the decoder, the implementation (CeilingDDA) leaves the reference walker intact, and the effect is decisive, bringing the daily solar arc from the order of a fortnight down to the order of an hour (proof and tests in Appendix A, costs in Table 12 in Appendix C, Figure 21 in Appendix D); an oblique ray, which leaves its column, makes do with the weaker global ceiling, whose cost section 4.4.6 gives.

#### 4.4.3 Class-aware transmittance

Objective (iv) implies ray casting that accounts for the "interactions between voxels of different territorial classes". A first-intersection query cannot express that, because it is binary: the reference walker stops at the first occupied voxel whatever its class, so a leaf stops a ray the way a concrete wall does. The question is whether a canopy lets some light through, and a boolean cannot answer it.

voxelizer/transmittance.py replaces the boolean with a transmittance in  $[0, 1]$  accumulated along the ray by the Beer-Lambert law [35], in the tradition of light extinction in canopies of Monsi and Saeki [36] and Campbell and Norman [37]. Two richer families were considered and rejected: full radiative-transfer models such as SAIL [38] and geometric-optical models [39] require leaf-angle distributions

and explicit geometries LiDAR does not supply. Path length is counted in true metres rather than voxels, so the answer depends on the geometry and not the grid; this resolution invariance is verified by test (detail in Appendix A).

$$\tau = \exp(-\sum_c k_c L_c)$$

where  $L_c$  is the path length in metres through class  $c$  and  $k_c$  is that class's extinction coefficient, in  $\text{m}^{-1}$ . Opaque materials (building, ground, water, plus the decoder's opaque-interior and underground states) take an infinite coefficient and terminate the walk at zero.

The coefficients are not supplied by default: an invented coefficient would turn every downstream figure into an assumption with a decimal point. Their derivation from the corpus is the subject of section 4.4.5, and any figure produced with a provisional coefficient is labelled as such.

The measured behaviour of this machinery, and what should and should not be read into it, is reported in chapter 5 with Figure 10.

#### 4.4.4 Solar geometry, and why this algorithm

Casting a shadow requires having the sun's direction. The implementation (voxelizer/solar.py) follows the formulation of the NOAA Solar Calculator [41], Meeus's low-accuracy algorithm [42]: its accuracy already far exceeds the grid's quantisation, the more accurate (NREL SPA [43]) or simpler (PSA [44], Michalsky [45]) solutions were set aside, and no solar library was added to the environment (comparison in Appendix C). Validation is against facts independent of the code, spherical astronomy and the published extremes of the equation of time (Table 14, Appendix C). One real trap deserves mention: solar azimuth is measured from north while the grid axes are projected; `sun_vector_grid` converts the meridian convergence explicitly, and an azimuth branch inversion, invisible to any elevation test, was caught by the east/west check alone.

#### 4.4.5 Extinction coefficients derived from the corpus

The coefficients left unsupplied by section 4.4.3 are derived from the corpus itself: LiDAR measures canopy penetration directly, and for each column carrying vegetation above a ground return, the fraction  $p$  of pulses reaching the ground and the canopy thickness  $H$  give  $k = -\ln(p) / H$ . The order obtained is physically sensible without having been imposed (values and monotonic decrease in Table 3).

Table 3: Extinction coefficient per dominant vegetation class, 0.5 m cubed, from the per-pulse ground fraction.

| Class               | $n$     | $k$ median (1/m) | p10   | p90   |
|---------------------|---------|------------------|-------|-------|
| 3 low vegetation    | 7,147   | 0.497            | 0.178 | 1.198 |
| 4 medium vegetation | 103,330 | 0.429            | 0.212 | 0.942 |
| 5 high vegetation   | 396,145 | 0.326            | 0.176 | 0.656 |
| 8 upper canopy      | 233,294 | 0.250            | 0.141 | 0.501 |
| all                 | 739,916 | 0.311            | 0.163 | 0.666 |

Four limitations, detailed in Appendix C, stop this being a finished result:  $k$  is a distribution and not a per-column property; the estimate depends on resolution;  $p$ , first estimated from return counts, had to be reconstituted pulse by pulse; and the coefficient, derived from a near-nadir survey, is applied without angular correction to oblique rays. Figure 22 (Appendix D) shows the distributions, and the derivation is reproducible from the repository.

#### 4.4.6 Direct sun hours: the experiment

For each column and each solar position above the horizon, a ray starts from 1.5 m above the top of the column's lowest interval, eye height where ground is measured and roof level where it is not, towards the sun, and transmittance accumulates into sun hours. The computation is run twice, all materials opaque and then with the coefficients of section 4.4.5: the difference isolates what the volumetric material model contributes. The exact edge rules are in Appendix C, and the measured results in chapter 5.

These runs carry the oblique-ceiling correction, which lowered every absolute value without inverting any (detail and costs in Appendix C).

Scope of the experiment: the direct beam only, with no diffuse sky component, reflection, atmospheric attenuation or cloud; a geometric statement about the city, not a radiometric statement about energy, whose conversion would require terms LiDAR geometry does not supply. Nor are the coefficients settled: the return-count bias was corrected, the resolution dependence remains, and this computation, run at 1.0 m with the lowest coefficients, would report a smaller gain on a 0.5 m structure, by an unmeasured amount (figures in Appendix C).

Beyond the solar checks, the shadow geometry is validated analytically against the closed form of a tower's shadow on flat terrain: the computed boundary agrees to within 0.36 m, less than one horizontal cell (detail in Appendix A).

A native failure (Windows access violation) hit the first complete run; the diagnosis and the fixes, which roughly halved the run time with results proven identical, are recorded in section 5.4 and Appendix C.

#### 4.4.7 Comparison with 2-D and 2.5-D approaches

The comparison with 2-D and 2.5-D approaches (objective v) was carried out with no external dataset at all: four compared models are built from one and the same column structure, so that the representation is the only variable. Model V is the structure itself, read opaque and then under the transmittance of section 4.4.3; model S, a 2.5-D surface model that reduces each column to a single body; model A, a set of 2-D rasters reproducing the data model of the neighbouring greening module (section 3.3); model S2, a control that changes only the height of the sampling point, separating that from the occlusion model. The material is ten stratified square kilometres of Grand Lyon at two grids, and the models are verified on four synthetic scenes, three where they must agree and one where they must diverge (detail in Appendix C).

The measured results of this comparison, and the share of the divergence owed to the grid rather than the representation, are reported in chapter 5.

Two measurements remain due and are stated as such: a figure of occlusion as a function of resolution with the ray origin defined relative to the ground surface, and a Metropolis-scale sun-hours pass under the pipeline's per-chunk process isolation.

## 5 Results

This chapter follows the supervisors' plan for the results: what was tested (5.1), how the design was validated quantitatively (5.2), what the system produces (5.3). It then opens a discussion of limitations and failures (5.4), and an overall assessment against the objectives of chapter 2 (5.5).

### 5.1 Tests

The test suite grew with the system and covers each algorithmic module in isolation as well as the properties that hold the pipeline together. Per-module tests exercise in turn the decoder, the ground index, the overlap resolver, the absorption pass, the denoising filters, the ray caster, the ceiling-bounded ray walker, the transmittance accumulator, the solar geometry, the grid codec, the tileset exporter, stage isolation and area persistence.

Three equivalence properties carry more weight than any isolated unit test, because they licence the architecture's flexibility.

First, chunked reading equals whole-file reading: voxelising a tile streamed in memory-bounded chunks produces a structure identical to the one obtained in a single pass, which lets the pipeline choose its reading strategy on memory considerations alone.

Second, area processing equals the clipped union: processing an area tile by tile against the common origin produces the same structure as voxelising the union of the clipped inputs, which makes the merged and partitioned modes interchangeable in meaning.

This equality was confronted at production scale. A one-square-kilometre box, four tiles and 52,705,942 points, was voxelised twice at 0.5 m cubed, once as a single in-memory area and once as four fragments merged out of memory: both paths give 3,995,579 columns and 5,593,880 intervals, element-wise equal column keys, and no divergent column. Point-count conservation, checked on every one of Appendix C's forty structures, holds everywhere: the sum of the sixteen fragments' counts equals, to the unit, the merged structure's total.

The comparison also revealed a since-corrected format defect: the direct path wrote counts as int64 against the int32 the class documents, with values always identical, so loading already brought the array back to the declared type. With the defect fixed, the direct path writes 14.1% less raw array, and direct voxelisation rerun on the same tiles produces an archive whose seven members are byte-for-byte identical (SHA-256) to those of the merged path.

Finally, interrupted runs fail cleanly: a run stopped by the memory watchdog leaves partial outputs in a well-defined state; the files are not corrupted.

The campaign snapshots recorded during development (30 of 30, 32 of 32, 41 of 41, then 14 of 14 and 19 of 19 end-to-end CLI checks) are recorded in Appendix C; the resolution sweep of section 5.2 doubles as the final non-regression campaign, eight complete runs with five isolated output stages, all forty stage runs succeeding.

The package is covered by an extensive automated test suite, collected by pytest under tests/, alongside four script-style suites run separately and one platform-conditioned module, skipped on Windows. The subsets cited elsewhere in this report live inside that suite: ten tests for the ceiling bound, fifteen for transmittance, fourteen for solar geometry, and fifty spread over the two archive and round-trip files.

The inverse voxeliser closes the loop from the outside: reconstructing a LAS/LAZ point cloud from a structure and auditing it with standard tools confirms that positions and classes survive the conversion, up to quantisation. That loop was then closed exactly: the exact export mode of section 4.3.7 converts a structure to LAS/LAZ and back with SHA-256 equality of the re-saved structure, verified on nine real

tile-and-resolution combinations and pinned by the archive and round-trip tests covering the guarantee and each of the protections around it.

## 5.2 Validation

The quantitative validation campaign addresses the design's central claim: that run-compressed columns compress urban LiDAR strongly while preserving per-class semantics. The test area comprises three 500 m tiles in eastern Lyon (Villeurbanne and the ring road: tiles 18410\_51755, 18410\_51760 and 18415\_51760, together 74.3 million points covering ground, the three vegetation tiers, building, the class-8 upper canopy tier, unclassified returns and water, the latter present in two of the three tiles). The raw baseline and the four encoder variants of section 4.2.4 were applied at two resolutions, and each result was assessed on four criteria: serialised interval payload size, bytes per original point, top-class disagreement rate between the raw and encoded structures (the percentage of columns whose topmost class changed), and per-class Jaccard overlap [46] of the plan footprints (dominant class per column) for building, vegetation and ground (1.000 means the encoded structure occupies exactly that class's raw voxels). Table 4, below, gives the results at both resolutions.

Four conclusions follow, and each maps to a recommendation.

First, raw run-length encoding alone is a strong reduction, but the factor depends entirely on the baseline, so both are given. The 74,265,365 input points serialise to 37.54 MB at the coarse resolution, 0.51 byte per point. Against uncompressed LAS 1.2 point records (28 bytes each, 2,079 MB), that is a factor-55.4 reduction; against the tiles as actually delivered by IGN, already LAZ-compressed and totalling 327.4 MB for these three tiles, it is a factor of 8.7. The second is the defensible figure for a storage argument, since nobody stores uncompressed LAS; the first remains the right one for an information-content argument.

Second, gap fill (v2) removes about 30% of the intervals at the coarse resolution and the campaign records no semantic change (0.000% top-class disagreement, all Jaccards at 1.000). Both metrics are structurally blind to it: v2 merges same-class intervals only. Hence a column's top class cannot change, and the plan-view footprints cannot move. This null result is a consistency check, not a measurement of losslessness. The claim rests instead on the definition of the merge, which never invents a class change, together with the point-count caveat of Appendix C; on that basis it should always be applied for plan-view products.

Third, class smooth (v1) buys the largest reduction for a real but small cost: vegetation Jaccard of 0.951 at the coarse grid and 0.953 at the fine one, against interval reductions of 32.2 and 22.5%. It is the recommended default for analysis products.

Fourth, the majority variant (v4), the supervision-meeting idea implemented so it could be measured, is quantifiably harmful: at the coarse resolution its plan-view vegetation footprint departs from the raw footprint over 27% of their union ( $J_{veg}$  0.731), and over 18% at 0.5 m ( $J_{veg}$  0.820). One minus a Jaccard index is a symmetric difference, not a destroyed fraction, and the metric covers the entire vegetation footprint while the rule rewrites only the band above 5 m; what it establishes is that the raw and encoded canopies part ways over more than a quarter of their union at the coarse grid, on the canopy the project exists to protect. It remains useful for lightweight visualisation and is excluded from analysis.

Minor-class drop (v3) is the only variant whose effect does not survive the change of resolution, removing 8.9% of intervals at 1.0 m horizontal and 0.1% at 0.5 m: finer horizontal cells carry fewer points per column, so fewer intervals fall under a threshold set as a share of the column's returns. This reading is consistent with both tables and was not measured separately.

The resolution comparison carries a recommendation of its own: the coarse grid is 3.3 times smaller than the fine one (37.5 versus 124.6 MB raw), and the plan-view reconstructions show that every

building footprint and every crown in the area survive the coarsening; 1.0 m horizontal cells are therefore the sensible default for analysis products, with 0.5 m reserved for neighbourhood-scale studies. The software's own default stays unchanged at 0.5 m cubed (Appendix B): the recommendation concerns the products, and was not propagated to the shipped default.

Area-scale validation comes from a systematic production sweep, not from the experimental test bench: the same 3 x 3 km area (36 tiles, 751.6 million points, in download-voxelise-delete streaming, full output chain including the streaming viewer) was processed at four resolutions, each with and without interval grouping, the production option that merges per column consecutive same-class intervals before outputs are drawn.

Grouping and gap fill (v2) are a single operation run at different tolerances on different grids: the -29.5% of Table 4 and the -7.01% of section 5.2.1 measure the same merge on different inputs and are not comparable. Likewise, bytes per point rests on three distinct bases depending on the table; the exact definitions are in Appendix C, and Table 5 below summarises the eight archived runs.

*Table 4: Encoder validation at 1.0 m and 0.5 m horizontal resolution (0.5 m vertical; 74.3 M input points).*

| Encoder                            | Intervals | Payload   | vs raw (interv.) | B/pt | Disagr. % | J_bld | J_veg | J_gnd |
|------------------------------------|-----------|-----------|------------------|------|-----------|-------|-------|-------|
| <b>1.0 m horizontal resolution</b> |           |           |                  |      |           |       |       |       |
| v0 raw                             | 2,195,849 | 37.54 MB  | baseline         | 0.51 | 0.000     | 1.000 | 1.000 | 1.000 |
| v1 class smooth (3 voxels)         | 1,488,853 | 28.35 MB  | -32.2%           | 0.38 | 1.659     | 0.997 | 0.951 | 0.986 |
| v2 gap fill (1 m)                  | 1,547,692 | 29.11 MB  | -29.5%           | 0.39 | 0.000     | 1.000 | 1.000 | 1.000 |
| v3 minor class drop (1%)           | 1,999,664 | 34.99 MB  | -8.9%            | 0.47 | 0.486     | 0.999 | 0.977 | 0.998 |
| v4 majority above 5 m              | 1,718,806 | 31.34 MB  | -21.7%           | 0.42 | 5.578     | 0.992 | 0.731 | 0.997 |
| <b>0.5 m horizontal resolution</b> |           |           |                  |      |           |       |       |       |
| v0 raw                             | 6,833,550 | 124.58 MB | baseline         | 1.68 | 0.000     | 1.000 | 1.000 | 1.000 |
| v1 class smooth (3 voxels)         | 5,298,833 | 104.63 MB | -22.5%           | 1.41 | 1.527     | 0.998 | 0.953 | 0.988 |
| v2 gap fill (1 m)                  | 5,411,809 | 106.10 MB | -20.8%           | 1.43 | 0.000     | 1.000 | 1.000 | 1.000 |
| v3 minor class drop (1%)           | 6,828,027 | 124.51 MB | -0.1%            | 1.68 | 0.0003    | 1.000 | 1.000 | 1.000 |
| v4 majority above 5 m              | 5,329,452 | 105.03 MB | -22.0%           | 1.41 | 3.261     | 0.997 | 0.820 | 0.999 |

*Table 5: The resolution and grouping sweep: one 3 x 3 km area, 751.6 M points, eight complete pipeline runs.*

| Run | cell xy / z (m) | Grouping | Intervals   | Mean/col | Structure (.npz) | Bytes/pt | Streaming .bin |
|-----|-----------------|----------|-------------|----------|------------------|----------|----------------|
| 1   | 1.0 / 1.0       | grouped  | 17,958,311  | 2.46     | 70.5 MB          | 0.09     | 575 MB         |
| 2   | 1.0 / 1.0       | raw      | 20,719,785  | 2.83     | 78.2 MB          | 0.10     | 663 MB         |
| 3   | 0.5 / 0.5       | grouped  | 57,979,172  | 1.98     | 241.7 MB         | 0.32     | 1.86 GB        |
| 4   | 0.5 / 0.5       | raw      | 82,213,507  | 2.81     | 305.3 MB         | 0.41     | 2.63 GB        |
| 5   | 0.25 / 0.25     | grouped  | 187,211,562 | 1.61     | 817.5 MB         | 1.09     | 5.99 GB        |
| 6   | 0.25 / 0.25     | raw      | 270,144,263 | 2.32     | 1.04 GB          | 1.39     | 8.64 GB        |
| 7   | 0.25 / 0.1      | grouped  | 187,346,004 | 1.61     | 920.2 MB         | 1.22     | 6.00 GB        |
| 8   | 0.25 / 0.1      | raw      | 317,402,458 | 2.72     | 1.32 GB          | 1.76     | 10.16 GB       |

### 5.2.1 Resolution and grouping series over the entire Metropolis

Beyond the three-tile test bench, the complete Grand Lyon 2023 corpus was processed end to end: 2,842 tiles, 51,912,212,206 points, one common origin (1831000.0; 5151500.0; 130.75), bbox x [1831000, 1861000] and y [5151500, 5195500], 30 km by 44 km. At 500 m per side, 2,842 tiles make 710.5 km<sup>2</sup> of footprint, so the delivery covers the entire administrative territory and spills beyond it; a per-area figure computed against the administrative surface would be wrong by a third. The vertical origin, 130.75 m, is the minimum over the tile headers' bounding boxes, an indexing origin claiming no surface: class 7 collects returns located below ground, and some headers' ranges reach down there. Every run below is over this same corpus; only the cell size and the interval-grouping setting change. Odd-numbered runs are grouped, even-numbered ones are the ungrouped control at the same resolution. Table 6 below gives the per-run detail: columns, intervals and three on-disk sizes.

Table 6: Structure geometry over the entire Metropolis, by resolution and grouping (51,912,212,206 points).

| Run  | cell xy / z (m) | Grouping  | Columns       | Intervals     | iv/col | Payload  | area.npz | stream.bin |
|------|-----------------|-----------|---------------|---------------|--------|----------|----------|------------|
| Run1 | 2.0 / 2.0       | grouped   | 162,581,998   | 422,234,263   | 2.60   | 8.1 GB   | 1.56 GB  | 13.51 GB   |
| Run2 | 2.0 / 2.0       | ungrouped | 162,581,998   | 426,927,675   | 2.63   | 8.2 GB   | 1.58 GB  | 13.66 GB   |
| Run3 | 1.0 / 1.0       | grouped   | 648,546,151   | 1,351,550,067 | 2.08   | 27.9 GB  | 5.23 GB  | 43.25 GB   |
| Run4 | 1.0 / 1.0       | ungrouped | 648,546,151   | 1,453,375,280 | 2.24   | 29.3 GB  | 5.56 GB  | 46.51 GB   |
| Run5 | 0.5 / 0.5       | grouped   | 2,584,348,757 | 4,591,625,358 | 1.78   | 101.0 GB | 18.21 GB | 146.93 GB  |
| Run6 | 0.5 / 0.5       | ungrouped | 2,584,348,757 | 5,418,440,422 | 2.10   | 111.8 GB | 20.41 GB | 173.39 GB  |

The payload is the exact serialised size (16 bytes per column plus 13 bytes per interval); area.npz is the compressed structure and area\_stream.bin the streaming viewer's payload, both as written on disk. The three size columns are in decimal gigabytes. Grouping merged same-class intervals across gaps of one cell at 2.0 m, two cells at 1.0 m and three cells at 0.5 m. Two cross-checks hold inside the table: Run3's merge log records its pre-grouping count at 1,453,375,280 intervals, exactly Run4's total, so the grouped and ungrouped runs at 1.0 m agree on the structure they start from; and Run4's streaming payload divided by its interval count is exactly 32 bytes, the fixed record of section 4.3.4. The two 0.5 m runs were also streamed; each payload equals its interval count times the 32-byte record.

Two findings follow.

First, the structure scales as expected, but its cost is not linear: dividing the cell area by four, from 2.0 m to 1.0 m, multiplies columns by 4.0 and intervals by 3.2, and again from 1.0 m to 0.5 m by 4.0 and 3.4, comparing grouped runs with grouped runs throughout. Intervals per column stay close to two across a sixteen-fold change in cell area, which is the column-compressed design's central claim holding at Metropolis scale: a column costs about two intervals whatever the resolution, hence a structure that never degenerates towards a dense grid.

Second, the benefit of interval grouping depends strongly on resolution, and it runs against intuition.

Grouping removes almost nothing at 2.0 m and two fifths of the intervals at 0.25 m, and the series in between is monotonic. The mechanism is physical, not algorithmic: with coarse cells a column already collapses into a few long runs, so almost no same-class gaps remain to close; with fine cells, sparse LiDAR returns fragment one and the same physical object into many short runs separated by empty voxels, and grouping recovers that fragmentation.

| Resolution                                | Ungrouped     | Grouped       | Reduction |
|-------------------------------------------|---------------|---------------|-----------|
| 2.0 m cubed (Run2 -> Run1)                | 426,927,675   | 422,234,263   | -1.10%    |
| 1.0 m cubed (Run4 -> Run3)                | 1,453,375,280 | 1,351,550,067 | -7.01%    |
| 0.5 m cubed (Run6 -> Run5)                | 5,418,440,422 | 4,591,625,358 | -15.26%   |
| 0.25 xy / 0.1 z (Run8 -> Run7, 3 km area) | 317,402,458   | 187,346,004   | -40.98%   |

The practical implication is that --group-intervals should be considered mandatory at 0.5 m and below and is close to a no-op above 1 m. Of note: the last row concerns a 3 km area, so it establishes the trend rather than a metropolitan-scale figure; a full-corpus run at 0.25 m would be needed to close the series (future work, section 6.2). Runs 5 to 8 of Table 5 make the same point from the other end: refining the vertical cell from 0.25 m to 0.1 m takes the ungrouped interval count from 270.1 to 317.4 million while the grouped count moves by 0.07%, from 187,211,562 to 187,346,004. What the finer grid adds is therefore fragmentation that grouping removes at once.

The largest merged production run to date covers a 3 x 3 km bounding box of 36 tiles on the western side of the inventory (bbox 1834000 5170000 to 1837000 5173000, whose centre sits 10.5 km west and 2.0 km south of the corpus centre): 751,592,610 points became 82,213,507 intervals spread over 29,242,275 columns, a mean of 2.81 intervals per column at 0.5 m cells (an archived run, recorded in outputs/README.md). At area scale the average column costs fewer than three intervals, which is the regime the design was built for.

The comparison with the isolated dense tile 18410\_51825 calls for caution, because resolution differs there as much as urban character; brought to a common grid, a wall-rich tile costs about twice as many intervals per column as a mixed area, not five times (detail in Appendix C). Beyond capacity, the run demonstrates the architecture's claims in production: tile-by-tile processing against a common origin, merged statistics and whole-area maps assembled.

### 5.2.2 Ray-casting results

The query layer of section 4.4 delivers three measured results: the behaviour of the transmittance probe, the direct-sun-hours experiment, and the comparison with 2-D and 2.5-D approaches.

Three things are to be read in Table 13, reported in Appendix C, and one is not.

First, the model behaves as physics demands: for a fixed structure, raising the coefficient lowers mean transmittance monotonically, while the mean canopy path length stays practically constant, since the geometry has not changed. This separation of geometry and material is what the boolean walker could not express.

Second, the gap between the opaque row and the finite-coefficient rows is the quantity objective (iv) asks for. At 0.25 xy / 0.1 z with  $k = 0.1$ , mean transmittance rises from 0.650 to 0.815 and the fraction of columns receiving more than half the incident light rises from 0.65 to 0.84. That difference is attributable entirely to treating vegetation as partially transmissive rather than solid.

Third, cost is now dominated by the number of rays rather than by vertical resolution, which makes a whole-day simulation feasible.

What should not be read into it: the strong variation of the opaque baseline with resolution (0.000; 0.440; 0.650) is not a physical result. It is partly a real effect, in that coarser voxels are occupied as soon as a return falls anywhere inside them and therefore occlude more, and partly an artefact of this probe, which starts every ray at a fixed 0.5 m above the base of the lowest interval, a height that sits inside the ground voxel for 1.0 m cells and above it for 0.1 m cells. The probe is a demonstration that the machinery works and behaves monotonically, not a measurement of occlusion as a function of resolution. A defensible measurement of that effect requires a ray origin defined relative to the ground surface (future work, section 6.2). Figure 10, below, places the two material models side by side over a whole area at an origin of that kind: one vertical sky-facing ray per column of area B04, from 1.5 m above the top of the column's lowest interval, with all materials opaque in the first view and the class-aware coefficients of section 4.4.5 in the second.

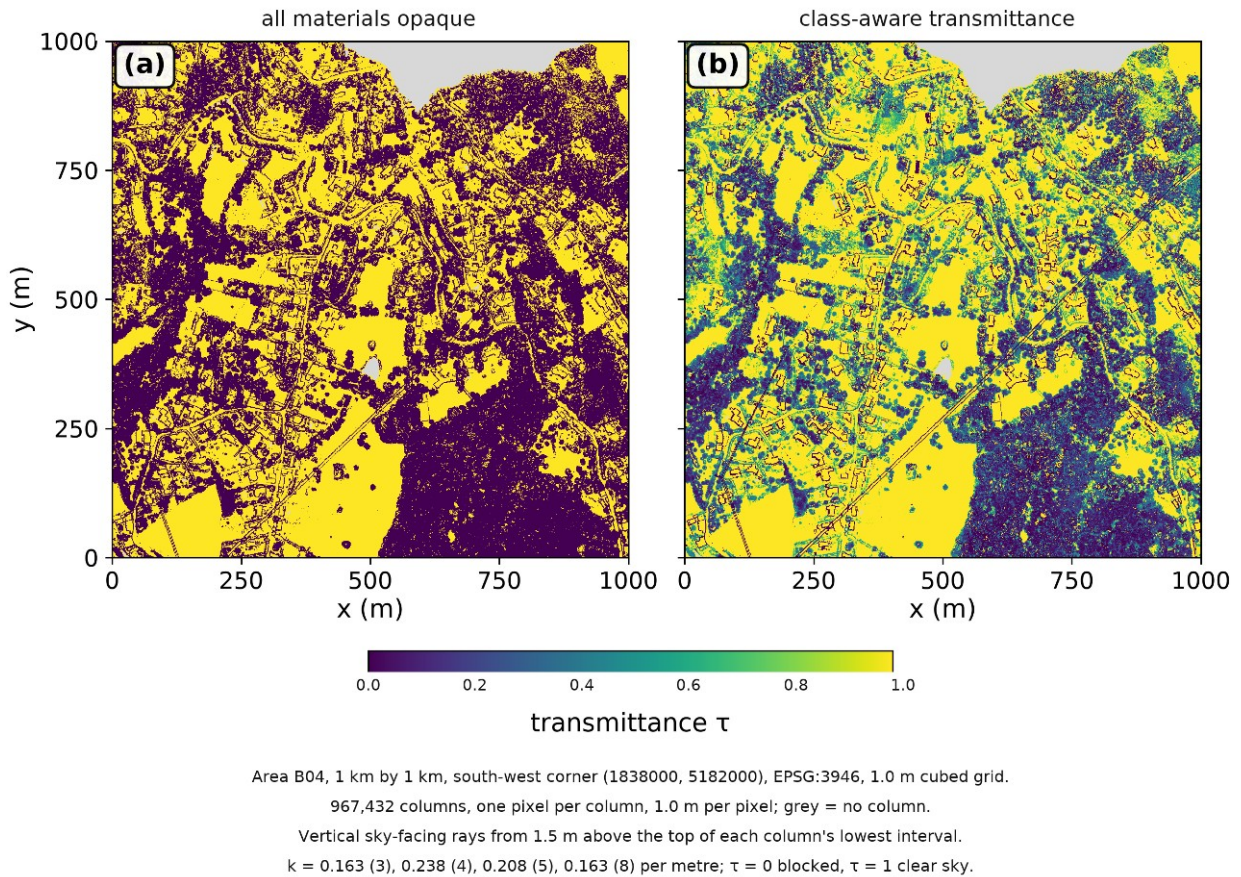


Figure 10: Vertical sky-facing transmittance over area B04, the same rays in both views: (a) all materials opaque, (b) class-aware transmittance at the coefficients of section 4.4.5. Mean transmittance rises from 0.461 to 0.635 and the blocked fraction falls from 0.539 to 0.086.

The passage from the opaque model to the class-aware model gains direct sun in both areas; Table 7 gives the measured means.

Table 7: Direct sun hours, summer solstice 2026-06-21, 15-minute sampling (63 solar positions), 12,000 columns sampled per area at 1.0 m cubed. B04 and B10 are 1 km by 1 km boxes whose south-west corners are at (1838000, 5182000) and (1853000, 5171000), drawn by a deterministic quintile rule over the corpus's 2,657 complete 1 km boxes (Experiments/run\_stratified\_n10.py); the composition percentages are shares of the box's raw LAZ points. The coefficients are section 4.4.5's 1.0 m fit, applied per metre: 0.163 for class 3, 0.238 for class 4, 0.208 for class 5 and 0.163 for class 8. Means are rounded to the nearest thousandth of an hour; differences are computed on the unrounded values.

| Area | Character                          | Opaque mean | Class-aware mean | Difference                         | Sunless columns (opaque -> class-aware) |
|------|------------------------------------|-------------|------------------|------------------------------------|-----------------------------------------|
| B04  | 58.6% vegetation,<br>3.7% building | 4.398 h     | 6.233 h          | <b>+1.834 h</b><br><b>(+41.7%)</b> | 47.9% -> 7.5%                           |
| B10  | 7.9% vegetation,<br>25.9% building | 10.357 h    | 11.064 h         | +0.707 h<br>(+6.8%)                | 15.1% -> 9.3%                           |

The result behaves as physics demands, and the comparison between the two areas is the proof that it measures what it claims to. The effect is six times larger in relative terms in the vegetated area than in the built one (+41.7% of the opaque mean against +6.8), because the difference between the two models is the treatment of the canopy: buildings are opaque in both. The two areas also differ sharply in vertical extent – the vegetated structure spans 234 occupied voxel levels, as much from terrain relief as from canopy, against 39 in the built box – and that extent governs how much down-the-road geometry a climbing ray must clear. Under the opaque model, 47.9% of the vegetated area's columns receive no direct sun at all; treating the canopy as partially transmissive brings that down to 7.5%. That gap – some two fifths of the area – is the practical consequence of the representation choice, and it is accessible to no surface model, which has no way of expressing "the crown above this point lets some light through".

Compared naively, the surface model reads 9.7322 hours of direct sun per day over the vegetated area against 4.4408 for the opaque structure, a gap of 5.2914 hours. The point is not the occlusion model. Model S2, which changes only the height the ray starts at, sits 0.3115 hour *below* the structure: merging the column and filling the measured air under a neighbouring crown costs about a third of an hour, and that cost runs the other way. The naive comparison flatters the volumetric model; the controlled comparison still shows the divergence and localises it. Where a column carries a single body, the two answers agree to within the hour on 79 and 92% of columns; where a canopy tops the ground, they part by 9.7 to 11.7 hours of direct sun per day. A height field has no underneath, and it is that, not the column merge, that the divergence measures. The error moreover runs both ways: against the class-aware model, the surface model is lower on 45.3% of the vegetated area's columns, because the volumetric structure lets part of the beam through a neighbouring crown, which the surface model does not. Over the ten stratified square kilometres, the surface model additionally overestimates canopy volume by 13 to 54% at 1.0 m, more at 0.5 m, and cannot represent the 8.5 to 40% of columns carrying two bodies separated by measured air.

Resolution moves only one side: model S is almost grid-invariant, whereas the volumetric models rise by about an hour when the cell is halved (a 1 m voxel carrying a single foliage return is entirely opaque while at 0.5 m only one of its eight sub-voxels is). The figure this report retains is therefore the 0.5 m one: 3.755 hours on the vegetated area and 1.299 on the built one against the opaque structure, versus 5.291 and 2.334 at 1.0 m. Model A is fixed in resolution by its own design: the fine comparison therefore pits the volumetric models at 0.5 m against a raster at 1 m, and that asymmetry is itself a

result. The four other queries, the per-area detail and the three measured costs are reported in Appendix C.

### 5.3 Demonstration

Figure 11 shows the encoder validation in plan view, the majority variant's vegetation loss visible to the naked eye.

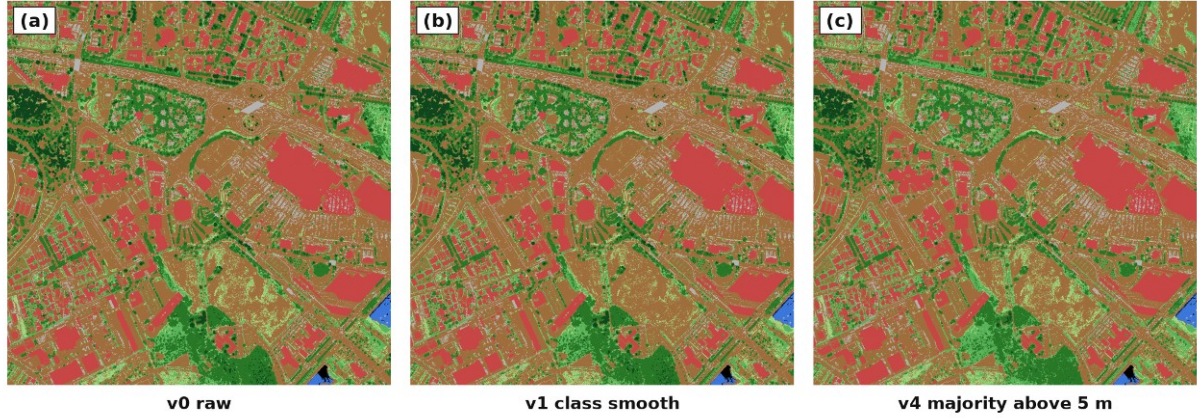


Figure 11: Encoder comparison over an urban area 1.5 km on a side (Bron, around the Porte des Alpes campus; 1.0 m horizontal by 0.5 m vertical cells): (a) raw, (b) class smooth and (c) majority above 5 m; the variants diverge from raw on only 2.7% and 3.4% of columns, the differences concentrating on canopy edges and mixed roofs. The dark patches near the water bodies are zones with no laser return (absorption by water).

Figure 12 gives the distribution of intervals per column for the finest raw run of section 5.2's sweep (0.25 m horizontal, 0.1 m vertical, ungrouped): the regime where fragmentation is at its worst and the diagnostics most informative.

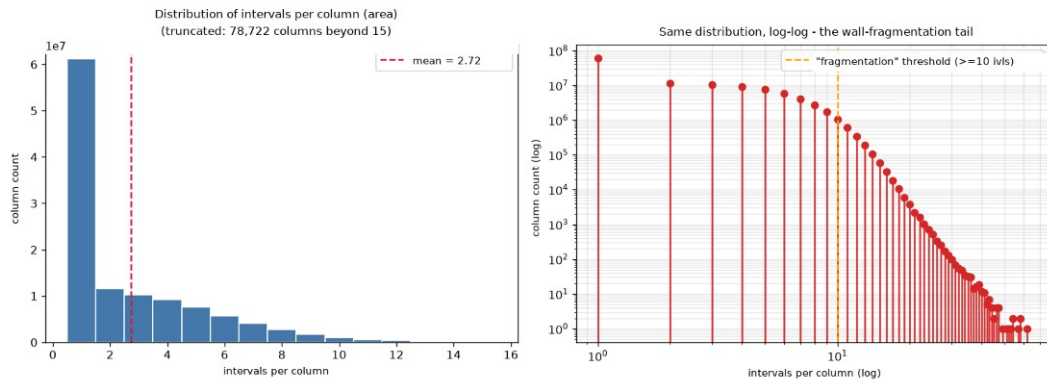


Figure 12: Distribution of intervals per column for the finest raw structure: the long tail is wall fragmentation.

Figure 13 presents the four plan-view map modes of the 3 x 3 km area run.

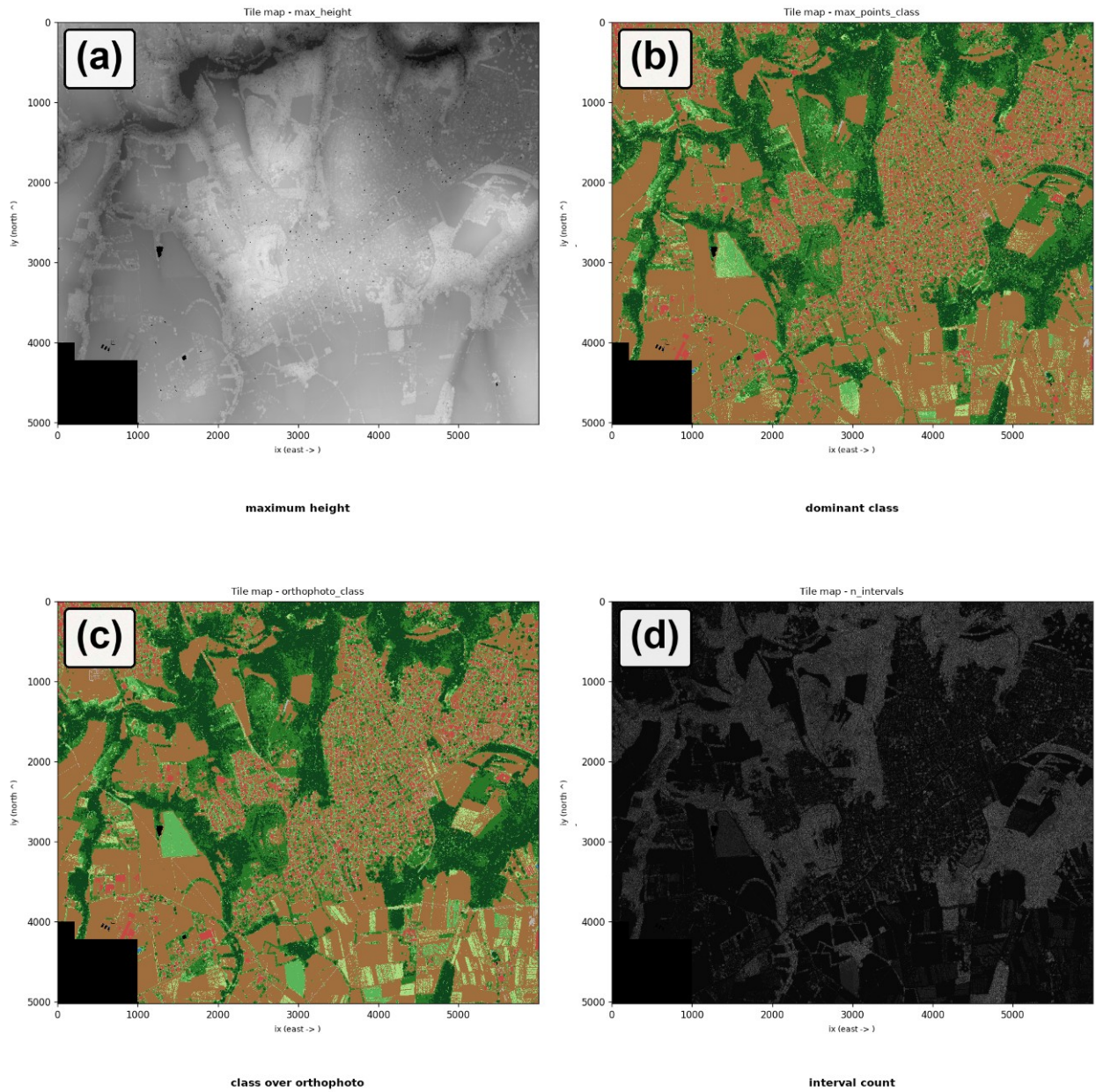


Figure 13: The four plan-view map modes of the 3 x 3 km area run: (a) maximum height, (b) dominant class, (c) class over orthophoto, (d) interval count.

Figures 14 and 15 show the column diagnostics, the direct visual form of the data structure: sample columns as interval stacks, then per-class panels of the same finest raw run.

# Witness columns of 18350\_51730 (cell\_xy=0.25 m, cell\_z=0.1 m)

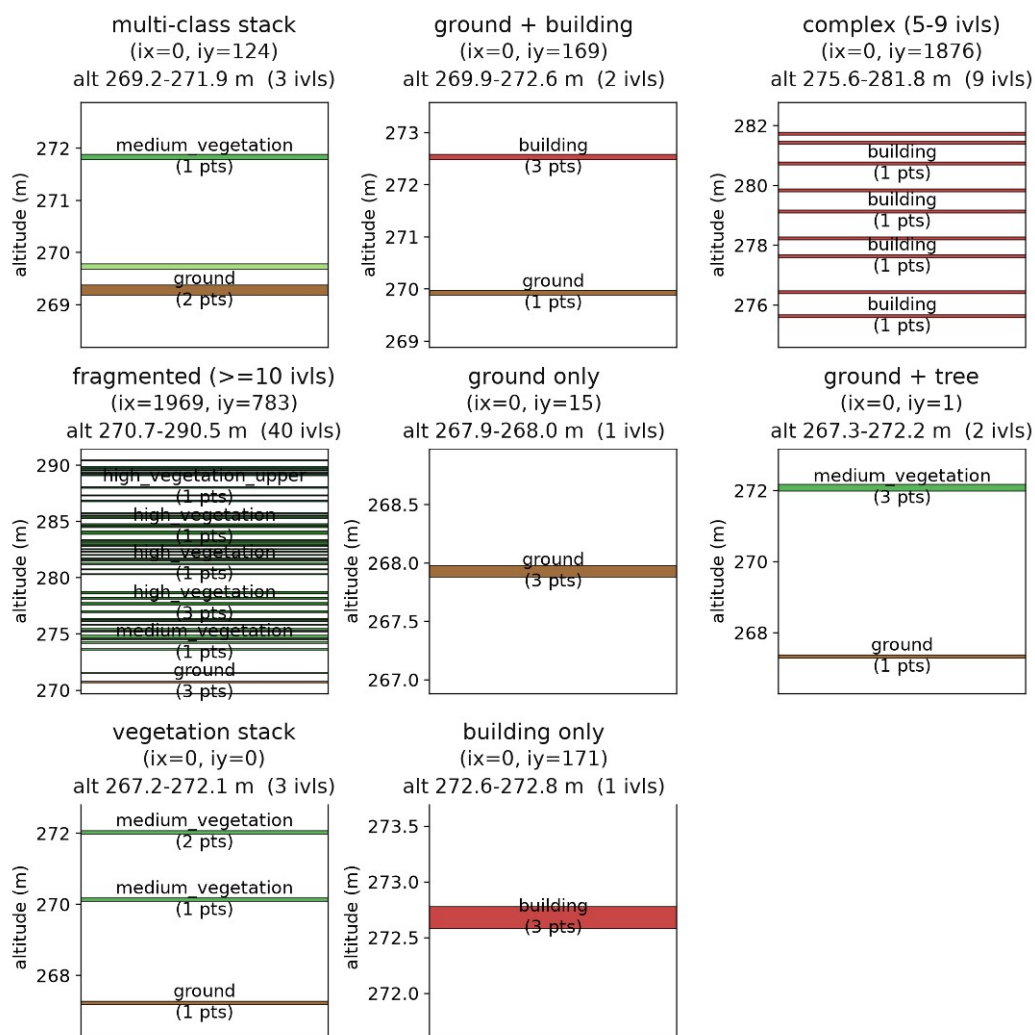


Figure 14: Column diagnostics of tile 18350\_51730, one sample column per category rendered as an interval stack.

One representative column per class (cell<sub>xy</sub>=0.25 m, cell<sub>z</sub>=0.1 m)

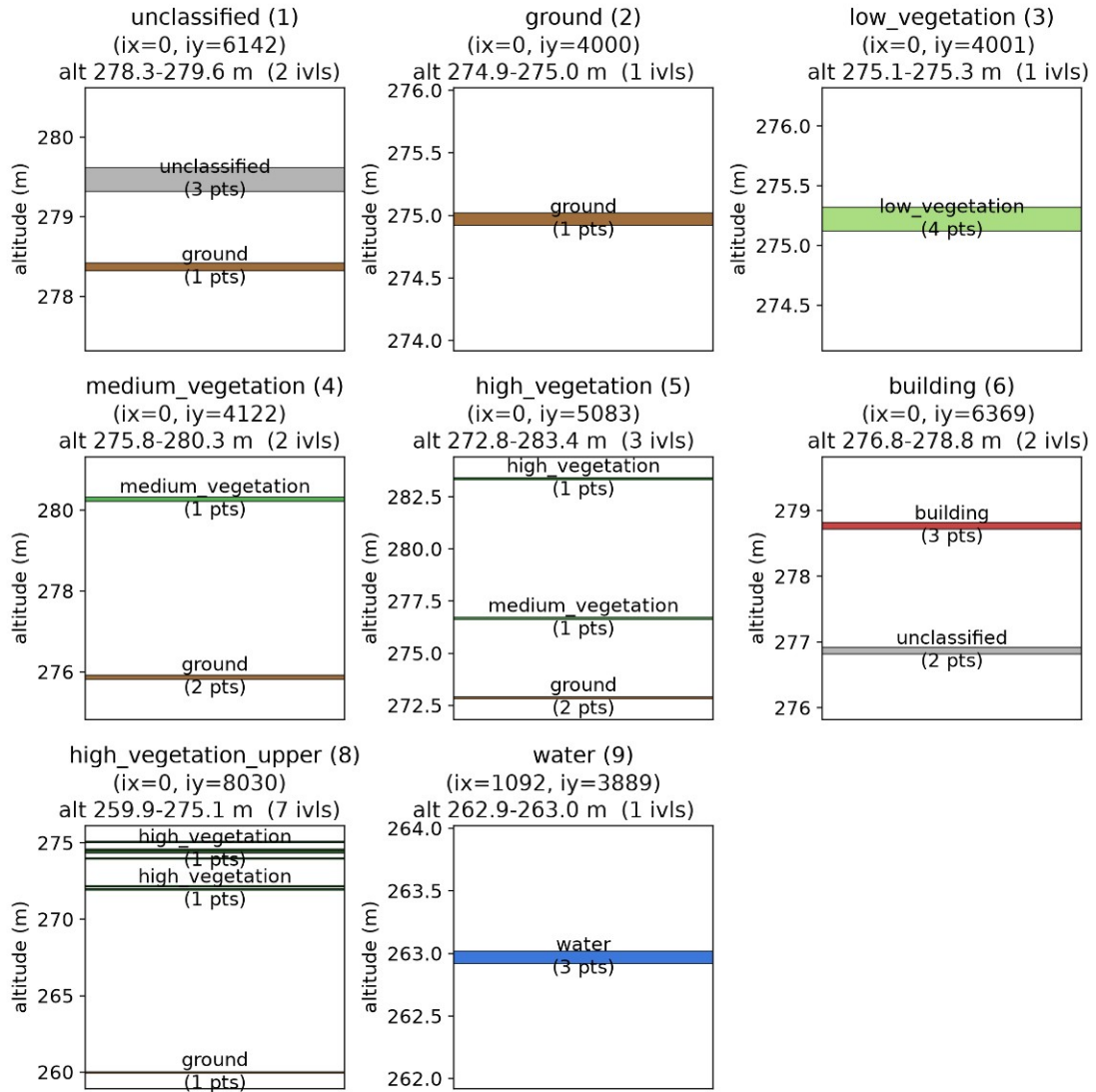


Figure 15: Per-class column panels of the finest raw structure.

Figure 16 shows the streaming viewer over a 2 x 2 km study area at 0.25 m horizontal and 0.1 m vertical cells, navigated in a browser: the payload carries one 32-byte record per full-detail interval box, 96.5 million for this area, and the per-frame budget decides which tiles stay resident on the GPU, here 46 tiles of 1,024 and about 4.5 million instances drawn at 35 frames per second. Figure 17 sets the same area against Google Earth, on 2023 imagery like the LiDAR campaign: the buildings and tree crowns are directly recognisable.



Figure 16: The streaming viewer over a 2 x 2 km study area at 0.25 m horizontal and 0.1 m vertical cells: street-level navigation with per-class counts and visibility toggles; 46 resident tiles of 1,024, about 4.5 million interval boxes drawn out of 96.5 million at full detail, at 35 frames per second.



Figure 17: Google Earth view of the same study area (3-D imagery from 2023, © Google): the buildings and tree crowns of Figure 16 are directly recognisable.

Beyond the toolchain's own viewers, the exported tileset demonstrates interoperability in practice: the same tileset.json, served as static files, displays in CesiumJS, in IGN's iTowns and in Unreal Engine via Cesium for Unreal, at its true Lyon coordinates, with detail streamed by the pyramid; the repository's tilesexport directory documents the full path.

## 5.4 Discussion and limitations

The limitations divide into what the input imposes, what the design accepts, and what the implementation taught us the hard way. Inherited from the input, the structure's semantics are no better than IGN's classification. Wall returns fragment columns, but the size of that effect must be stated with care: at a common grid, a wall-rich tile costs about twice as many intervals per column as a mixed area, not the five times a naive comparison across different resolutions suggests (section 5.2.1). The class-8 upper canopy tier is a de facto production convention, not a documented class, and mislabelled ground returns must be handled by heuristics (the highest-ground rule of section 4.2.3). None of this is fixable at our stage without re-solving classification, which was excluded from the scope of our study.

Accepted by design, the nadir-beam assumption leaves a residual error of a few suspect columns next to tall buildings, even after mitigation (section 4.4.1), whose width was measured over one square kilometre, not the whole delivery: a figure derived from an assumed field of view of plus or minus 15 degrees was disproven by measurement over the 2,842 tiles (the real range is minus 30 to plus 30 degrees, which doubles the lateral drift to 0.58 m per metre of height, 28.9 m at 50 m). The replacement is a fraction of building height rather than a fixed number of columns, and it rests on four tiles whose tallest building reaches 14.00 m, leaving tower heights untested; the neighbourhood-indifferent construction means horizontal analyses pay a recomputation cost; and the underground remains a decoder state rather than measured content, so plantability under the pavement is bounded by what the surface reveals, until underground data sources are integrated.

The streaming viewer's memory failure is the defining incident, the one chapter 4 argues from: an area structure of 317 million intervals over 116.5 million columns brought down the visualisation stage on a 32 GB workstation, the post-mortem finding the geometry collector materialising every column key as a Python tuple before a single box had been drawn; the numbered detail is in Appendix C.

The fixes brought everything back to the letter of the design's own principle: keys stay in their packed numpy array (0.9 GB for the same structure), packing became incremental by blocks, and the realisation that no browser should download 10.16 GB per page load produced the working-set viewer of section 4.3.4. The sweep of section 5.2 closes the story: the same structure, regenerated as the sweep's finest raw run, now crosses the redesigned exporter end to end with a per-stage peak under five gigabytes.

Two smaller incidents complete the picture: a tile containing NaN coordinates corrupted the common grid origin with no error reported, until input validation was added before quantisation, and an afternoon was lost to an error no reading of the sources could explain, caused by stale compiled bytecode, a reminder that the environment deserves as much suspicion as the code. The dataset-vintage residue recorded in section 4.3.8 (tile size and coordinate system inherited from the 2018 acquisition) belongs to the same family: every number in this report was re-derived from file headers or run outputs, never taken on trust from comments.

A ray-query incident, a Windows access violation on the finest structure of a study area (15.4 million columns at 0.25 m by 0.1 m cells), took two successive diagnoses before it held; the full account and its fixes, with results proven identical and run time halved, are recorded in Appendix C.

Hardware diagnostics on the development workstation later recorded processor machine-check errors, corrected but recurrent, degenerating into kernel-level faults. The machine itself is therefore a plausible contributor – possibly the whole – of that non-determinism; the mitigation holds under either explanation. The residue: a non-deterministic native fault cannot be proven impossible, so a Metropolis-scale shadow-detection pass should additionally run under the per-chunk process isolation already present in the pipeline – the very mechanism that already contains the LiDAR decoder's native failures – which turns any surviving fault into one lost chunk that resumes; the run itself is not lost. All reported figures come from runs that completed cleanly.

## 5.5 Overall assessment

Against the five objectives of chapter 2, the balance reads as follows.

Single-tile voxelisation (O1) is achieved and tested: construction discards nothing, the equivalence properties hold, and the LAS/LAZ round trip audits cleanly. Compactness at known fidelity (O2) is achieved and quantified: a factor of 55.4 against uncompressed LAS 1.2 point records and a factor of 8.7 against the delivered LAZ tiles, from the run-based structure alone, with further savings from encoders whose semantic cost is measured per class, strongly resolution-dependent (Table 4), and one encoder rejected for analytical use because measurement showed its cost.

Scale (O3) is achieved at the size of the entire Metropolis: the partitioned pipeline processed the complete Grand Lyon inventory, 2,842 tiles and 51.9 billion points, end to end at three resolutions (2.0, 1.0 and 0.5 m), with per-tile bounded memory, resumable fragments, and per-stage failure isolation; the only input defect actually encountered, a tile carrying NaN coordinates, was caught by the validation added before quantisation (section 5.4); the three-kilometre merged class (751.6 million points) and the visualisation class at hundreds of millions of intervals were exercised along the way.

Analysis-readiness (O4) is achieved: maps, statistics, diagnostics, three visualisation paths, a 3D Tiles export that passes the official validator with no errors, and a class-aware ray-query layer with a measured direct-sun-hours experiment (section 4.4.6) all consume the same structure. Two caveats accompany this result, detailed in the table below.

Table 8: The two official-validator reservations on the delivered tileset, their nature and their consequence.

| Validator reservation                     | Nature                                                               | Consequence                                                                                                                                                                                                                                           |
|-------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EXT_mesh_gpu_instancing extension skipped | The validator does not recognise the extension.                      | It carries all the geometry, which is therefore not checked by it.                                                                                                                                                                                    |
| Bounding-volume containment not checked   | A schema blind spot, not a violation; a hierarchical-culling defect. | 556 parent-child pairs out of box in the delivered tileset ("parent" being the bounding volume and "child" the bounded objects), 29 of them significant (beyond 1 mm), fixed by a bottom-up union of the volumes and pinned by a non-regression test. |

Void semantics (O5) is implemented with a stated error model: the three-state decoder distinguishes air from interior and underground, the nadir mitigation removes the main artefact, and the residue is characterised. The claim this chapter supports is precise: for vertical urban queries over classified airborne LiDAR, the run-compressed column structure does the job it was designed for, at the scales the project currently needs, with its trade-offs measured.

## 6 Conclusion and perspectives

Knowing that the final objective is the study of plantability, this internship set out to give the IA.rbre project a volumetric representation of the city, from LiDAR data, exploring a path beyond the classic octree approach: columns independent of their neighbours, and a run-compressed vertical encoding.

Ultimately, the user will have a solid basis for determining computationally the places where it would be advantageous to put trees.

In practical terms, the result obtained is a run-compressed column structure for classified airborne LiDAR, built by a fully vectorised two-step algorithm and held in a structure-of-arrays layout costing about 16 bytes per column and 13 bytes per interval. Around it, the mechanisms that make a data structure usable in practice: a three-state empty-space decoder anchored on a per-column ground index, a measured family of semantic simplification encoders, out-of-core area processing with upfront memory estimation, maps and diagnostics, three visualisation paths, a 3D Tiles 1.1 export, and a class-aware ray-query layer with validated solar geometry and a first direct-sun-hours experiment.

The validation campaign of chapter 5 quantified the most important trade-offs: compression by a factor of 55.4 against uncompressed LAS records and by a factor of 8.7 against the delivered LAZ, from the run-based structure alone, a further thirty percent from gap filling, lossless for plan-view products. A simplification idea born in a supervision meeting was implemented, measured and rejected for analytical use precisely because measurement showed that its plan-view vegetation footprint parts from the raw footprint over twenty-seven percent of their union at the 1.0 m grid, and eighteen percent at 0.5 m, on the canopy the project exists to protect. This report records failures next to results: the memory post-mortem that forced the streaming viewer's design is now part of the design argument.

### 6.1 Against the original framing note

Section 2.4 laid out how this report's five measurable objectives correspond to the five goals of the internship's framing note. This section closes the loop. Table 9 recaps these five goals; the paragraphs that follow detail each row.

*Table 9: Summary of the five framing-note goals: result obtained and status of each goal.*

| # | Framing-note goal                                                            | Result obtained                                                                                                                          | Status                                                                                |
|---|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1 | Semantic voxel model of the city (vegetation, ground, building, underground) | Run-compressed column structure (chapter 4), assessed over the entire Metropolis (chapter 5); a dedicated state reserves the underground | Partially met: the underground stays without data, a sensor limit                     |
| 2 | Construction from existing data                                              | 2,842 tiles, 51.9 billion points, three resolutions                                                                                      | Fully met on LiDAR alone; DSM and CityGML set aside by an argued choice (section 2.4) |
| 3 | Voxel classification                                                         | IGN classes kept; class 8 named upper canopy tier; three empty-space states                                                              | Fully met                                                                             |
| 4 | Volumetric ray-cast shadowing                                                | Class-aware transmittance validated, the volumetric contribution isolated (section 4.4.6)                                                | Fully met, at the scale of one study area; the Metropolis in perspective              |
| 5 | Comparison with 2-D and 2.5-D approaches                                     | 2.5-D overestimation of 9.7 to 11.7 h/day under canopy; canopy volume overestimated by 13 to 54% at 1.0 m (section 4.4.7)                | Fully met; an independent external comparison remains in perspective                  |

Goals 1 to 3 (model, construction, classification) are the subject of chapters 4 and 5; two choices are worth recalling. A single, uniformly classified source, rather than LiDAR joined with DSMs and CityGML, covers 2,842 tiles and 51.9 billion points and characterises the representation over an entire territory, which partially overlapping sources would not allow. The underground remains a measurement limit: the sensor records only the first surface it hits, the delivered classification carries no underground class, and no modelling supplies what the sensor did not measure.

The two application-facing goals were carried out at area scale (goal 4, section 4.4.6) and over ten stratified square kilometres (goal 5, section 4.4.7), their scopes stated: geometric sun hours rather than radiometric irradiance, and a control separating the sampling height from the occlusion model; the values appear in the table above and in section 4.4.7. A comparison with an external product of independent provenance remains to be done, the two reference models being reimplementations of that model on our own data; chapter 5's internal encoder comparison complements it without replacing it.

## 6.2 Perspectives

**Physical:** take the direct-sun-hours computation of section 4.4.6 from sampled areas to the entire Metropolis under per-chunk process isolation, extend it to the diffuse sky component that separates geometric sun hours from irradiance, validate the nadir mitigation against a reference ray caster, rerun the transmittance probe with a ray origin defined relative to the ground surface, which would make the measurement of occlusion as a function of resolution defensible (section 5.2.2), and connect the result to the use cases of the pySunlight line. The structure can serve its own 2-D views: materialising the rasters it derives answers height-field-style queries at raster speed, so the read gap of section 4.4.7 is an implementation cost and not a property of the representation.

**Scale:** first step done, the complete Grand Lyon inventory (2,842 tiles, 51.9 billion points) was processed end to end by the partitioned pipeline at three resolutions; what remains is to close the resolution-grouping series with a full-corpus run at 0.25 m (section 5.2.1), then to promote these one-off

runs into an operated, regularly refreshed pipeline, whose merged structures, assembled maps and streaming viewers are served to the platform rather than produced on demand.

Data: integrating underground sources (networks, soil maps) would replace the decoder's underground state with measured content and unlock in depth the feasibility side of the plantability question; the SBRT-style material modelling [27] studied during the design phase (exact vertical extents, physical attributes per material) is the representation such data would call for.

Temporal: if a new IGN HD LiDAR measurement campaign is organised, two structures of the same territory at different dates will subtract naturally, column by column (subject to identical alignment) into a change map of canopy growth, construction and demolition, moving from a comparison of images to a comparison of structures.

Integration: already partly done, the 3D Tiles export now produces validated tilesets with a level-of-detail pyramid that display in CesiumJS, iTowns and Unreal Engine via Cesium for Unreal, at true Lyon coordinates. What remains is to enrich the tiles with per-instance metadata for click-and-query styling, and to expose the tilesets through the IA.rbre platform; the orthometric geoid offset (+49.70 m near Lyon, from IGN's RAF grid) is now applied automatically on export.

The main objective remains tree plantability. Once the plantable zones are identified, their feasibility will still need confirming against underground networks, sewers or electrical cables, a check the LiDAR data supplied by the Lyon Metropolis cannot carry alone.

## 6.3 Acknowledgements

I thank my laboratory supervisors, John Samuel, Gilles Gesquière, Corentin Gautier and Arthur Villarroja-Palau, for a subject of real substance and for their constant guidance; Didier Puzenat, my academic supervisor, for the principle that a report must argue its choices; the VCity team and the LIRIS colleagues; and the IA.rbre consortium partners, whose platform gives this work its purpose.

## Bibliography

- [1] VCity Team, "pySunlight: Light pre-calculation based on real data (urban data and sun position) with 3D Tiles," GitHub, 2023. [Online]. Available: <https://github.com/VCityTeam/pySunlight>. [Accessed: Aug. 2026].
- [2] TelesCoop and IA.rbre Consortium, "IA.rbre: Intelligence artificielle pour l'arbre - Plateforme web Big Data d'aide à la décision pour la végétalisation urbaine," France 2030 project, 2025. [Online]. Available: <https://iarbre.fr>. [Accessed: Jul. 2026].
- [3] Banque des Territoires, "Démonstrateurs d'IA frugale au service de la transition écologique des territoires (DIAT) - Appel à projets France 2030," Caisse des Dépôts et Consignations, Paris, France, 2023. [Online]. Available: <https://www.banquedesterritoires.fr>. [Accessed: Jul. 2026].
- [4] Institut national de l'information géographique et forestière (IGN), "LiDAR HD: Vers une nouvelle cartographie 3D du territoire," IGN, Saint-Mandé, France, 2023. [Online]. Available: <https://geoservices.ign.fr/lidarhd>. [Accessed: Jul. 2026].
- [5] Métropole de Lyon, "Data Grand Lyon - Open data portal," 2023. [Online]. Available: <https://data.grandlyon.com>. [Accessed: Jul. 2026].
- [6] M. Isenburg, "LASzip: Lossless compression of lidar data," *Photogramm. Eng. Remote Sens.*, vol. 79, no. 2, pp. 209-217, 2013.
- [7] American Society for Photogrammetry and Remote Sensing (ASPRS), LAS Specification 1.4-R15, ASPRS, Baton Rouge, LA, USA, 2019.
- [8] American Society for Photogrammetry and Remote Sensing (ASPRS), LAS Specification Version 1.2, approved by the ASPRS Board 2 Sep. 2008. [Online]. Available: [https://www.asprs.org/wp-content/uploads/2010/12/asprs\\_las\\_format\\_v12.pdf](https://www.asprs.org/wp-content/uploads/2010/12/asprs_las_format_v12.pdf). [Accessed: Aug. 2026].
- [9] Open Geospatial Consortium, 3D Tiles Specification 1.1, OGC Document 22-025r4, 2023. [Online]. Available: <https://docs.ogc.org/cs/22-025r4/22-025r4.html>. [Accessed: Jul. 2026].
- [10] M. Aleksandrov, S. Zlatanova, and D. J. Heslop, "Voxelisation algorithms and data structures: A review," *Sensors*, vol. 21, no. 24, p. 8241, Dec. 2021, doi: 10.3390/s21248241.
- [11] M. Wang and Y.-H. Tseng, "Incremental segmentation of lidar point clouds with an octree-structured voxel space," *The Photogrammetric Record*, vol. 26, no. 133, pp. 32-57, Mar. 2011, doi: 10.1111/j.1477-9730.2011.00624.x.
- [12] B. Gorte, S. Zlatanova, M. Pilouk, A. Diakité, and J. Barton, "3D data integration in the voxel domain," *ISPRS Ann. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. X-4-2024, pp. 133-140, Oct. 2024, doi: 10.5194/isprs-annals-X-4-2024-133-2024.
- [13] B. Gorte, "Analysis of very large voxel data sets," *Int. J. Appl. Earth Obs. Geoinf.*, vol. 119, p. 103316, May 2023, doi: 10.1016/j.jag.2023.103316.
- [14] V. Kämpe, E. Sintorn, and U. Assarsson, "High resolution sparse voxel DAGs," *ACM Trans. Graph.*, vol. 32, no. 4, p. 101, Jul. 2013, doi: 10.1145/2461912.2462024.
- [15] K. Museth, "VDB: High-resolution sparse volumes with dynamic topology," *ACM Trans. Graph.*, vol. 32, no. 3, pp. 1-22, Jun. 2013, doi: 10.1145/2487228.2487235.
- [16] K. Museth, "NanoVDB: A GPU-friendly and portable VDB data structure for real-time rendering and simulation," in *ACM SIGGRAPH 2021 Talks*, New York, NY, USA, 2021, pp. 1-2, doi: 10.1145/3450623.3464653.
- [17] J. C. Hart, "Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces," *Vis. Comput.*, vol. 12, no. 10, pp. 527-545, 1996, doi: 10.1007/s003710050084.
- [18] T. Saito and J.-I. Toriwaki, "New algorithms for euclidean distance transformation of an n-dimensional digitized picture with applications," *Pattern Recognit.*, vol. 27, no. 11, pp. 1551-1565, Nov. 1994, doi: 10.1016/0031-3203(94)90133-3.
- [19] D. E. O. Tzamaras, K. Chow, I. Blanes, and J. Serra-Sagrà, "Fast run-length compression of point cloud geometry," *IEEE Trans. Image Process.*, vol. 31, pp. 4490-4501, 2022, doi: 10.1109/TIP.2022.3185541.

- [20] S. W. Golomb, "Run-length encodings," *IEEE Trans. Inf. Theory*, vol. 12, no. 3, pp. 399-401, Jul. 1966, doi: 10.1109/TIT.1966.1053907.
- [21] D. Abadi, P. Boncz, S. Harizopoulos, S. Idreos, and S. Madden, "The design and implementation of modern column-oriented database systems," *Found. Trends Databases*, vol. 5, no. 3, pp. 197-280, 2013, doi: 10.1561/19000000024.
- [22] H. Aljumaily, D. F. Laefer, D. Cuadra, and M. Velasco, "Point cloud voxel classification of aerial urban LiDAR using voxel attributes and random forest approach," *Int. J. Appl. Earth Obs. Geoinf.*, vol. 118, p. 103208, Apr. 2023, doi: 10.1016/j.jag.2023.103208.
- [23] F. Poux and R. Billen, "Voxel-based 3D point cloud semantic segmentation: Unsupervised geometric and relationship featuring vs deep learning methods," *ISPRS Int. J. Geo-Inf.*, vol. 8, no. 5, p. 213, May 2019, doi: 10.3390/ijgi8050213.
- [24] C. Gaydon, "Myria3D: Deep learning for the semantic segmentation of aerial lidar point clouds," GitHub, Institut national de l'information géographique et forestière (IGN), 2022. [Online]. Available: <https://github.com/IGNF/myria3d>. [Accessed: Aug. 2026]. The segmentation model is a PyTorch Geometric implementation of RandLA-Net (Q. Hu *et al.*, CVPR 2020).
- [25] C. Gaydon, M. Daab, and F. Roche, "FRACTAL: An ultra-large-scale aerial lidar dataset for 3D semantic segmentation of diverse landscapes," 2024, arXiv:2405.04634.
- [26] M. Kress, "LiVoxGen: LiDAR voxelization and generation of canopy metrics," GitHub, Michigan State University, 2015. [Online]. Available: <https://github.com/MeganKress/LiVoxGen>. [Accessed: Jul. 2026].
- [27] A. Peytavie, E. Galin, J. Grosjean, and S. Mérillou, "Arches: A framework for modeling complex terrains," *Comput. Graph. Forum*, vol. 28, no. 2, pp. 457-467, 2009, doi: 10.1111/j.1467-8659.2009.01385.x.
- [28] K. Fujiwara, R. Tsurumi, T. Kiyono, Z. Fan, X. Liang, B. Lei, W. Yap, K. Ito, and F. Biljecki, "VoxCity: A seamless framework for open geospatial data integration, grid-based semantic 3D city model generation, and urban environment simulation," *Comput. Environ. Urban Syst.*, vol. 123, Art. 102366, 2026 (online 2025; volume year 2026), doi: 10.1016/j.compenvurbsys.2025.102366 (arXiv:2504.13934).
- [29] V. Jaillot, F. Pedrinis, S. Servigne, and G. Gesquière, "A generic approach for sunlight and shadow impact computation on large city models," in *Proc. 25th Int. Conf. in Central Europe on Computer Graphics, Visualization and Computer Vision (WSCG 2017)*, Plzeň, Czech Republic, 2017. [Online]. Available: <https://hal.science/hal-01559175>. [Accessed: Aug. 2026].
- [30] Institut national de l'information géographique et forestière (IGN), "FLAIR-HUB: Multi-source semantic segmentation model for land cover mapping," GitHub, 2023. [Online]. Available: <https://github.com/IGNF/FLAIR-HUB>. [Accessed: Aug. 2026].
- [31] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [32] laspy Development Team, *laspy: A Python library for reading, modifying, and writing LAS files*, 2023. [Online]. Available: <https://github.com/laspy/laspy>. [Accessed: Jul. 2026].
- [33] G. M. Morton, "A computer oriented geodetic data base and a new technique in file sequencing," IBM Ltd., Ottawa, ON, Canada, Tech. Rep., 1966.
- [34] J. Amanatides and A. Woo, "A fast voxel traversal algorithm for ray tracing," in *Proc. Eurographics*, Amsterdam, Netherlands, 1987, vol. 87, no. 3, pp. 3-10.
- [35] A. Beer, "Bestimmung der Absorption des rothen Lichts in farbigen Flüssigkeiten," *Annalen der Physik und Chemie*, vol. 86, pp. 78-88, 1852.
- [36] M. Monsi and T. Saeki, "Über den Lichtfaktor in den Pflanzengesellschaften und seine Bedeutung für die Stoffproduktion," *Japanese Journal of Botany*, vol. 14, pp. 22-52, 1953. English translation: *Annals of Botany*, vol. 95, no. 3, pp. 549-567, 2005, doi: 10.1093/aob/mci052.
- [37] G. S. Campbell and J. M. Norman, *An Introduction to Environmental Biophysics*, 2nd ed. New York, NY, USA: Springer, 1998, doi: 10.1007/978-1-4612-1626-1.

- [38] W. Verhoef, "Light scattering by leaf layers with application to canopy reflectance modelling: the SAIL model," *Remote Sensing of Environment*, vol. 16, no. 2, pp. 125-141, 1984, doi: 10.1016/0034-4257(84)90057-9.
- [39] X. Li and A. H. Strahler, "Geometric-optical modeling of a conifer forest canopy," *IEEE Trans. Geosci. Remote Sens.*, vol. GE-23, no. 5, pp. 705-721, 1985, doi: 10.1109/TGRS.1985.289389.
- [40] T. Nilson, "A theoretical analysis of the frequency of gaps in plant stands," *Agricultural Meteorology*, vol. 8, pp. 25-38, 1971, doi: 10.1016/0002-1571(71)90092-6.
- [41] National Oceanic and Atmospheric Administration, Global Monitoring Laboratory, "NOAA Solar Calculator," 2024. [Online]. Available: <https://gml.noaa.gov/grad/solcalc/>. [Accessed: Jul. 2026].
- [42] J. Meeus, *Astronomical Algorithms*, 2nd ed. Richmond, VA, USA: Willmann-Bell, 1998.
- [43] I. Reda and A. Andreas, "Solar position algorithm for solar radiation applications," *Solar Energy*, vol. 76, no. 5, pp. 577-589, 2004, doi: 10.1016/j.solener.2003.12.003.
- [44] M. Blanco-Muriel, D. C. Alarcón-Padilla, T. López-Moratalla and M. Lara-Coira, "Computing the solar vector," *Solar Energy*, vol. 70, no. 5, pp. 431-441, 2001, doi: 10.1016/S0038-092X(00)00156-0.
- [45] J. J. Michalsky, "The Astronomical Almanac's algorithm for approximate solar position (1950-2050)," *Solar Energy*, vol. 40, no. 3, pp. 227-235, 1988, doi: 10.1016/0038-092X(88)90045-X.
- [46] P. Jaccard, "Étude comparative de la distribution florale dans une portion des Alpes et du Jura," *Bulletin de la Société Vaudoise des Sciences Naturelles*, vol. 37, pp. 547-579, 1901. English statement of the same coefficient: "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37-50, 1912, doi: 10.1111/j.1469-8137.1912.tb05611.x.

## Appendices

These appendices carry the material that supports the body of the text without interrupting it: the detailed algorithms of chapter 4 (A), the command-line and module reference (B), the experiment data and serialisation format of chapter 5 (C), supplementary figures (D), the input classification table (E), and the glossary (F).

### Appendix A - Detailed algorithms and pseudo-code

Voxelisation implementation decisions (section 4.2.2), analytical shadow validation (section 4.4.6) and the mechanics of variant v2 (section 4.2.4), moved from the body of the text:

Two implementation-level decisions shape the algorithm's behaviour at scale. The grid origin is fixed once per area, at the grid-aligned corner of the requested bounding box, with the vertical origin at the global minimum of the tile headers, so all tiles quantise against the same origin and their structures concatenate without resampling. And reading is available in a chunked variant that streams the LAZ in memory-bounded chunks and voxelises incrementally; the chunked and whole-file paths are verified to produce identical structures (that equivalence is part of the test suite described in chapter 5), which lets the area pipeline choose its reading strategy on purely memory considerations.

**Validation.** Beyond the solar checks above, the shadow geometry is itself validated analytically: a 20 m tower on flat terrain casts a shadow of height /  $\tan(\text{elevation})$  along the antisolar azimuth. Measured against that closed form at elevations of 20, 30 and 45 degrees and at two azimuths, the computed shadow boundary agreed to within 0.36 m, less than one horizontal cell.

v2 mechanics: the fill merges "tree, missed branch, tree" into a single run (two cells at 0.5 m vertical resolution), never invents a class change, and is the same merge as the production interval-grouping option (store.grouped, section 5.2.1), run at a different gap tolerance.

Exact ceiling bound (section 4.4.2): proof, implementation and verification.

The proof follows from the decoder's own rule (decoder.classify\_gap). An empty voxel is flagged as opaque interior only when every penetrated neighbour is building at that exact height. Above a neighbour's highest occupied voxel, that neighbour is itself empty, hence not building, hence the condition fails and the voxel is measured air. The underground state cannot apply either, since it requires the voxel to sit below the ground index. The bound is therefore a theorem about the decoder, not a tolerance.

It is implemented as voxelizer/ray\_columns.CeilingDDA, a subclass that leaves the reference walker intact. Two cases are distinguished, because only one is valid in general: for a strictly vertical ray, the column never changes and the local ceiling prevails; for an oblique ray, the column changes as the ray advances and only the maximum over the whole structure is safe. The shortcut disables itself whenever the configuration would change the meaning of "nothing above" (air treated as solid, or absent columns treated as solid).

The last row of Table 12, in Appendix C, is the shadow-casting inner loop. One solar position over a million columns goes from about 8.2 hours to a little over 2 minutes, and a forty-position daily arc from about a fortnight to about 1.5 hours. Both projections hold for vertical rays, which keep the per-column local ceiling; an oblique ray leaves its origin column and must use the weaker structure-wide ceiling, and section 4.4.6 gives the measured cost in that regime. The "from ground" row at 1.0 m shows no gain because the baseline already terminated on the first solid voxel; the bound can only help a ray that would otherwise cross air. Figure 21, in Appendix D, plots both walkers as a function of vertical cell size, on the same measurement run.

Verification. `tests/test_ray_columns_equivalence.py` checks results identical to the reference walker's for 2,700 randomly drawn rays, 900 at each of three vertical resolutions, each bounded to a 40 m walk, plus cases written to defeat the bound rather than confirm it: rays starting exactly at the ceiling, just below and just above; configurations that must disable it; isolated columns with no neighbours; and descending rays, which must never take it. All ten tests pass successfully.

Class-aware transmittance (section 4.4.3): parameterisation of the law and resolution invariance.

`voxelizer/transmittance.py` replaces the boolean with a transmittance in  $[0, 1]$  accumulated along the ray by the Beer-Lambert law [35]. Its application to a vegetation canopy follows Monsi and Saeki [36], whose formulation of light extinction in canopies remains the reference. The form implemented here is volumetric extinction: the coefficient carries units of inverse metres and multiplies a geometric path length, where Monsi and Saeki's coefficient is dimensionless and multiplies a leaf-area index; [36] supports the form of the law rather than this parameterisation. The extinction coefficient's dependence on canopy structure is treated as in Campbell and Norman [37]. Two richer families were considered and rejected: full radiative-transfer models such as SAIL [38] require leaf-angle distributions and optical properties LiDAR geometry does not supply, and geometric-optical models [39] require explicit canopy geometry the column structure does not keep by design. The gap-fraction reasoning that makes the coefficient estimable from observed transmission is Nilson's [40], and that is what section 4.4.5 does with ground returns. The law itself reads

**Path length in metres.** Counting voxels would make the answer depend on the grid instead of the geometry: an oblique ray crosses more voxels than a vertical one for the same canopy thickness, and halving the vertical cell size doubles the count without changing the physics. The implementation therefore normalises the direction to unit length and accumulates true path length per class. The parametric distance is thus in metres. A 45-degree ray correctly accumulates  $\sqrt{2}$  metre per metre of height.

This property is verified by an automated test: `test_resolution_invariance` builds the same physical scene (2 m of ground, 10 m of canopy from 4 m to 14 m) at vertical cell sizes of 1.0, 0.5, 0.25 and 0.1 m and requires the resulting transmittances to agree within 0.02. It runs at  $k = 0.1$  and  $k = 0.3$ , which place the transmittance through those 10 m near 0.37 and 0.05, so the tolerance is tight against the first and loose against the second. A voxel-counting implementation passes every other test in the file and fails this one, which is why it is there. Fifteen tests in all, including a closed-form check that  $\tau$  equals  $\exp(-k L)$  for a known thickness, and an equivalence check showing that a purely opaque configuration reproduces the boolean walker's intersection or non-intersection exactly.

This Appendix gives chapter 4's algorithms in compact pseudo-code. Any loop written here for clarity is implemented as a vectorised numpy pass; no per-point or per-column Python loop exists on the construction path. The ray walker of A.3 is the exception, per-voxel Python by construction. A.1 is the main voxeliser (section 4.2.2), A.2 the ground index and empty-space decoder (sections 4.2.3 and 4.2.4), A.3 the column-grid ray walker (section 4.3.5), A.4 the absorption rules and A.5 the overlap resolver (section 4.2.4), and A.6 the streaming construction of the LOD pyramid (section 4.3.4).

## A.1 Two-step voxelisation

```
function VOXELIZE(points, cell_xy, cell_z, origin):  
# step 1: quantisation and reduction (vectorised: sort + reduce; the  
implementation derives the indices from the LAS's raw integer coordinates and  
the header's scale factors, the floating form below being the mathematical  
definition)  
ix = floor((x - x0) / cell_xy); iy = floor((y - y0) / cell_xy); iz = floor((z  
- z0) / cell_z)  
sort the (ix, iy, iz, cls) tuples lexicographically  
reduce equal tuples to unique cells, summing point counts  
# step 2: vertical run-length encoding  
for each maximal run of cells with equal (ix, iy, cls) and consecutive iz:  
emit the interval (iz_first, iz_last + 1, cls, summed counts)  
return the ColumnStore built from the interval list
```

## A.2 Ground index and three-state void classification

```
function GROUND_INDEX(store): # a single vectorised pass in practice  
for each column c:  
g = highest interval of c whose class is GROUND  
ground[c] = g.z_end if g exists else NODATA  
  
function CLASSIFY_GAP(column c, height iz):  
if ground[c] == NODATA: # roof-only or canopy-only column  
return MEASURED_AIR above c's highest return, else OPAQUE_INTERIOR  
if iz < ground[c] - SUBSURFACE_BAND: return SUBSURFACE # SUBSURFACE_BAND = 2  
voxels; the band just under the anchor falls back to the rules below  
if c has at least one penetrated orthogonal neighbour and every penetrated  
neighbour is building at iz:  
return OPAQUE_INTERIOR # nadir-beam mitigation  
return MEASURED_AIR
```

## A.3 Ray traversal over the column grid

```
function TRACE(origin, direction, config): # Amanatides-Woo 3-D DDA  
(ix, iy, iz) = voxel containing origin  
initialise the per-axis steps and next-boundary distances tMaxX, tMaxY, tMaxZ  
while t <= config.max_t:  
state = class of (ix, iy, iz): occupied interval if there is one,  
else CLASSIFY_GAP / subsurface / measured air from the decoder  
if config declares state solid: return hit(position, class, t)  
advance the axis (x, y or z) whose next boundary is nearest  
return no_hit
```

## A.4 Vegetation absorption rules

```
for each vegetation run r in column c: # absorption pass  
if r is not vertically sandwiched by building: keep # real tree, shrub  
else if r is short noise (at most 2 voxels): absorb into the building  
else if fewer than 75% of the 8 penetrated neighbours are building at r's  
heights, or none is penetrated: keep  
else: absorb, unless at least 2 cardinal directions have no building at those  
heights # balcony-tree escape hatch
```

## A.5 Overlap resolution

```
for each voxel carrying superposed intervals of different classes: # resolver
votes = point count per class
if one class wins beyond the tie thresholds: assign it
else: demote the uncertain classes, prefer the taller structure, then the
smaller code
each surviving segment keeps the share of its own run's count that its height
represents;
the losing class's points are discarded, not transferred to the winner
```

## A.6 Streaming construction of the LOD pyramid

```
function BUILD_PYRAMID(leaves):
sort the leaves by MORTON(i, j) # bit-interleaved key: the descendants of any
quadtree node
# form a contiguous range at every level of the tree
for each leaf in that order:
for level L = 1 .. top: if the open node at L changed: FLUSH(L)
write the leaf's GLB; coarsen it by a factor of 2 into the open level-1 node
for level L = 1 .. top: FLUSH(L) # drain: children strictly before parents
function FLUSH(L):
concatenate the open node's parts, sorted by child identity # content
independent of traversal
write the node's GLB; record its bounding volume
coarsen the node by a factor of 2 into its parent at L + 1; free it
# one open node per level: peak memory follows tree depth, not area extent
```

Nadir-beam caveat (section 4.4.1): angle and suspect-band measurements supporting the mitigation.

The recorded scan angles cover  $\pm 30$  degrees, a figure established from the scan-angle field measurements over the delivery's 2,842 tiles, not from a scanner datasheet (the measurement is section 6 of tests/diagtests/genstats/LAZ\_STATS.md: the smallest per-tile minimum is -30.0 and the largest per-tile maximum +30.0, the full envelope rather than a percentile of the distribution); a beam that returns ground at  $(x, y)$  crossed air lying up to  $\tan(30 \text{ degrees}) \approx 0.58$  m laterally per metre of height away from that column, which is several columns at typical tree heights and dozens at tower heights. The visible failure mode is a "light shaft" of air declared phantom immediately next to tall buildings (Figure 9), which would leak light into shadow zones and bias sky-view factors along façades.

The comfortable argument would be that swath overlap and the two-scanner geometry make flat-terrain ground returns come overwhelmingly from near-nadir beams. Most of the crossed air would then be attributed to the column actually scanned; a per-point pass over four adjacent tiles has since tested that argument on the data and does not support it, ground returns lying one to six cells from a building having a mean absolute angle of 12.87 degrees against 13.17 degrees beyond forty cells, and 74% of those immediately bordering a building arriving more than 5 degrees off nadir. The width of the suspect band along a building's edge has since been measured over one square kilometre, by joining the scan angles of near-edge returns to the building heights of the same structure: it is 1.00 m at the median and 2.46 m at the ninth decile, about 0.45 times the building height. The three-to-four-column rule of thumb the design notes carried holds at those heights and fails as a fixed column count.

Post-processes of section 4.2.4: detailed rules.

Three standalone post-processes complete the per-column variants, and all three are bounded exceptions to the neighbourhood-indifference rule: batch operations that temporarily consult neighbourhoods, then return a structure as neighbourhood-free as before. The overlap resolver reduces superposed intervals of different classes (the conservative construction's mixed voxels) to one label per voxel, by majority vote accounting for near-ties: each surviving segment keeps the share of its own run's count that its height

represents, and the losing class's points are discarded rather than transferred to the retained class, which costs 6.33% of the point counts on the reference structure (figures in Appendix C, rules in Appendix A.5).

The absorption pass reclassifies vegetation runs vertically sandwiched and horizontally enclosed by building; greenery apparently growing inside a roof is almost always made of façade returns or classification errors. An escape clause applies when two or more cardinal directions are open, so inner courtyards and green roofs survive. Finally, two denoising filters (a point-count threshold and a morphological same-class-neighbour test) remove isolated-interval noise.

The order in which these post-processes run is not neutral: two sequences differing only in order do not converge, and the absorption pass widens their disagreement in all four measured cases, from +5.8% to +23.5% of divergent columns, over two areas and two resolutions, with points conserved exactly on both sides (Appendix C). Any quantity derived from a post-processed structure must therefore carry the order it came from.

## Appendix B - CLI, module and configuration reference

Dependency roles (section 4.3.1): numpy; laspy [32] with the LASzip engine [6] for LAZ input/output and lazrs as the second-opinion decoder behind isolated failure recovery; matplotlib and Pillow for 2-D rendering; psutil for the memory watchdog and diagnostics; pyproj for coordinate-system validation and 3D Tiles georeferencing; pygltflib for the 3D Tiles export; PyVista and tkinterdnd2 as optional extras for interactive 3-D and drag-and-drop.

Structure internals and persistence (section 4.3.2), as implemented:

The structure-of-arrays layout of section 4.2.1 is realised by six flat numpy vectors, with column keys packed into a single uint64 per column so that sorting and merging operate on one primitive-typed array. The scale factors are folded into the quantisation constants once per tile, so voxel indices are computed directly from the raw integer coordinates stored in the LAS record, avoiding a (floating-representation) round trip over tens of millions of points. Persistence has two forms with distinct roles: a compressed .npz archive for exchange and archiving, and a raw directory holding each array in a separate file for the pipeline itself. The raw form exists because of stage isolation: child processes memory-map the arrays read-only. Output stages can run over one structure without making copies of it, and a failing stage leaves the structure intact on disk.

Mechanisms of the 3-D visualisation paths and the 3D Tiles export (section 4.3.4):

The first is a standalone HTML viewer: interval boxes are exported as instanced geometry in a Three.js scene, one compact record per box, so a single file opens in any browser with no server and no installation; automatic thinning keeps the box count within what a browser tolerates. The voxel payload travels inside the page, but the two Three.js modules (and a decompression fallback for browsers without DecompressionStream) are fetched from a CDN, so the first opening requires a network connection.

The second is a PLY export for CloudCompare, MeshLab or Blender, and an optional PyVista window for interactive inspection during development; the three rendering engines share a single geometry-collection pass, so a fix in the collector benefits every path.

The third, the streaming viewer, exists because the first path has a limit: at fine resolution an area structure can hold hundreds of millions of intervals, and no browser loads that as a single scene.

The streaming exporter therefore traverses the structure one tile-row band at a time (memory stays bounded whatever the structure size) and writes a binary payload of fixed 32-byte instance records next to a spatial index; the viewer treats the configured instance budget as a GPU working set, keeping tiles near the camera resident at full detail while distant tiles collapse into simplified silhouettes.

Chapter 5 relates the memory failure that forced this design, and the vectorised key handling that came out of the post-mortem. A fourth exporter aims at interoperability rather than visualisation: it converts a structure or an existing streaming payload into a 3D Tiles 1.1 tileset [9], with per-instance translation and scale carried by GPU instancing, so the voxel city can be loaded by any standards-conforming geospatial client next to IA.rbre's other layers. Tile elevations are orthometric while the tileset transform expects ellipsoidal heights: the exporter therefore applies by default the geoid undulation from IGN's grid, about +49.70 m near Lyon; if the grid cannot be fetched, the export emits a warning and the model sits some fifty metres too low. The exporter thus builds a quadtree level-of-detail pyramid: 100 m leaf tiles carry full detail, internal nodes carry genuinely subsampled geometry at double voxel size, and the declared error cascade lets any client fetch a roughly 10 MB overview of a 3 x 3 km area before streaming detail only where the camera looks.

By construction, small overlaps can appear at the lowest levels.

The pyramid is built in a streaming fashion: the leaves are traversed in Morton order [33], in which the descendants of any quadtree node are contiguous at every level. Each internal node is written as soon as its last "child" arrives and then freed – peak memory follows the depth of the tree rather than the extent of the area, and halving the leaf size leaves it unchanged. The output passes the official 3D Tiles validator with no errors, subject to the limits section 5.5 records: the validator skips the EXT\_mesh\_gpu\_instancing extension, which carries all the geometry, and it does not check bounding-volume containment at all. Figures 18 to 20, reported in Appendix D, depict the four paths as the code implements them: the geometry pass the first two share, the streaming payload and the viewer that loads it tile by tile, and the 3D Tiles cascade.

The package exposes its functionality through the entry points below; the shared option groups (cell sizes, 3-D geometry options) are factored into a single module so the tools stay consistent. The default cell size is 0.5 m cubed everywhere – the library function, the two command-line tools and the graphical interface – while 1.0 m horizontal by 0.5 m vertical is what section 5.2 recommends for analysis products. The five dependency levels of section 4.1 and their modules are documented in the repository README; the forty-eight modules run from class configuration and the LAZ reader at the base to the CLI and GUI consumers at the top.

| Entry point                                                             | Purpose                                                                                     |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>python -m voxelizer.single TILE.laz</code>                        | Voxelise one tile: four PNG maps, statistics, column diagnostics (optional 3-D)             |
| <code>python -m voxelizer.area_cli area --xmin ... --ymax ...</code>    | Voxelise every tile intersecting a bounding box into a single merged grid, with all outputs |
| <code>... area ... --shard</code>                                       | Same area pipeline, out-of-core: peak memory bounded by a single tile                       |
| <code>... area ... --download / --stream --json INVENTORY</code>        | Fetch the intersecting tiles first, or download-voxelise-delete each tile on the fly        |
| <code>... area ... --preflight-only</code>                              | Predict structure size and peak RAM from headers, then exit                                 |
| <code>... area ... --resume-from-store PATH</code>                      | Rerun the output stages from a cached structure, without redoing voxelisation               |
| <code>python -m voxelizer.viz3d_cli single   from-store   stream</code> | 3-D HTML viewer from a LAZ, from a saved structure, or as a streaming payload               |
| <code>python -m voxelizer.tileset_cli from-store   from-payload</code>  | Export a 3D Tiles 1.1 tileset from a structure or an existing streaming payload             |
| <code>python -m voxelizer.reconstruct to-laz   to-npz   verify</code>   | Inverse voxelisation: structure to LAS/LAZ, LAZ to a structure, or round-trip verification  |
| <code>python -m voxelizer.serve_voxel_html --open</code>                | Serve the latest streaming viewer over HTTP (Range requests supported)                      |
| <code>python -m voxelizer.download_laz / download_orthos</code>         | Download LiDAR or orthophoto tiles from the Grand Lyon inventory                            |
| <code>python -m voxelizer.gui_area</code>                               | Tkinter interface: single-file and area modes, with the upfront dialogue                    |

## Appendix C - Experiment data and serialisation format

Three development records are kept in brief. The conservation defect found while validating the encoders (section 4.2.4): an initial loss of 20.59% of point counts on a real structure for class smooth, brought down to 6.33% by the fix; the return-count bias correction (section 4.4.6) raised the direct-sun hours by 0.122 h. The access violation under ray load: diagnosed in three stages (a real but innocent per-voxel allocation, then the combined volume of binary searches, billions of keys per ray); reusing the current column and reading the flat arrays directly brought searches from thousands to about fifty per ray and halved the runtime, with results reproduced exactly. The streaming viewer's memory failure, decisive for chapter 4: 317 million intervals drove the geometry collector towards 26 GB (keys as Python tuples, a second sorted copy) on a 32 GB workstation; the streaming redesign followed. The successive CLI verification campaigns (30/30, 32/32, 41/41, 14/14, 19/19) and the sweep of section 5.2, forty output stages without failure, stand as the non-regression record.

Grouping and gap fill (v2) of section 4.2.4 are a single operation in the code (store.grouped), run at different gap tolerances on different grids: the -29.5% of Table 4 corresponds to a 1 m tolerance on the three-tile structure at 1.0 m horizontal by 0.5 m vertical, and the -7.01% of section 5.2.1 to a two-cell tolerance at 1.0 m cubed over the entire Metropolis. The two figures thus measure the same merge on different inputs and are not comparable. Bytes per point likewise rests on three bases in this chapter: Table 4 divides the experiment's uncompressed serialisation (12 bytes per column plus 13 per interval) by the input points, Table 5 divides the compressed on-disk .npz, and Table 6 divides the exact payload (16 bytes per column plus 13 per interval). Table 5, in chapter 5, summarises the eight archived runs.

The isolated dense tile 18410\_51825 shows 13.69 intervals per column at the finest resolution, a different regime from the areas: the caution of section 5.2 applies to any comparison.

Choice of the solar algorithm (section 4.4.4): comparison and validation.

The implementation (voxelizer/solar.py) follows the formulation of the NOAA Solar Calculator [41], which is Meeus's low-accuracy solar position algorithm [42]. It is accurate to about 0.01 degree in the vicinity of J2000. Three alternatives were considered:

Three alternatives, NREL SPA [43], PSA [44] and Michalsky [45], offered equal or better precision at the cost of ten times more code or bounded time windows.

NOAA/Meeus sits where accuracy stops mattering for this application, while the complexity stays low enough to implement, read and verify without a dependency, which counts because the design constraints of section 2.1 rule out adding libraries lightly. No solar library is installed in the working environment (pvlib, astral, ephemeris and skyfield are all absent), so the alternative to implementing it was to add one.

**Validation against facts independent of the code.** Internal consistency proves nothing. Each check is therefore made against something derivable without the implementation: the solar-noon elevation follows from latitude and declination by spherical astronomy, and declination cannot exceed the obliquity of the ecliptic (Table 14, Appendix C).

Fourteen tests cover this, including the published extremes of the equation of time (-14.2 min in February, +16.4 min in November), sunrise in the north-east and sunset in the north-west, and the fact that refraction raises the apparent sun rather than lowering it.

**The meridian-convergence trap.** A solar azimuth is measured from true north, while the voxel structure's axes are projected grid axes. At Lyon the meridian convergence is about 1.3 degrees, varying with position (1.28 to 1.41 degrees over the studied areas). Feeding a true azimuth to a grid-aligned ray rotates every shadow by that angle and moves a shadow tip by about 2.3 m at 100 m. `sun_vector_grid` performs the conversion explicitly, deriving the convergence numerically by finite differences (the same technique the 3D Tiles exporter uses for its georeferencing), because closed forms invite sign and hemisphere errors that fail silently.

An error of that kind occurred during development and deserves recording: the two branches of the NOAA azimuth formula were initially inverted, which mirrored the sun across the north-south axis and had sunrise announced at a north-west bearing. No elevation test detects that; only the east/west check did.

Derivation of the extinction coefficients (section 4.4.5): detail and limitations.

The transmittance law of section 4.4.3 left its coefficients unsupplied; deriving them from the corpus fills that gap. LiDAR measures canopy penetration directly: a pulse that reaches the ground through a canopy is a transmitted pulse. For columns showing vegetation above a ground return, the fraction  $p$  of pulses reaching the ground and the canopy thickness  $H$  give  $k = -\ln(p) / H$ .  $H$  is the summed thickness of the column's vegetation intervals above the ground top, in voxels multiplied by the vertical cell size. Gaps internal to the crown are therefore excluded, and  $H$  stays below the crown extent by its gap fraction. Three conditions select the columns, applied as five successive filters: at least 2 m of summed vegetation thickness, at least 20 returns, and the presence of both a ground return and a vegetation return. Applied to the 0.5 m cubed structure this qualifies 739,923 columns, of which the derivation's degeneracy guard then drops seven (Appendix C); the 2 m floor excludes precisely the thin canopies limitation (b) below deals with. The exclusions have since been counted one by one: of the structure's 3,969,467 columns, 3,229,544 are excluded by the five geometric filters and none by the two zero-count guards. The partition is thus exact and reproduces the 739,923 qualified columns, and 291,177 at 1.0 m cubed (Appendix C).

The order is physically sensible and was not imposed: extinction decreases monotonically from low vegetation to the upper canopy tier, which a denser understorey and a sparser crown should produce.

The coefficients are, in  $1/m$ , 0.497 for class 3, 0.429 for class 4, 0.326 for class 5 and 0.250 for class 8, against 0.311 all classes combined (Table 3, in chapter 4).

**(a) It does not predict individual columns.** Fitting on a randomly drawn half and predicting the other half gives a median relative error of 57.7% on the ground-return fraction, with a mean absolute error of 0.120 on it: the fitted coefficient does not predict isolated columns. The interquartile spread within a single class covers nearly a factor of two (class 5: 0.233 to 0.463). The defensible reading is that  $k$  is a *distribution*, usable as a class-typical value for an area-scale computation, and not a per-column property.

**(b) The estimate depends on resolution, so it is not yet a pure physical quantity.** Redoing the fit on the 1.0 m cubed structure gives systematically lower coefficients: class 5 falls from 0.326 to 0.208, class 3 from 0.497 to 0.163 (ratios of 1.5 to 3.1; the coefficient table the sun-hours computation applies, cited with Figure 10 and Table 7, carries class 3 at 0.163, the same fit transcribed to three decimals), and at 1.0 m the class-3 median falls to the bottom of the four, below class 4's and level with class 8's to three decimals (0.1627 against 0.1626), the thin-canopy overestimate at its worst. The mechanism is direct –  $H$  is measured in voxels, and a thin canopy fills a whole 1 m voxel while occupying only part of a 0.5 m one; coarse cells overestimate thickness and therefore underestimate  $k$ . Any quoted coefficient must carry the resolution it was derived at, and shading computations must use the coefficient derived at the resolution they run at.

**(c)  $p$  was estimated from return counts, and the coefficients above carry the correction.** The structure keeps per-interval aggregated counts, not pulse identity, so the first derivation took  $p$  as  $n_{\text{ground}} / (n_{\text{ground}} + n_{\text{vegetation}})$ . A pulse that penetrates the canopy counts once in the numerator and twice in the denominator, and a pulse absorbed in the canopy several times in the denominator alone: both deflate  $p$  and inflate  $k = -\ln(p) / H$ , which is why those coefficients were published as *upper bounds* on the true extinction. The pulses have since been reconstituted from the delivered LAZ by the (source id, GPS time) pair over the whole derivation corpus, 98,946,764 points into 76,743,324 pulses, 1.289 returns per pulse, and Table 3, in chapter 4, is the redone fit: same columns, same  $H$ , same filters,  $p$  taken per pulse. Every coefficient falls and none rises, by 7.4, 6.6, 3.7 and 3.0% for classes 3, 4, 5 and 8 at 0.5 m and by 17.9, 13.6, 7.7 and 6.4% at 1.0 m, the all-class median by 4.3 and 8.8%. The class ordering is unchanged at both grids. Every order claim above therefore survives the correction, and the count-based coefficient exceeds the per-pulse one on 99.978% of columns at 0.5 m and 99.953% at 1.0 m. Its scope has since been fixed by redoing the derivation over the entire delivery (campaign of 2026-08-08, run E12\_per\_pulse\_metropolis, archived outside this delivery): the fit applied at 1.0 m is confirmed to within 3.3, 1.1, 2.6 and 6.6% per class over 174.9 million qualified columns, and the per-column bias ratio is scale-invariant, median 1.078 over the Metropolis against 1.071 here. The aggregate shifts nevertheless grew, to 5.2 and 11.1% against 4.3 and 8.8, for two measured reasons: like the 39.05%, the delivery carries 34.5% of multi-return points (the 19.68% counts non-first returns, a different measure), and the composition of the qualified population shifts, taking class 3 below class 8 at 1.0 m, the mechanism of limitation (b) at scale (Appendix C).

**(d) The coefficient is derived from a near-nadir survey and applied as is to oblique rays.**  $H$  is a vertical thickness measured from pulses descending essentially vertically, while the sun-hours run of section 4.4.6 sends the same coefficient all along the daily arc; for any non-spherical leaf-angle distribution, the extinction coefficient itself depends on the beam's zenith angle, a dependence for which Campbell and Norman [37] is cited, and no angular correction is applied.

Figure 22, in Appendix D, shows the fitted distribution per class. The count-based derivation is reproducible from the repository: script Experiments/derive\_extinction.py, fit Experiments/extinction\_fit.json; the per-pulse fit that replaces it comes from the campaign of 2026-08-08 (run E10\_per\_pulse\_refit\_scopeA), archived outside this delivery.

Direct sun hours (section 4.4.6): ray-origin rules, oblique-ceiling correction and native incident.

For each column and each solar position above the horizon, a ray is cast from 1.5 m above the top of the column's lowest interval towards the sun, and transmittance is accumulated as  $\text{sun\_hours} = \sum(\tau * dt)$ . Where a column has a ground return, that origin is 1.5 m above the ground, roughly eye height; where it has none – typically under a roof – it is 1.5 m above the lowest measured surface. Those columns therefore report roof-level rather than street-level sun hours. The rule can also anchor from below: when a column's lowest interval is a low class-7 outlier below the terrain, the origin follows it there, and the ground index the structure already carries is not consulted. Computing this twice – once with all materials opaque, once with the coefficients of section 4.4.5 – isolates what the volumetric material model contributes. Neither map alone answers objective (iv); it is the difference between them that does.

These figures follow from the oblique-ceiling correction applied to the transmittance walk. The walk now receives the structure-wide ceiling, valid in every direction, in place of the origin column's local ceiling, which an oblique ray leaves before it can meet the geometry standing further down its road. Every absolute sun-hours value falls by one and a half to two hours, none of the 48,000 sampled column-model pairs increased, and the difference the experiment exists to measure keeps its sign and its ordering (Experiments/sun\_hours\_ceilingfix/). This ceiling also sets the price of the run: 1.54 ms per ray, the mean of three element-wise identical repeats, under the derivation's original coefficients. At that original 1.54 ms rate, one solar position over the area's 967,432 columns costs about 25 minutes in this regime, and the full daily arc about 26 hours, rather than the two minutes per position the vertical-ray projection of section 4.4.2 gives (Appendix C); under the applied per-pulse fit, the same walk measures 1.86 and 1.67 ms per ray over the two areas, a weaker extinction letting rays carry further. The arc over the vegetated area is thus closer to 31 hours.

The full account of this native failure and its fixes, with results proven identical and run time halved, appears higher in this Appendix, among the chapter 5 incident accounts; the hardware finding that appeared later and the belt-and-braces measure for Metropolis-scale runs are recorded in section 5.4.

LAS/LAZ export modes, protections and cost study (section 4.3.7):

Section 4.3.7 summarises the three export modes (density, one\_per\_voxel, exact) and their guarantee; what follows records the detail the body omits.

In exact mode, point density and voxel geometry both survive. It emits one point per voxel of the interval, restoring occupancy and class, plus count - ( $z_{\text{end}} - z_{\text{start}}$ ) extra points stored in the interval's first voxel, restoring the count.

This fill placement looks arbitrary and is in fact the only one the data allows: the voxeliser sums point counts per interval and never per voxel. The fill's position within the interval is unobservable to a re-voxelisation, and the structure never held the original per-voxel distribution that would allow spreading them. Storing them at the interval base is the cheapest correct choice and, unlike a uniform spread, the convention is made explicit so as not to present a constructed value as an observation.

Two design decisions support the guarantee. First, the grid travels inside the file. A re-voxelisation reproduces a structure only if it uses the same origin, and an origin derived from the reconstructed cloud lands within half a cell, because that cloud's minimum coordinate is a voxel centre and not a voxel corner; the file therefore carries a proprietary 52-byte IARBRE record holding the origin, the two cell sizes and the EPSG code, so a reconstruction needs no side file and cannot be separated from its geometry by a copy or an upload. Both halves are pinned: one test verifies the half-cell drift that motivates the record, another reconstructs a structure from a file with no arguments at all, and a third refuses to guess when the record is absent.

To the two protections summarised in 4.3.7 (the grid travels inside the file; only raw structures are archived) is added per-fragment verification, a second line of defence, and deleting a source structure is refused until that verification is enabled. Fifty tests spread across tests/test\_reconstruct\_roundtrip.py and tests/test\_archive\_cli.py assure conformance and each of the protections around it, including the

negative cases: a hand-assembled structure that is not the canonical encoding of its own cell set must fail, because exactness is a property of what the voxeliser emits and not of any well-formed structure.

Two header details deserve recording because both were real defects. The writer module emits a coordinate reference system, which looks like housekeeping and is not: the package's own LAZ reader rejects files that lack one, so before the existence of this parameter, nothing the reconstruction path produced could be fed back into the pipeline. And the classification field is only five bits in LAS point formats 0 to 5: IGN's custom codes 64 to 67 overflowed inside the LAS library without raising anything a caller could read. Files whose classes exceed 31 are now promoted to LAS 1.4 and point format 6, where classification occupies a full byte, and a test also pins the negative case, requiring that ordinary Grand Lyon data, whose declared class range runs from 0 to 22, does not change format unintentionally.

Coordinate quantisation is a happier accident: the scales stay at the millimetre with the offset placed at the minimum coordinate, a voxel centre for a reconstructed cloud. Every exported coordinate therefore differs from the offset by an exact multiple of the cell size, and the worst-case rounding error, about half a millimetre, stands against a quarter-metre margin to the nearest voxel boundary. Millimetre quantisation is therefore lossless for this content, not merely tolerable, which is why the two-way conversion survives a real LASzip file and not just an in-memory expansion.

The cost remains. The intuition that an exact LAZ must be larger than the structure it encodes – it does, after all, name every voxel individually – held for the first measurement made and turned out to be a property of that measurement rather than of the format. Generalising from a single tile at a single resolution would lead to concluding the archive saves nothing; that is true for that tile at that resolution, and false everywhere else. Table 10 gives the corrected picture: three real tiles from the 2,842-tile Grand Lyon inventory, the densest of the delivery and the two sparsest, each voxelised at three resolutions, with archive size expressed as a multiple of the structure it replaces. Each of the nine cells was moreover verified as the exact conversion to the bit. Table 10, reported in Appendix C, thus measures cost alone and not a trade against fidelity.

The ratio moves with resolution because the two artefacts are characterised by different quantities. The structure costs a roughly fixed amount per interval, and the unit here is the compressed on-disk byte rather than the 13-byte in-memory record: on the largest tile, the .npz stands at 3.66, 3.60 and 4.00 bytes per interval across the three resolutions. Its size tracks the interval count closely, and the interval count is what a finer grid multiplies – 1.34 million, then 5.67 million, then 25.9 million on that tile, factors of 4.2 and 4.6.

The archive does not follow. Over the same two refinement steps it grows only by respective factors of 2.9 and 2.8, because the number of points it must write is fixed by the source tile and never changes: refining the grid adds no points, it only converts fill duplicates into distinct voxel centres. The duplicates cost almost nothing, about 0.02 byte each, since the compressor's predictor codes identical consecutive points at practically zero, and the voxel centres they become are themselves consecutive and cheaply predicted. The structure's cost therefore grows with the grid while the archive's does not, and the two curves cross.

At 0.25 m horizontal and 0.1 m vertical, the archive was smaller than the structure on all three tiles, by factors of 1.5 to 3.4. At the project's 0.5 m default, the two are near parity overall – the projection for the whole delivery is about 34 GB of archive against about 33 GB of fragments, that fragment figure being the sum of 2,842 separately compressed per-tile files where Table 6's 20.41 GB forms a single merged structure; the per-file container overhead and the dictionaries unshareable across fragments explain the gap, which is not decomposed here – but the mean hides a spread: the archive costs 17% more on the densest tile and saves on the two sparse ones.

At 1 m, the case must be measured run by run (section 4.3.7).

Calibration of the upfront estimator (section 4.2.5): detailed fit and validation.

Its two shape constants are now calibrated on measurement. The occupancy fraction and the intervals per column are given by power laws in horizontal cell size, fitted on thirty of the forty structures: ten 2 km by 2 km areas voxelised at four grids (Appendix C). The fourth grid is a control that refines the vertical cell alone by a factor of 2.5: intervals per column move by 0.05%. The model carries no vertical-cell term.

Validation covers eight cases held out of the fit: the six metropolitan runs of section 5.2.1, whose 2.0 m grid falls outside the fitted range, and the equivalence pair of section 5.1. The central estimate stays within -3.9% and +9.3% of the measured size, and the planning figure, safety margin included, 1.14 to 1.30 times above that size and never below any of the eight, where the previous constants underestimated all eight by 22 to 46%.

At 2.0 m, the grid where the old estimate was dangerous, the calibrated model predicts 19.0 GB against 19.7 GB observed; at fine grids it describes the merged structure resident in a single process, which the partitioned mode exists precisely to avoid. There the figure is a ceiling, not an expectation.

Memory layout of the structure (section 4.2.1): the next two paragraphs record the redesign and its numbered justification.

The way intervals are held in memory was rethought once, and that redesign is an argued choice worth recording. The first implementation stored a Python dictionary mapping `(ix, iy)` tuples to `Column` objects, each owning small `numpy` arrays: true to the language's idioms, readable, and ruinous at scale, with tens of millions of Python objects, pointer overhead dwarfing the payload, and a dictionary whose keys alone consumed gigabytes (chapter 5 returns to the 116-million-column incident that settled the question).

The production layout is a structure-of-arrays: six flat `numpy` vectors (column keys packed as `uint64`, per-column offsets as `int64`, interval bounds as `int32`, classes as `uint8`, counts as `int32`) giving 16 bytes per column plus 13 bytes per interval, a measured factor-16 reduction over the object form. That figure is measured, not estimated: 208.7 bytes per `Interval` dataclass instance under `tracemalloc` against 13 bytes in parallel `numpy` arrays, a ratio of 16.1; an earlier `sys.getsizeof` estimate (factor 30 to 50) double-counted the instance dictionary CPython amortises across instances.

On tile 18410\_51825 (60.9 million points), the structure at 1.0 m by 0.5 m cells holds 3.42 million intervals, a factor-17.8 reduction; the ratio runs from 10.7 for ground (thin runs spread over many columns) to 44 for buildings (solid blocks with few class breaks), and the mean of 13.7 intervals per column, against the metropolitan means of 1.78 to 2.63 in Table 6, measures the fragmentation caused by vertical surfaces. Table 10 counts the same tile at isotropic grids, which is why its figures differ.

The encoder experiments of section 5.2 serialise each structure into a compact binary payload: per column, a header (`ix` `int32`, `iy` `int32`, `n_intervals` `int32` – the indices are signed, and negative near a grid origin) followed by the interval arrays (`z_start` `int32`, `z_end` `int32`, `class` `uint8`, `count` `int32`, each repeated `n_intervals` times). The three test tiles were 18410\_51755, 18410\_51760 and 18415\_51760 (20.9 to 27.1 million points; classes 1 to 6, the class-8 upper canopy tier, and water 9 for the first two). The campaign delivers into the repository its manifests, its metrics table and the plan views reproduced in chapter 5; the bulky working data is archived outside the delivery.

### C.1 Supporting tables for chapters 4 and 5

The tables below carry the numbered detail chapters 4 and 5 summarise; their numbering and the body's cross-references are unchanged.

Table 10: Exact LAZ archive size expressed as a multiple of the .npz structure it encodes, for three real Grand Lyon tiles at three resolutions. Below 1.00, the archive is smaller than the structure. All nine cells were verified byte-for-byte identical on conversion (SHA-256 of the re-saved structure).

| Tile        | Character                        | Points     | 1.0 m cubed | 0.5 m cubed | 0.25 m xy / 0.1 m z |
|-------------|----------------------------------|------------|-------------|-------------|---------------------|
| 18410_51825 | dense (largest of the 2,842)     | 60,937,237 | 1.65        | 1.17        | 0.65                |
| 18385_51830 | sparse                           | 16,864     | 0.62        | 0.37        | 0.29                |
| 18380_51835 | sparsest (smallest of the 2,842) | 7,676      | 1.14        | 0.82        | 0.50                |

Table 11: Cost of a climbing ray, before optimisation (100-column samples, 3 repetitions; data Experiments/ray\_benchmark.json).

| Structure          | Ray starting above the column top | Ray starting from the ground |
|--------------------|-----------------------------------|------------------------------|
| 1.0 m cubed        | 6.27 ms                           | 0.0033 ms                    |
| 0.25 m xy, 0.1 m z | 57.10 ms                          | 29.35 ms                     |

*Table 12: Cost of a climbing ray, after the ceiling bound (same measurement run as Table 11). The ratios track the timer's precision on the CeilingDDA side and are to be read as orders of magnitude, like the times they come from.*

| Structure       | Start                | Baseline  | CeilingDDA | Speed-up |
|-----------------|----------------------|-----------|------------|----------|
| 1.0 m cubed     | above the column top | 6.27 ms   | 0.0005 ms  | 12,547   |
| 1.0 m cubed     | from the ground      | 0.0033 ms | 0.0050 ms  | 0.7      |
| 0.25 xy / 0.1 z | above the column top | 57.10 ms  | 0.0005 ms  | 114,207  |
| 0.25 xy / 0.1 z | from the ground      | 29.35 ms  | 0.1363 ms  | 215      |

*Table 13: Behaviour on real structures. The three structures are one and the same 1 km by 1 km box in eastern Lyon (bbox 1841000, 5175500 to 1842000, 5176500; the four 500 m tiles, 98,946,764 points) voxelised at three grids, so the rows differ only in resolution. 600 columns sampled per structure, vertical sky-facing rays, coefficient applied to classes 3, 4, 5 and 8, rays bounded by the per-column ceiling of section 4.4.2; data Experiments/transmittance\_probe.json.*

| Structure       | k_veg  | mean $\tau$ | Fraction $\tau > 0.5$ | with | Mean canopy path (m) | ms/ray |
|-----------------|--------|-------------|-----------------------|------|----------------------|--------|
| 1.0 m cubed     | opaque | 0.0000      | 0.0000                |      | 0.034                | 0.004  |
| 1.0 m cubed     | 0.1    | 0.0293      | 0.0333                |      | 0.459                | 0.007  |
| 1.0 m cubed     | 0.3    | 0.0137      | 0.0083                |      | 0.459                | 0.006  |
| 1.0 m cubed     | 0.8    | 0.0045      | 0.0017                |      | 0.449                | 0.006  |
| 0.5 m cubed     | opaque | 0.4400      | 0.4400                |      | 0.117                | 0.029  |
| 0.5 m cubed     | 0.1    | 0.5768      | 0.5967                |      | 1.049                | 0.039  |
| 0.5 m cubed     | 0.3    | 0.5193      | 0.5117                |      | 1.049                | 0.037  |
| 0.5 m cubed     | 0.8    | 0.4752      | 0.4600                |      | 1.049                | 0.038  |
| 0.25 xy / 0.1 z | opaque | 0.6500      | 0.6500                |      | 0.023                | 0.143  |
| 0.25 xy / 0.1 z | 0.1    | 0.8145      | 0.8350                |      | 0.592                | 0.189  |
| 0.25 xy / 0.1 z | 0.3    | 0.7693      | 0.7717                |      | 0.592                | 0.188  |
| 0.25 xy / 0.1 z | 0.8    | 0.7271      | 0.7250                |      | 0.592                | 0.195  |

Controlled cases, gap fill with a one-metre tolerance, before the correction:

| Case                      | Input counts   | Output       |
|---------------------------|----------------|--------------|
| same class, one-voxel gap | $20 + 30 = 50$ | 30           |
| same class, contiguous    | $20 + 30 = 50$ | 30           |
| same class, superposed    | $20 + 30 = 50$ | 30           |
| different classes, gap    | $20 + 30 = 50$ | 50 (correct) |

*Table 14: Solar geometry at Lyon (45.7647 N, 4.8386 E) confronted with theory. The predicted column takes declination as exactly 0 or exactly plus or minus 23.44 degrees on the calendar date, which it is not at the instant of local solar noon, so the 0.04-degree residuals at the equinox and the December solstice are properties of the reference value rather than of the implementation.*

| Date              | Predicted noon elevation           | Computed      | Computed day length |
|-------------------|------------------------------------|---------------|---------------------|
| June solstice     | $90 - 45.76 + 23.44 = 67.68^\circ$ | $67.67^\circ$ | 15.67 h             |
| Equinox           | $90 - 45.76 = 44.24^\circ$         | $44.20^\circ$ | 12.08 h             |
| December solstice | $90 - 45.76 - 23.44 = 20.80^\circ$ | $20.84^\circ$ | 8.58 h              |

## C.2 Complementary measurements

The measurements below were carried out after chapters 4 and 5 were written, to close the points they left open. Each is verified by exact integer identities rather than tolerance comparisons; the logs and SHA-256 digests are kept with the outputs, outside the delivery.

Structures and grids. Ten 2 km by 2 km areas were drawn by a recorded seed from the 2,287 blocks whose sixteen tiles are available locally, under a 3 km minimum separation, then voxelised at four grids: 1.0 m, 0.5 m and 0.25 m cubed, and 0.25 m horizontal by 0.1 m vertical. All forty structures completed, each at the first attempt, in 4 h 24 min of continuous execution, for 3,546,676,808 points per grid. Two hundred exact checks pass: sixteen fragments present with no missing tile, fragment point-count sums equal to the merged total, no column outside its area's lattice over 1,388,297,158 columns, no malformed interval or zero count over 2,124,475,488 intervals, and statistics written from the fragments agreeing with the merged structure.

Intervals per column are set by the horizontal cell alone. They are 2.116 at 1.0 m, 1.776 at 0.5 m and 1.480 at 0.25 m, and 1.481 at the fourth grid, which refines the vertical cell by a factor of 2.5 at equal horizontal cell. Columns and intervals per grid are 38,045,963 and 80,495,729 at 1.0 m cubed, 151,496,569 and 269,071,024 at 0.5 m, 599,377,313 and 887,229,468 at 0.25 m, and 599,377,313 and 887,679,267 at the fourth grid. The column set there is moreover byte-for-byte identical to the 0.25 m cubed one for all ten areas, and the total interval count rises by only 0.051%. The occupancy fraction is 0.951, 0.947, 0.937 and 0.937, against the fixed 0.6 the old estimator assumed, with a per-area range of 0.690 to 1.000.

Point density and vertical structure do not go together. The ten areas' density runs from 38.8 to 129.2 points per square metre, a factor of 3.3, while intervals per column at 0.5 m run from 1.461 to 2.046; the densest area is among the vertically simplest. A chapter 5 figure drawn from a single area therefore carries an unstated factor of three in density and a factor of 1.4 in vertical structure, which is the argument for the ten-area draw.

Extinction column selection. The exclusion count of section 4.4.5 breaks down as follows at 0.5 m, over 3,969,467 columns: 1,571,325 with no ground return (39.59%), 1,357,755 with no vegetation (34.20%), 41,217 with no canopy above the ground (1.04%), 233,222 under the 2 m floor (5.88%), 26,025 under the 20-return threshold (0.66%), and none by the two zero-count guards; 3,229,544 excluded, 739,923 qualified, 18.64%. At 1.0 m, of 999,170 columns, 707,993 are excluded and 291,177 qualified, 29.14%. The fit itself covers 739,916 columns at 0.5 m and 291,053 at 1.0 m: for the others, the per-pulse ground fraction is not strictly between 0 and 1 and the degeneracy guard the derivation already applies excludes them. The 2 m floor excludes proportionally 2.4 times more columns at 0.5 m than at 1.0 m, so the 1.0 m fit does not cover a subsample of the 0.5 m columns but a differently constituted population: that is the mechanism of limitation (b) of section 4.4.5.

Per-pulse census. The pulses were reconstituted from the LAZ by the (source id, GPS time) pair, first over boxes B04 and B10 at 1.0 m cubed, then over the whole derivation corpus. On the two boxes, 347,946 and 60,782 qualified columns, the identity holds: the number of points read equals, to the unit, the structure's, 65,513,414 and 45,753,292, and per-column ground and vegetation returns equal the structure's with no divergent column; the ratio of the count-based to the per-pulse coefficient is 1.050 at the median on B04 and 1.098 on B10, 1.222 and 1.413 at the ninth decile. On the derivation corpus, 98,946,764 points group into 76,743,324 pulses, 1.289 returns per pulse, with 49 colliding groups, a  $6.4e-7$  rate that excludes them from the fit. The count-based estimator is first reproduced there field for field, 41 fields at each grid and no gap, so the redone fit corrects an object it first reproduces. The per-column median ratio is 1.0300 at 0.5 m and 1.0710 at 1.0 m, the second value falling between the two boxes' values and the first below both, and the count-based coefficient exceeds the per-pulse one on 99.978% and 99.953% of columns. The bias remains strongest for class 3 and weakest for class 8. Finally, 39.05% of the derivation block's points are multi-return against 34.5% on the delivery measured the same way, about 1.13 times, which explains part of the shift's size. The 19.68% cited elsewhere counts non-first returns on the delivery, a different measure, which compares to neither.

Scan angles near edges. A per-point pass over the equivalence measurement's four tiles, 52,705,942 points and 211,221 building columns, joins the absolute scan angle to the Chebyshev distance to the nearest building column. Ground returns one to six cells from a building have a mean absolute angle of 12.87 degrees against 13.17 degrees beyond forty cells: the gap has the expected sign and is negligible, and 74% of ground returns immediately neighbouring a building arrive more than 5 degrees off nadir, so the plus-or-minus-5-degree figure the design notes carried is not supported. The resulting suspect width is 1.00 m at the median, 2.46 m at the ninth decile and 3.67 m at the ninety-ninth, and it grows with building height, which this survey carries at 4.50 m at the median and 14.00 m at the highest.

Upfront estimator. The six metropolitan runs of section 5.2.1 were compared with their recorded resident-memory peak, by two analysers sharing no code and agreeing exactly on all six readings. The old model underestimated the peak by 35% at 2.0 m and overestimated it by a factor of 3.7 at 0.5 m,

two errors of opposite sign: the too-low assumed occupancy fraction pushes down, whole-structure residence in a single process pushes up. The calibration laid out in section 4.2.5 follows from it.

Sun-load repeats. The 756,000-ray load of section 4.4.6 was repeated three times. All three repeats give the same per-column hours for the 12,000 columns, element by element, and also reproduce the recorded run's: four independent computations, one cut by column blocks and two cut by solar position, arrive at the same values. The mean of the times is 1,165.2 s and the spread 2.0% of that mean, against 1,295.7 s for the recorded run. The cost per solar position runs from 2.6 s at the horizon's edge to 25.6 s at the most expensive, and the shape reproduces position for position from one repeat to the next.

Post-process order. All four cross order comparisons now exist. Points are conserved exactly in all four, the structures coincide in none, and the absorption pass widens the disagreement everywhere: from 55,391 to 61,769 divergent columns on the first area at 0.5 m, from 45,263 to 47,869 at 0.25 m (xy) / 0.1 m (z), from 48,312 to 59,660 on the second area at 0.5 m and from 35,595 to 39,735 at 0.25 m (xy) / 0.1 m (z), that is +5.8% to +23.5%.

### C.3 Comparison with 2-D and 2.5-D approaches

The comparison of section 4.4.7 covered ten 1 km squares of the Grand Lyon inventory, stratified by built fraction from 0.2 to 25.9%, at 1.0 m and 0.5 m cubed: twenty structures and 694,428,978 points. The models were verified on four 40-by-40 synthetic scenes, three in agreement and the canopy-above-ground diagnostic, each model computed twice with identical results. On a horizontal plane, on solid blocks and on a closed elevated ring with 80.0% of columns under the full day, 0 columns of 1,600 differ, and on the plane both models render the closed form 15.75 hours – 63 solar positions at a quarter hour – for every column. The fourth scene, a canopy over ground, is the one where they must diverge, and it separates the two mechanisms: of the 400 columns carrying a canopy above ground, 395 differ and model S reads the full day on every one of them, while of the 1,200 single-body ground columns, 828 differ and model S is lower by 1.42 hours on average, because filling the measured air under a neighbouring crown blocks the grazing rays that pass beneath it in the structure.

Direct sun hours per model, on the paired columns of the vegetated area B04 and the built area B10. Each 0.5 m column is one of the four covering a drawn 1.0 m key, so both grids are read over the same ground; 3,000 columns per area and per grid, summer solstice, 15-minute sampling. The last row gives the share of columns on which the two models render exactly the same value.

| model                                             | B04, 1.0 m | B04, 0.5 m | B10, 1.0 m | B10, 0.5 m |
|---------------------------------------------------|------------|------------|------------|------------|
| column structure, opaque                          | 4.4408 h   | 5.6997 h   | 10.4015 h  | 11.4364 h  |
| column structure, class-aware                     | 6.2593 h   | 6.7399 h   | 11.1239 h  | 11.8814 h  |
| surface model (S)                                 | 9.7322 h   | 9.4548 h   | 12.7352 h  | 12.7349 h  |
| terrain-sampled control (S2)                      | 4.1293 h   | 5.3190 h   | 10.2286 h  | 11.2146 h  |
| model S minus the opaque structure                | +5.2914 h  | +3.7550 h  | +2.3337 h  | +1.2985 h  |
| model S2 minus the opaque structure               | -0.3115 h  | -0.3808 h  | -0.1729 h  | -0.2218 h  |
| columns where model S equals the opaque structure | 35.1%      | 43.4%      | 71.3%      | 75.8%      |

The other queries over the ten areas. Canopy volume is the volume the structure measures relative to the volume the height field implies, footprint times object height; gap columns carry two or more occupied bodies separated by measured air, which a height field cannot represent; top class gives the agreement

between the structure's highest interval and the class of the same cell's highest raw point. Four readings complete the picture: the structure measures 0.65 to 0.88 of the canopy volume the height field implies at 1.0 m, so footprint times height overestimates by 13 to 54%; almost every top-class disagreement is a pair of neighbouring classes inside one voxel; crown base height reads 1.0 to 1.7 m higher at 0.5 m, so a planting-clearance question asked at a 1 m grid inherits about a metre and a half of bias; and the same pass recovered the ten areas' raw point count exactly, an end-to-end conservation check over 694,428,978 points.

| area | built | vegetation | canopy volume, 1.0 m | 0.5 m  | gap columns, 1.0 m | 0.5 m  | top class, 1.0 m | 0.5 m  |
|------|-------|------------|----------------------|--------|--------------------|--------|------------------|--------|
| B01  | 0.2%  | 22.3%      | 0.8675               | 0.4936 | 9.26%              | 13.47% | 94.76%           | 97.99% |
| B02  | 0.7%  | 44.9%      | 0.8823               | 0.5188 | 16.70%             | 30.78% | 86.61%           | 95.03% |
| B03  | 2.4%  | 32.6%      | 0.7586               | 0.4518 | 18.18%             | 22.69% | 95.63%           | 98.31% |
| B04  | 3.7%  | 58.6%      | 0.6512               | 0.3457 | 40.04%             | 40.48% | 94.92%           | 97.94% |
| B05  | 6.7%  | 49.4%      | 0.7935               | 0.4609 | 30.21%             | 38.76% | 95.01%           | 97.78% |
| B06  | 8.3%  | 23.5%      | 0.7821               | 0.4368 | 17.59%             | 18.22% | 97.33%           | 98.89% |
| B07  | 11.9% | 37.3%      | 0.7432               | 0.3955 | 26.18%             | 31.31% | 94.24%           | 97.91% |
| B08  | 13.9% | 12.2%      | 0.7906               | 0.4492 | 11.70%             | 15.76% | 97.21%           | 98.86% |
| B09  | 19.8% | 22.6%      | 0.7498               | 0.4201 | 19.20%             | 23.83% | 96.41%           | 98.61% |
| B10  | 25.9% | 7.9%       | 0.7734               | 0.3869 | 8.53%              | 10.95% | 98.32%           | 99.42% |

Cost, measured head to head on one and the same square kilometre (x from 1,845,000 to 1,846,000, y from 5,175,000 to 5,176,000, EPSG:3946). The acquisitions differ – the raster reference is the 2018 delivery, 12,156,432 points, while our corpus of the same square is the 2023 delivery, 8.46 times denser – so the head-to-head runs our voxelisation on the 2018 file, which makes the point sets identical.

| axis                                                                               | measure                                                | what it quantifies                |
|------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------|
| build, our voxelisation on the 2018 tile (12,156,432 points)                       | 2.50 s at 1.0 m, 3.58 s at 0.5 m                       | our implementation                |
| build, the point-by-point reference loop, same points                              | 18.65 s                                                | the reference implementation      |
| build, identical on hole filling, identical output                                 | 0.54 s against 0.478 s for ours                        | the algorithm                     |
| build, our voxelisation on our 2023 corpus of the same square (102,896,384 points) | 117.4 s at 1.0 m, 99.0 s at 0.5 m                      | our implementation on denser data |
| storage, the four LZW GeoTIFFs                                                     | 7,310,972 B                                            | the raster representation         |
| storage, our structure at 1.0 m                                                    | 6,474,827 B, 0.886 times the rasters                   | the volumetric representation     |
| storage, our structure at 0.5 m                                                    | 20,807,936 B, 2.846 times the rasters                  | the volumetric representation     |
| query, one scalar read per cell                                                    | raster 0.231 us, structure 5.064 us, 21.9 times slower | both representations              |
| query, the vertical questions                                                      | the raster cannot answer them                          | both representations              |

Two of these rows quantify implementations rather than representations. The reference build is a Python loop over every point, and two concessions worked in its favour: a progress display absent from the environment was replaced by simple iteration, which makes the loop faster than in its shipped version, and its four raster writes are excluded from its total while ours includes writing the structure. On the fragment both pipelines run to identical output, hole filling, they are within a factor of 1.1 of each other. The storage result has a mechanism behind it: the structure carries 999,736 columns and 1,955,696 intervals for this square kilometre, just under two runs per column, so vertical run-length encoding does real work on a surface that is most often a single solid body per cell.

## Appendix D - Supplementary figures

Figures 18 to 23, cited in chapters 4 and 5, are gathered here: the three export paths of section 4.3.4, the ray-cost measurement of section 4.4, the extinction distribution of section 4.4.5, and the complete import graph of section 4.1.

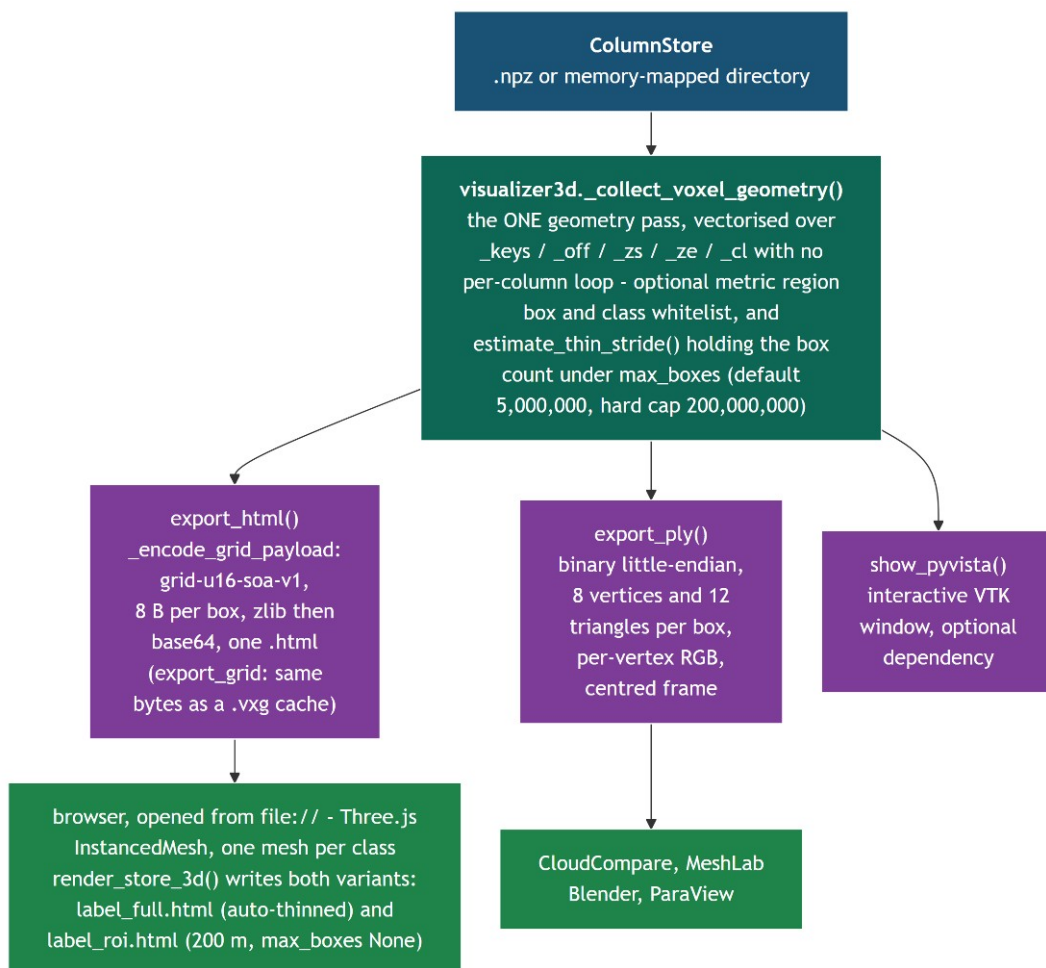


Figure 18: The in-process export paths: `export_html`, `export_ply` and `show_pyvista` are terminal encoders on top of a single call to `visualizer3d._collect_voxel_geometry`. The thinning budget and the region filter thus apply identically to all three. The HTML file carries its voxel payload inline and loads only its two Three.js modules from a CDN.

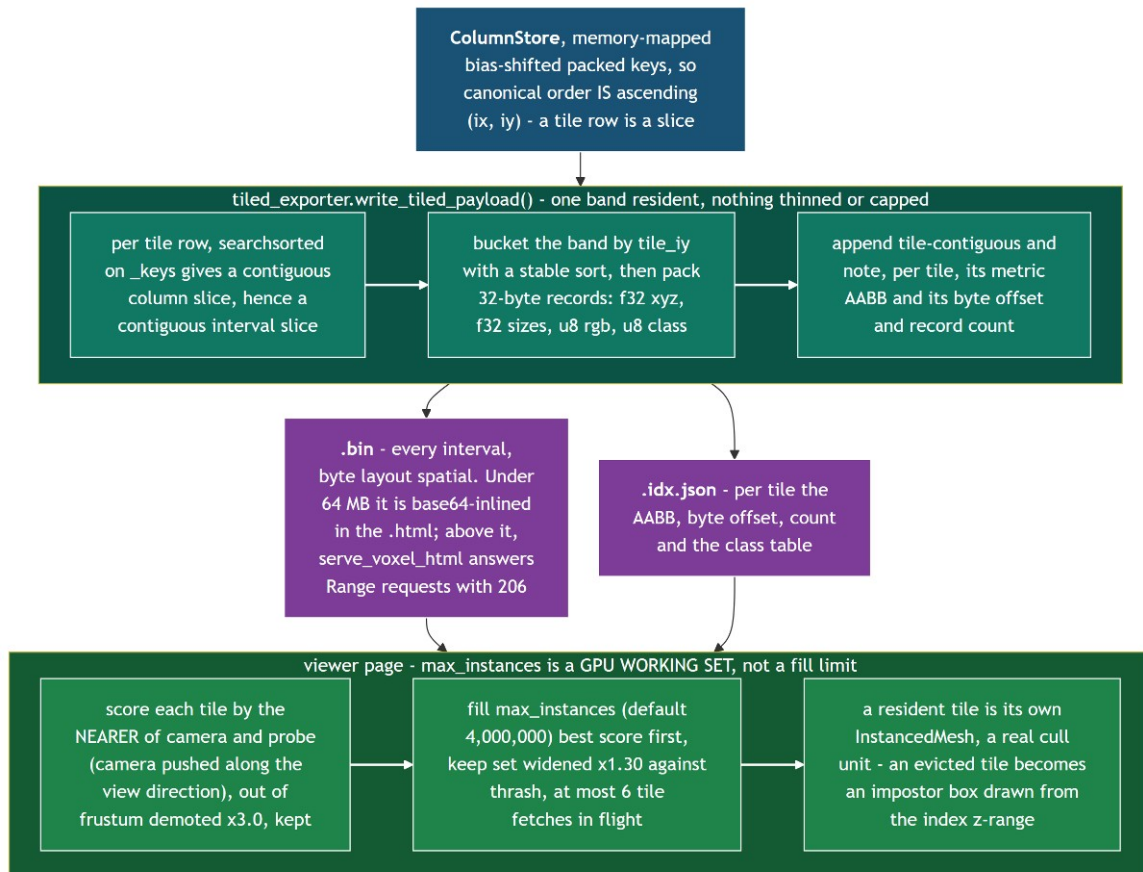


Figure 19: The streaming export path: `write_tiled_payload` traverses the structure one tile row at a time. Peak memory is a band's, not the payload's, and the viewer treats `max_instances` as a GPU working set – tiles near the camera stay resident at full detail, the others collapse into simplified silhouettes.

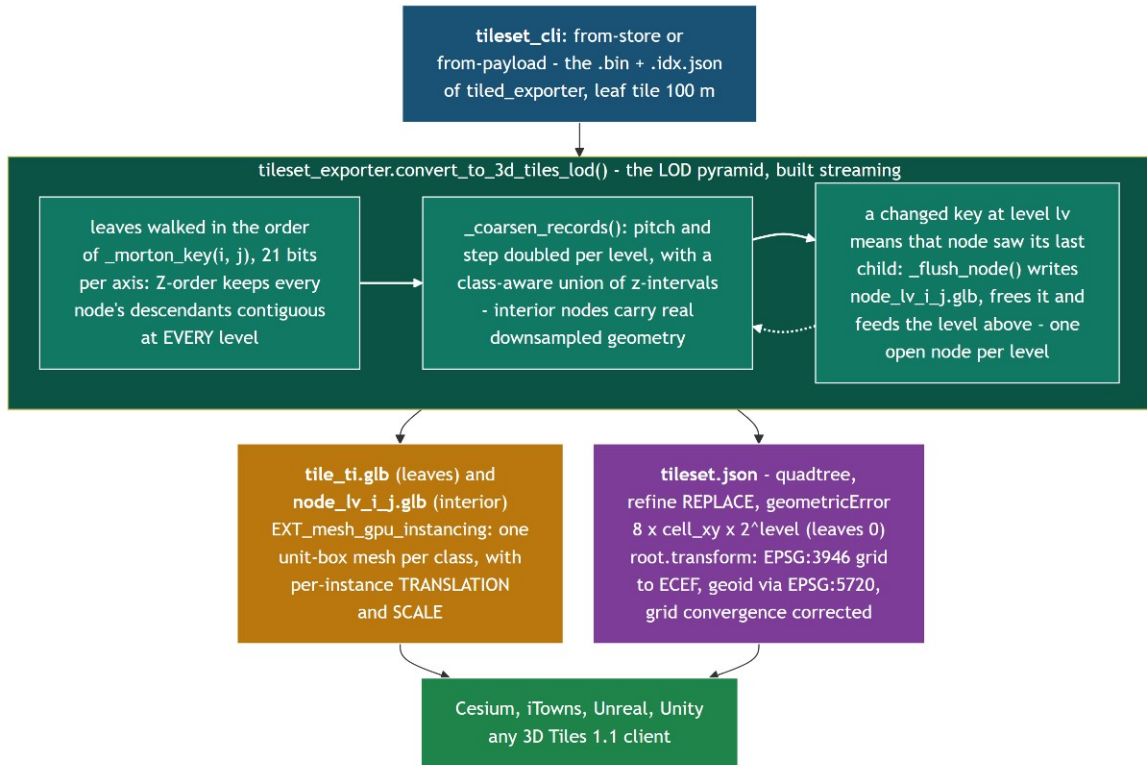


Figure 20: The 3D Tiles export path, which consumes the streaming payload rather than the structure: traversing the leaves in Morton order lets convert\_to\_3d\_tiles\_lod write each internal node as soon as its last child arrives, hence a memory peak that follows the depth of the tree and not the extent of the area.

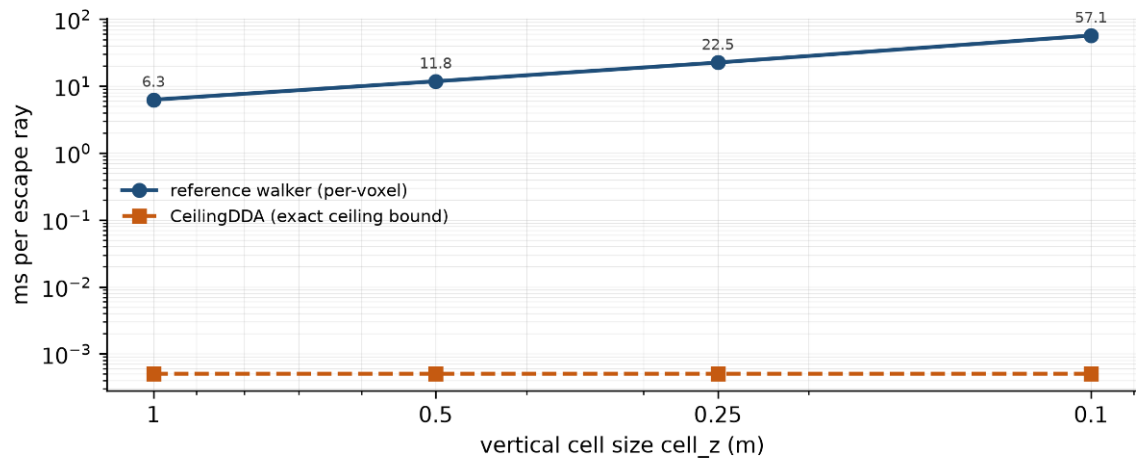


Figure 21: Cost of a climbing escape ray as a function of vertical cell size, reference walker against ceiling-bounded walker (logarithmic scale; times taken on a single machine, orders of magnitude; data Experiments/ray\_benchmark.json).

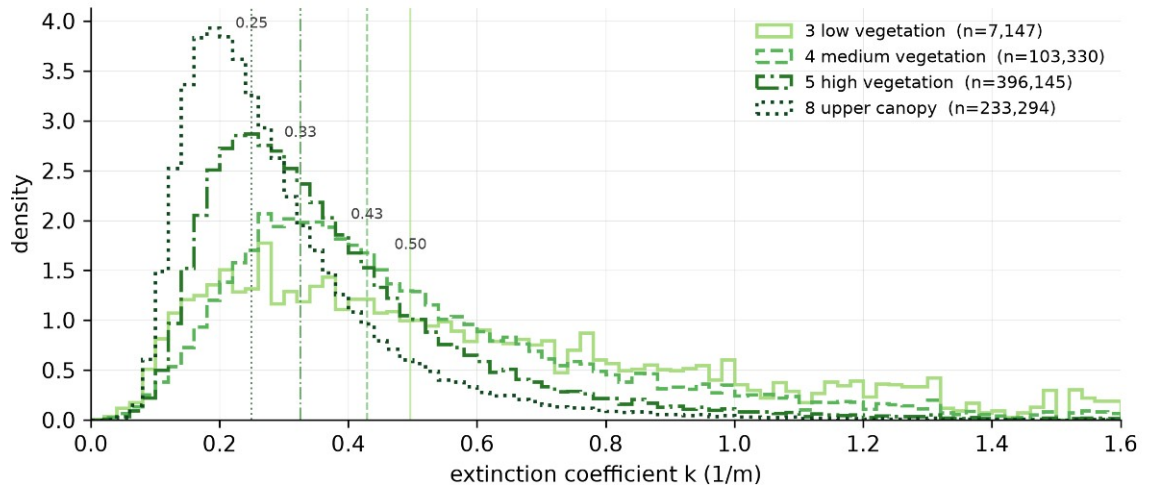


Figure 22: Distribution of the extinction coefficient per dominant vegetation class on the 0.5 m structure, class medians marked (739,916 columns after the degeneracy guard; the derivation of section 4.4.5).

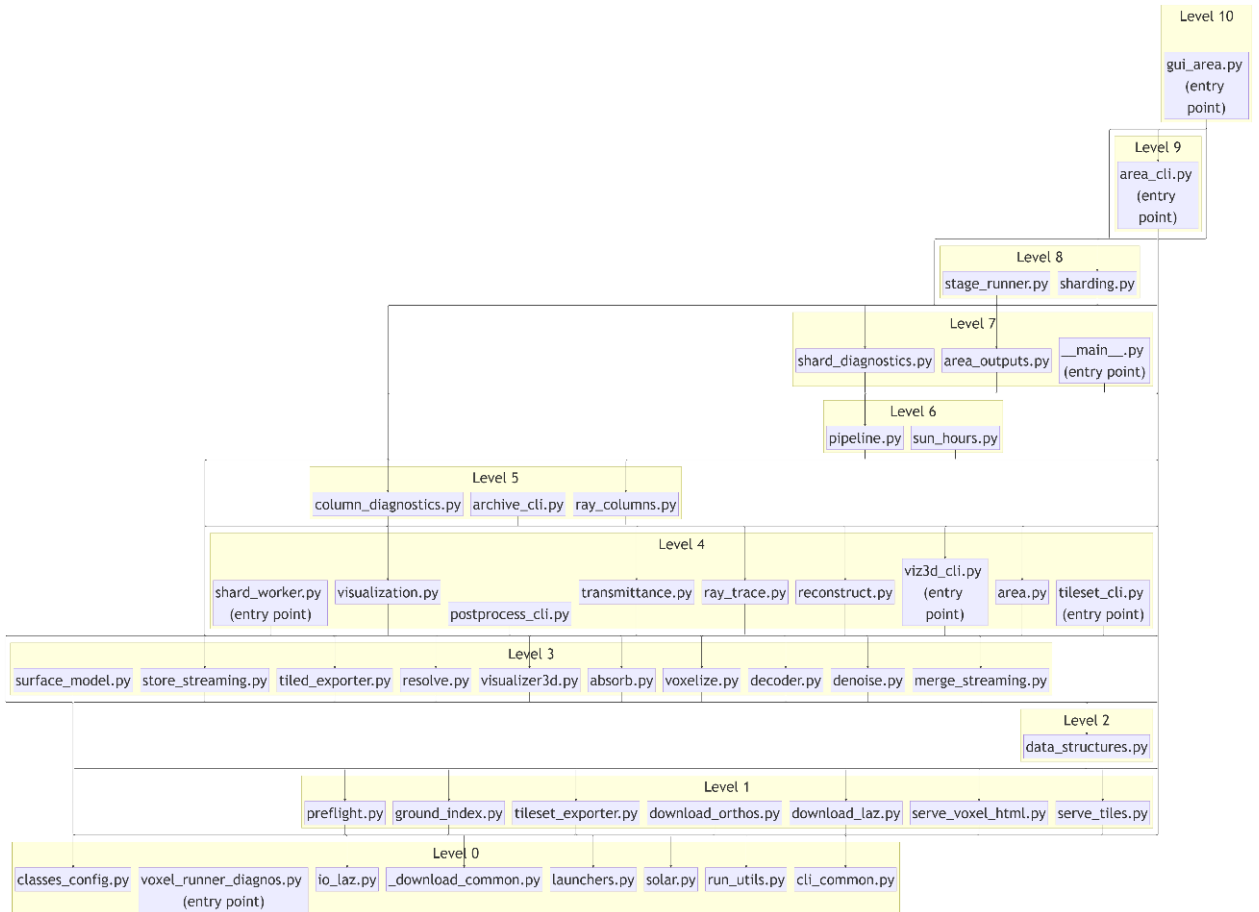


Figure 23: The complete module import graph of the voxelizer package – the forty-seven modules importing one another, over eleven dependency levels; the package initialiser is not drawn. Each arrow goes from a module to a module it imports. The entry points stand at the top and the level-0 leaves at the bottom.

## Appendix E - IGN HD LiDAR classification codes

The classification codes carry the input's semantics. The delivered tiles are LAS 1.2, point record format number 1 (LAS 1.4 numbers the point record formats 0 to 10 [7]; number 1 is the base format augmented with a GPS time field [8]), verified over all 2,842 files, and the classification codes actually present run from 1 to 9 only: unclassified (1), ground (2), vegetation split by height above ground into low (3, under 50 cm), medium (4, 50 cm to 1.5 m) and high (5, above 1.5 m), building (6), low-point noise (7), an upper canopy tier (8) and water (9).

One clarification is in order, because it is easy to get wrong: this project's code table is sized for LAS 1.4; it declares the standard ASPRS codes 0 to 22 (except 12, which the table omits) plus IGN's custom range 64 to 67 ("sursol pérenne", artefact, virtual point, probable unconfirmed building) – twenty-six in all. That is a property of the software, not of the data. Those custom codes can only exist in LAS 1.4's extended point formats 6 to 10; in point format number 1, the classification field is five bits, maximum value 31 [8]: codes 64 to 67 therefore cannot physically appear in this corpus, and none was observed in it.

The table below lists the classification codes this report refers to, with the names and remarks used throughout, drawn from the twenty-six the project's code table declares; the ten omitted appear nowhere in this corpus. Codes 0 to 22 belong to the standard ASPRS space; codes 64 to 67 are IGN's documented extensions in the user range. The wider table is kept so the same software ingests a LAS 1.4 delivery without modification.

| Code | Name                     | Remark                                                                                                                                                                                                                                             |
|------|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | created_never_classified | Default value of raw capture; should not survive quality control                                                                                                                                                                                   |
| 1    | unclassified             | Processed but not assigned (vehicles, street furniture, noise)                                                                                                                                                                                     |
| 2    | ground                   | Bare terrain surface, natural or artificial; anchors the ground index                                                                                                                                                                              |
| 3    | low_vegetation           | Vegetation under 50 cm (IGN's height split)                                                                                                                                                                                                        |
| 4    | medium_vegetation        | Vegetation between 50 cm and 1.5 m                                                                                                                                                                                                                 |
| 5    | high_vegetation          | Vegetation above 1.5 m: trees and canopy                                                                                                                                                                                                           |
| 6    | building                 | Roofs, façades, chimneys, balconies                                                                                                                                                                                                                |
| 7    | low_point_noise          | Low outlier points and spurious returns below the surface                                                                                                                                                                                          |
| 8    | high_vegetation_upper    | Model Key-point (mass point) in the class table of point formats 0-5, Reserved only in LAS 1.4's extended table for formats 6-10; de facto upper canopy tier in the Grand Lyon deliveries – 3.32 billion points, 6.4%, in 2,691 of the 2,842 tiles |
| 9    | water                    | Rivers, lakes, ponds                                                                                                                                                                                                                               |
| 11   | road_surface             | Roadway surfaces (declared; not present in this delivery's tiles)                                                                                                                                                                                  |
| 17   | bridge_deck              | Bridge deck only; piers and parapets end up unclassified (declared; absent from this delivery)                                                                                                                                                     |
| 64   | perennial_above_ground   | IGN: durable structures such as pylons, power lines, wind turbines, antennas                                                                                                                                                                       |
| 65   | artefact                 | IGN: unexplained or spurious points                                                                                                                                                                                                                |
| 66   | virtual_point            | IGN: synthetic points inserted under bridges for terrain modelling                                                                                                                                                                                 |
| 67   | misc_building_like       | IGN: building-like objects not confirmed by the reference databases                                                                                                                                                                                |

## Appendix F - Glossary

**3D Tiles:** an OGC standard for streaming large 3-D geospatial datasets; this work's export target for interoperability.

**ASPRS:** American Society for Photogrammetry and Remote Sensing; publisher of the LAS format and its standard classification codes.

**CC46 (RGF93/CC46, EPSG:3946):** the conformal conic projection zone covering Lyon, in the French geodetic system RGF93; the coordinate reference system of all input tiles.

**ColumnStore:** this project's central data structure; a sparse set of columns, each carrying run-compressed semantic intervals, laid out in flat numpy arrays.

**CRS:** Coordinate Reference System; the frame that gives coordinates a position on Earth.

**DDA:** Digital Differential Analyzer; the family of grid-traversal algorithms the ray walker uses to visit columns along a ray.

**IGN:** Institut national de l'information géographique et forestière; France's national mapping agency and producer of the HD LiDAR.

**Jaccard index [46]:** overlap of two sets (intersection over union); used per class to measure how much of the raw occupancy an encoder preserves.

**Ground index:** the per-column voxel index of the terrain surface, taken on the highest ground-classified interval; it anchors the empty-space decoder.

**Interval:** a maximal vertical run of same-class voxels in a column, stored as (z\_start, z\_end, class, count) with z\_end exclusive.

**Gap / void:** a gap is the vertical space between two occupied intervals of one column, which gap filling (variant v2) and interval grouping can close; void names the state of an unoccupied voxel as the decoder classifies it (measured air, opaque interior or underground). The report keeps the two terms distinct.

**LAS / LAZ:** the ASPRS point-cloud exchange format and its LASzip-compressed variant; this work's input format.

**HD LiDAR:** France's national high-density airborne laser survey by IGN, delivered as classified point clouds.

**Nadir:** the straight-down viewing direction of an airborne sensor; the nadir-beam assumption treats each column as if it had been scanned vertically.

**NODATA:** sentinel value marking a column with no ground return; those columns default to opaque interior in the decoder.

**O(1), O(n):** asymptotic cost notation, describing how an operation's cost grows with its input size n; O(1) is a constant cost, independent of size, O(n) one proportional to it.

**Octree:** a tree structure that recursively subdivides 3-D space into eight octants; the classic sparse-voxel representation, excluded here by the project's framing note.

**OpenVDB:** the open-source sparse volumetric data structure developed at DreamWorks, the de facto standard for volumetric data in visual effects; NanoVDB is its lightweight counterpart for GPUs and out-of-core streaming.

**Orthophoto:** a georeferenced, distortion-corrected aerial photograph; used in this work only as a display backdrop, never as a computation input.

Out-of-core: processing data larger than memory by streaming bounded chunks; the property of the partitioned area pipeline.

Sharding: cutting an area run into independent per-tile structures so peak memory stays bounded by a single tile.

Ray walker: the traversal algorithm that advances voxel by voxel along a ray (DDA family); distinct from the ray caster, the query layer that employs it.

RLE: Run-Length Encoding; compressing a sequence by storing each run of identical values once with its length.

SBRT: Stack-Based Representation of Terrains; the layered material-stack terrain model of the Arches line of work, conceptual ancestor of the column structure.

F1 score: the harmonic mean of precision and recall, used as a single-figure summary of classification quality; 1 is perfect agreement.

Structure-of-arrays: a memory layout holding one flat array per field instead of one object per record; the ColumnStore's layout.

Top-class disagreement rate: the percentage of columns whose topmost class differs between the raw and encoded structures. It counts how often the encoded top surface disagrees with the raw one; it is a rate, not a distance.

Voxel: volume element; the 3-D analogue of a pixel, the atomic cell of a regular volumetric grid.